

COMPLETE TECHNICAL DOCUMENTATION

Shop Sathi E-Commerce

Full Stack E-Commerce Platform — End-to-End System
Architecture, Schemas, API Reference & Deployment
Specifications

Platform Stack

**Node.js | Express | Next.js | React |
Tailwind | PostgreSQL / Prisma**

Documentation Version

v1.0.0 (Master Release)

Author / Team

Shop Sathi Engineering Team

Generated Date

August 2026



Table of Contents

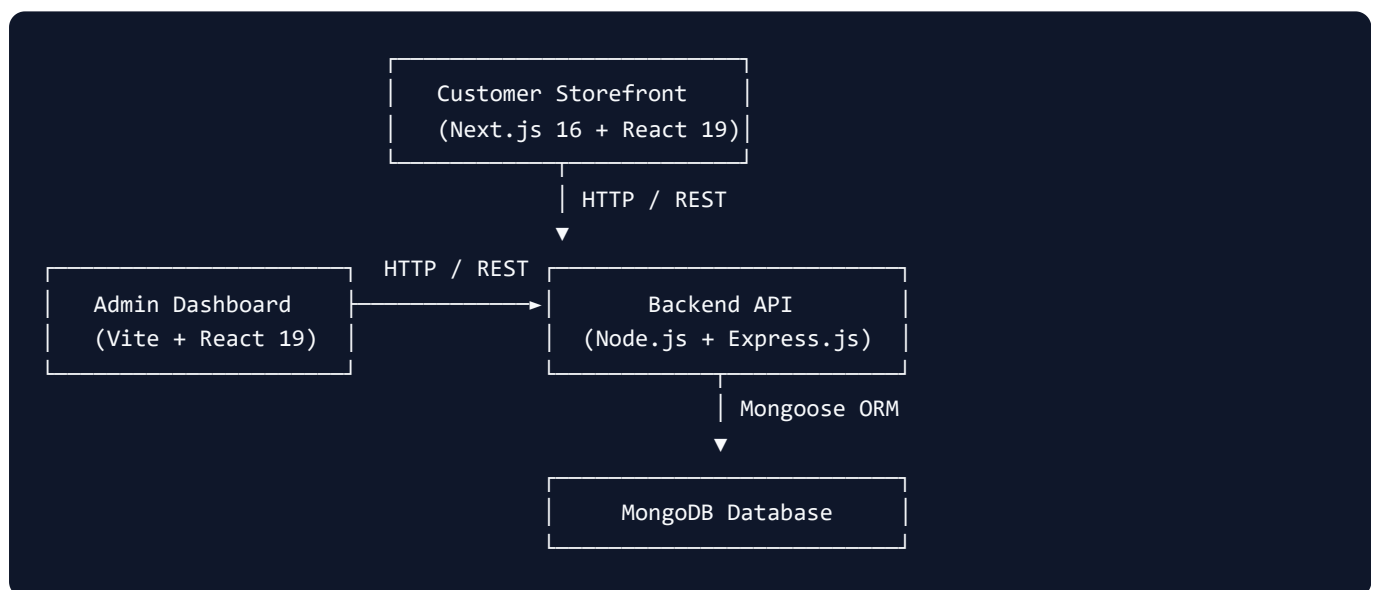
- [Section 1: Executive Overview](#)
- [Section 2: Phase 1: Product Requirements Document \(PRD\)](#)
- [Section 3: Phase 2: Technical Requirements Document \(TRD\)](#)
- [Section 4: Phase 3: Application Flow & Architecture](#)
- [Section 5: Phase 4: UI/UX Brief & Design System](#)
- [Section 6: Phase 5: Database Schema & Data Models](#)
- [Section 7: Phase 6: API Documentation & Endpoints](#)
- [Section 8: Phase 7: Implementation Plan & Roadmap](#)
- [Section 9: Phase 8: Deployment Guide & Infrastructure](#)
- [Section 10: Phase 9: Maintenance & Operations Guide](#)
- [Section 11: Product Catalog & Taxonomy Reference](#)

Section 1: Executive Overview

Shop Sathi - Production-Ready E-Commerce Platform

Welcome to **Shop Sathi**, an end-to-end, production-grade E-Commerce platform engineered for high performance, security, and scalability. This repository is structured as a monorepo containing the **Customer Storefront**, **Admin Dashboard**, **Backend REST API**, and exhaustive **Technical Documentation**.

Architecture Overview



Tech Stack & Technologies

1. Storefront (/website)

- **Framework:** Next.js 16 (App Router) + React 19
- **Styling:** Tailwind CSS v4 + PostCSS
- **State Management:** Zustand

- **Form Validation:** React Hook Form + Zod
- **Animations:** Framer Motion
- **Internationalization:** i18next + React i18next
- **Icons & UI:** Lucide React, Swiper, React Hot Toast
- **Deployment Target:** Netlify

2. 🏠 Admin Dashboard (/dashboard)

- **Build Tool & Framework:** Vite 8 + React 19
- **Routing:** React Router DOM v7
- **State Management:** Redux Toolkit + React-Redux
- **Styling:** Tailwind CSS v4 + PostCSS
- **Data Visualization:** Recharts
- **Data Tables:** TanStack Table v8
- **Form Validation:** React Hook Form + Zod
- **Deployment Target:** Netlify

3. ⚙️ Backend API (/backend)

- **Runtime & Framework:** Node.js (>=20.0.0) + Express.js v4
- **Database & ORM:** MongoDB + Mongoose v8
- **Authentication:** JWT (JSON Web Tokens) + Cookie Parser + Bcrypt.js
- **Payment Gateway Integration:** Razorpay SDK
- **Communication Services:** Nodemailer (Email) + Twilio (SMS/OTP)
- **File Uploads:** Multer
- **Security & Optimization:** Helmet, CORS, Express Rate Limit, Compression, Morgan Logging

📁 Repository Structure

```

shop-sathi-e-commerce/
├── README.md # Main Project Documentation
├── backend/ # Express & Node.js REST API
│   ├── scripts/ # Utility & bulk image update scripts
│   ├── src/ # Controllers, Models, Routes, Middlewares, Services
│   ├── server.js # Server entry point
│   ├── seed.js # Database seed script
│   └── package.json # Backend dependencies & npm scripts
├── dashboard/ # Admin Dashboard Application (Vite + React)
│   ├── public/ # Public assets
│   ├── src/ # Admin pages, components, redux slices, routes
│   ├── vite.config.js # Vite configuration
│   └── package.json # Dashboard dependencies & npm scripts
├── website/ # Storefront Application (Next.js 16)
│   ├── public/ # Static assets & images
│   ├── src/ # Next.js pages, components, stores, hooks
│   ├── next.config.mjs # Next.js configuration
│   └── package.json # Storefront dependencies & npm scripts
└── documentation/ # Complete Project Specification Guides
    ├── phase-1-PRD.md # Product Requirements Document
    ├── phase-2-TRD.md # Technical Requirements Document
    ├── phase-3-AppFlow.md # User Flow & Navigation Specs
    ├── phase-4-UIUXBrief.md # Design System & UI/UX Guidelines
    ├── phase-5-Schema.md # Database Schema Specifications
    ├── phase-6-APIDocs.md # Complete API Endpoint Documentation
    ├── phase-7-ImplementationPlan.md # Step-by-Step Build Roadmap
    ├── phase-8-DeploymentGuide.md # Production Deployment Guide
    ├── phase-9-MaintenanceGuide.md # Maintenance & Operations
    └── prodecats-categories.md # Product Catalog Specifications

```

🌟 Key Features

Customer Storefront (/website)

- **Product Catalog & Discovery:** Category hierarchy, dynamic search, multi-faceted filtering, sorting, and pagination.
- **Product Details:** Variant selections (size, color), image galleries, customer reviews, rating breakdowns, and stock availability.
- **Shopping Cart & Checkout:** Interactive cart drawer, persistent session cart, discount code application, and Razorpay payment processing.
- **User Accounts:** Authentication, order history tracking, saved addresses, wishlist management, and profile settings.
- **Multilingual & Responsive:** Multi-language translation support (i18n) and 100% responsive design across all viewports.

Admin Dashboard (/dashboard)

- **Analytics Overview:** Real-time sales metrics, revenue analytics, top-selling product charts, and recent order feeds.
- **Product Management:** Full CRUD operations for products, categories, dynamic inventory tracking, and variant handling.
- **Order Fulfillment:** Order processing workflow (Pending → Processing → Shipped → Delivered → Cancelled), invoice generation, and status updates.
- **Customer Management:** User account listing, order history, activity logs, and account status management.

Backend API (`/backend`)

- **Role-Based Access Control (RBAC):** Fine-grained permissions for Admin, Manager, and Customer roles.
- **Secure Authentication:** HTTP-only cookie JWT delivery, refresh tokens, and password hashing with bcrypt.
- **Payment Processing:** Verification signatures for Razorpay webhooks and payment flow integration.
- **Automated Communication:** Transactional emails (order confirmation, password reset) via Nodemailer & OTP validation via Twilio.
- **Production Hardened:** Rate limiting protection, security headers via Helmet, payload compression, and centralized error handling.

Quick Start Guide

Prerequisites

Make sure you have the following installed on your environment:

- **Node.js:** `v20.0.0` or higher
- **npm:** `v10.0.0` or higher
- **MongoDB:** Local instance running on `mongodb://localhost:27017` or MongoDB Atlas URI

1. Setup & Start Backend

```
# Navigate to the backend directory
cd backend

# Install dependencies
npm install

# Setup environment variables (copy example file)
cp .env.example .env

# Seed initial database data (categories, products, admin account)
npm run seed

# Start development server
npm run dev
```

The backend server runs at `http://localhost:5000` by default.

2. Setup & Start Storefront Website

```
# Navigate to the website directory
cd website

# Install dependencies
npm install

# Start Next.js development server
npm run dev
```

The customer storefront runs at `http://localhost:3000` by default.

3. Setup & Start Admin Dashboard

```
# Navigate to the dashboard directory
cd dashboard

# Install dependencies
npm install

# Start Vite development server
npm run dev
```

The admin dashboard runs at `http://localhost:5173` by default.

Database Seeding & Utility Scripts

The backend includes pre-configured utility scripts for populating seed data and updating product catalogs:

Command	Description
<code>npm run seed</code>	Seeds default admin user, initial product categories, and sample products.
<code>npm run update-images</code>	Updates product image URLs across all existing products in MongoDB.
<code>npm run bulk-update-images</code>	Executes bulk update for product media assets.
<code>npm run update-pet-supplies</code>	Updates pet supplies category catalog.
<code>npm run update-mobile-accessories</code>	Updates mobile accessories category catalog.
<code>npm run update-toys-games</code>	Updates toys and games category catalog.

Complete Documentation Index

For detailed specifications and architectural deep dives, check out the documentation files in the `documentation/` folder:

-  [Phase 1: Product Requirements Document \(PRD\)](#)
-  [Phase 2: Technical Requirements Document \(TRD\)](#)
-  [Phase 3: App Flow & Navigation Specs](#)
-  [Phase 4: UI/UX Brief & Design System](#)
-  [Phase 5: Database Schema Specifications](#)
-  [Phase 6: Complete API Documentation](#)
-  [Phase 7: Step-by-Step Implementation Roadmap](#)
-  [Phase 8: Production Deployment Guide](#)
-  [Phase 9: Maintenance & Security Operations](#)

Contributing & License

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/AmazingFeature`)
3. Commit your changes (`git commit -m 'Add some AmazingFeature'`)

4. Push to the branch (`git push origin feature/AmazingFeature`)
 5. Open a Pull Request
-

Developed with ❤️ for **Shop Sathi E-Commerce Platform**.

Section 2: Phase 1: Product Requirements Document (PRD)

Product Requirements Document (PRD)

Project: Production-Ready E-Commerce Platform

Version: 1.0

Status: Planning

Date: July 2026

Phase: 1 of 9

Table of Contents

1. [Executive Summary](#)
2. [Product Overview](#)
3. [Target Users & Personas](#)
4. [Functional Requirements](#)
 - 4.1 [Client Website \(Next.js\)](#)
 - 4.2 [Admin Dashboard \(React.js\)](#)
 - 4.3 [Backend API \(Node.js\)](#)
5. [Non-Functional Requirements](#)
6. [User Stories](#)
7. [Feature Prioritization](#)
8. [Out of Scope \(v1.0\)](#)
9. [Acceptance Criteria](#)
10. [Glossary](#)

1. Executive Summary

This document defines the product requirements for a **Production-Ready E-Commerce Platform** consisting of three primary applications:

- **Client Website** — A customer-facing storefront built with Next.js for browsing, searching, and purchasing products.
- **Admin Dashboard** — An internal management interface built with React.js for managing products, orders, customers, and analytics.
- **Backend API** — A RESTful API built with Node.js + Express.js + MongoDB that powers both frontend applications.

The platform is designed to be scalable, secure, SEO-optimized, and maintainable for long-term business growth.

2. Product Overview

2.1 Problem Statement

Businesses need an online presence to sell products. Existing off-the-shelf solutions are either too expensive, too limited, or too difficult to customize. This platform provides a fully custom, production-grade solution built from the ground up with full ownership of the codebase.

2.2 Solution

A custom-built, three-tier e-commerce platform where:

- **Customers** can browse products, manage their cart, and place orders.
- **Administrators** can manage every aspect of the business from a dedicated dashboard.
- **The Backend** securely handles all data, authentication, file storage, and business logic.

2.3 Success Metrics

Metric	Target
Page Load Time (LCP)	< 2.5 seconds
Mobile Responsiveness	100% responsive on all screen sizes
API Response Time	< 200ms for standard requests
Authentication Security	JWT + HTTP-only cookies
SEO Score (Lighthouse)	≥ 90
Bug Rate (post-launch)	< 5 critical bugs in first month
Uptime	99.9%

3. Target Users & Personas

3.1 Customer (End User)

Attribute	Details
Who	General online shoppers
Age Range	18 – 55
Device	Mobile (primary), Desktop (secondary)
Goals	Browse products, find deals, checkout easily, track orders
Pain Points	Slow websites, complicated checkout, no order tracking
Tech Comfort	Low to Medium

3.2 Administrator

Attribute	Details
Who	Store owners, managers, staff
Age Range	25 – 50
Device	Desktop (primary)
Goals	Manage inventory, process orders, view analytics, manage customers
Pain Points	Complex UIs, no real-time updates, poor reporting
Tech Comfort	Medium

3.3 Super Admin

Attribute	Details
Who	Platform owner / developer
Goals	Full system access, role management, audit logs
Tech Comfort	High

4. Functional Requirements

4.1 Client Website (Next.js)

4.1.1 Home Page

- [] Display hero banner / promotional slider
- [] Display featured product categories
- [] Display featured / trending products (grid)
- [] Display promotional banners / offers
- [] Display recently viewed products
- [] Display newsletter subscription form

4.1.2 Product Listing Page

- [] Display all products in a responsive grid
- [] Filter products by: Category, Price Range, Brand, Rating, Availability
- [] Sort products by: Price (Low-High / High-Low), Newest, Best Selling, Rating
- [] Pagination or infinite scroll
- [] Display product count and active filters
- [] Clear all filters option

4.1.3 Product Detail Page

- [] Display product images (gallery with zoom)
- [] Display product name, description, price, original price, discount
- [] Display product variants (size, color, etc.)
- [] Quantity selector
- [] Add to Cart button
- [] Add to Wishlist button
- [] Display stock status (In Stock / Out of Stock)
- [] Product specifications / details table
- [] Customer reviews & ratings section
- [] Related products section
- [] Share product (social media links)

4.1.4 Search

- [] Real-time search bar (with debounce)
- [] Search suggestions / autocomplete
- [] Search results page with filters
- [] No results state with suggestions

4.1.5 Shopping Cart

- ☐ View all cart items
- ☐ Update item quantity
- ☐ Remove item from cart
- ☐ Display subtotal, taxes, shipping estimate
- ☐ Apply coupon / promo code
- ☐ Proceed to checkout button
- ☐ Cart persists across sessions (logged in user)
- ☐ Guest cart support

4.1.6 Checkout

- ☐ Shipping address form (add/select saved address)
- ☐ Order summary review
- ☐ Payment method selection (COD, Online Payment)
- ☐ Place order confirmation
- ☐ Send order confirmation email
- ☐ Redirect to order success page

4.1.7 User Authentication

- ☐ Register with email & password
- ☐ Login with email & password
- ☐ Logout
- ☐ Forgot password (email reset link)
- ☐ Reset password
- ☐ JWT token management (HTTP-only cookies)
- ☐ Protected routes for authenticated pages

4.1.8 User Profile

- ☐ View and edit profile (name, email, phone, avatar)
- ☐ Manage saved addresses (add, edit, delete)
- ☐ Change password
- ☐ View order history
- ☐ View single order details with status
- ☐ Cancel order (if eligible)
- ☐ View wishlist

4.1.9 Wishlist

- ☐ Add/remove products to wishlist

- ☐ View all wishlisted items
- ☐ Move item from wishlist to cart
- ☐ Wishlist persists for logged-in users

4.1.10 Order Tracking

- ☐ View order status (Pending → Processing → Shipped → Delivered)
- ☐ Order timeline / progress indicator
- ☐ Estimated delivery date

4.1.11 Reviews & Ratings

- ☐ Submit a review (only for purchased products)
 - ☐ Star rating (1–5)
 - ☐ Edit / delete own review
 - ☐ Display average rating and review count
-

4.2 Admin Dashboard (React.js)

4.2.1 Authentication

- ☐ Admin login (email & password)
- ☐ Role-based access (Admin, Super Admin)
- ☐ Logout
- ☐ Protected dashboard routes

4.2.2 Dashboard Overview

- ☐ Total revenue (today, this week, this month)
- ☐ Total orders count with status breakdown
- ☐ Total products count
- ☐ Total customers count
- ☐ Recent orders list
- ☐ Revenue chart (line/bar graph)
- ☐ Top selling products
- ☐ Low stock alerts

4.2.3 Product Management

- ☐ List all products (with search, filter, sort)
- ☐ Add new product (name, description, price, images, category, stock, variants)
- ☐ Edit existing product
- ☐ Delete product (soft delete)

- [] Upload product images to Cloudinary
- [] Manage product categories (CRUD)
- [] Bulk product actions (delete, status change)
- [] Product status toggle (Active / Inactive)

4.2.4 Order Management

- [] List all orders (with filters: status, date range, customer)
- [] View single order details
- [] Update order status (Pending → Processing → Shipped → Delivered → Cancelled)
- [] View customer details for each order
- [] Generate / view invoice
- [] Export orders to CSV

4.2.5 Customer Management

- [] List all customers
- [] View customer profile and order history
- [] Block / unblock customer
- [] Search and filter customers
- [] Export customer list

4.2.6 Category Management

- [] List all categories
- [] Add new category (name, description, image)
- [] Edit category
- [] Delete category
- [] Nested sub-categories support

4.2.7 Coupon Management

- [] Create coupon (code, discount type, value, expiry, usage limit)
- [] List all coupons
- [] Enable / disable coupon
- [] Delete coupon
- [] View coupon usage statistics

4.2.8 Analytics & Reports

- [] Revenue report (daily, weekly, monthly, yearly)
- [] Orders report
- [] Top products by sales
- [] Customer acquisition report

- [] Export reports to CSV / PDF

4.2.9 Settings

- [] Store settings (name, logo, address, contact)
 - [] Email template settings
 - [] Shipping settings (zones, rates)
 - [] Tax settings
 - [] Payment method configuration
 - [] Admin profile management
 - [] Admin user management (Super Admin only)
-

4.3 Backend API (Node.js + Express.js)

4.3.1 Authentication Module

- [] POST /api/auth/register
- [] POST /api/auth/login
- [] POST /api/auth/logout
- [] POST /api/auth/forgot-password
- [] POST /api/auth/reset-password
- [] GET /api/auth/me (get logged-in user)
- [] POST /api/auth/refresh-token

4.3.2 User Module

- [] GET /api/users (Admin: list all)
- [] GET /api/users/:id
- [] PUT /api/users/:id (update profile)
- [] DELETE /api/users/:id (Admin)
- [] PUT /api/users/:id/block (Admin)

4.3.3 Product Module

- [] GET /api/products (with filters, pagination)
- [] GET /api/products/:id
- [] GET /api/products/featured
- [] POST /api/products (Admin)
- [] PUT /api/products/:id (Admin)
- [] DELETE /api/products/:id (Admin)
- [] POST /api/products/:id/images (Cloudinary upload)

4.3.4 Category Module

- [] GET /api/categories
- [] GET /api/categories/:id
- [] POST /api/categories (Admin)
- [] PUT /api/categories/:id (Admin)
- [] DELETE /api/categories/:id (Admin)

4.3.5 Cart Module

- [] GET /api/cart
- [] POST /api/cart/add
- [] PUT /api/cart/update
- [] DELETE /api/cart/remove/:productId
- [] DELETE /api/cart/clear

4.3.6 Wishlist Module

- [] GET /api/wishlist
- [] POST /api/wishlist/add
- [] DELETE /api/wishlist/remove/:productId

4.3.7 Order Module

- [] POST /api/orders (place order)
- [] GET /api/orders (Admin: all orders)
- [] GET /api/orders/my-orders (Customer)
- [] GET /api/orders/:id
- [] PUT /api/orders/:id/status (Admin)
- [] PUT /api/orders/:id/cancel (Customer)

4.3.8 Review Module

- [] GET /api/reviews/:productId
- [] POST /api/reviews/:productId
- [] PUT /api/reviews/:reviewId
- [] DELETE /api/reviews/:reviewId

4.3.9 Coupon Module

- [] GET /api/coupons (Admin)
- [] POST /api/coupons (Admin)
- [] POST /api/coupons/validate
- [] PUT /api/coupons/:id (Admin)

- [] DELETE /api/coupons/:id (Admin)

4.3.10 Upload Module

- [] POST /api/upload/image (Cloudinary)
- [] DELETE /api/upload/image/:publicId

4.3.11 Analytics Module (Admin)

- [] GET /api/analytics/dashboard
- [] GET /api/analytics/revenue
- [] GET /api/analytics/orders
- [] GET /api/analytics/top-products

5. Non-Functional Requirements

5.1 Performance

Requirement	Target
Client Website LCP	< 2.5s
Time to First Byte (TTFB)	< 200ms
API response time (avg)	< 200ms
Image optimization	WebP format, lazy loading
Bundle size	Code splitting per route

5.2 Security

Requirement	Implementation
Authentication	JWT with HTTP-only cookies
Password storage	bcrypt (salt rounds ≥ 12)
Input validation	Server-side validation on all endpoints
Rate limiting	100 req/min per IP
CORS	Whitelist allowed origins only
XSS Protection	Helmet.js middleware
SQL/NoSQL Injection	Mongoose sanitization
HTTPS	SSL certificate (Let's Encrypt)

5.3 Scalability

- Stateless API architecture (horizontal scaling ready)
- MongoDB indexing on frequently queried fields
- Image storage on Cloudinary (CDN-backed)
- Environment-based configuration

5.4 Usability

- Fully responsive: mobile, tablet, desktop
- Accessible: WCAG 2.1 AA compliance
- Loading states for all async operations
- Error messages in user-friendly language
- Empty states for all list views

5.5 Maintainability

- Modular folder structure
- Consistent code style (ESLint + Prettier)
- Environment variables for all secrets
- Clear README for each app
- Git branching strategy (main / develop / feature branches)

5.6 SEO (Client Website)

- Next.js SSR / SSG for product and category pages
- Dynamic `<title>` and `<meta description>` per page

- Open Graph tags
- Structured data (JSON-LD for products)
- Sitemap.xml auto-generation
- robots.txt

6. User Stories

Customer Stories

ID	As a...	I want to...	So that...
US-01	Customer	Browse products by category	I can find what I'm looking for
US-02	Customer	Search for a specific product	I can find it quickly
US-03	Customer	Filter and sort products	I can narrow down choices
US-04	Customer	View product details and images	I can make an informed decision
US-05	Customer	Add products to my cart	I can buy multiple items at once
US-06	Customer	Save products to wishlist	I can buy them later
US-07	Customer	Checkout securely	I can complete my purchase
US-08	Customer	Track my order status	I know when it will arrive
US-09	Customer	Create and manage my account	I can save my details
US-10	Customer	Write a review	I can share my experience
US-11	Customer	Use a coupon code	I can get a discount
US-12	Customer	View my order history	I can see past purchases

Admin Stories

ID	As an...	I want to...	So that...
US-13	Admin	Add / edit / delete products	I can manage the catalog
US-14	Admin	View all orders and update status	I can process orders efficiently
US-15	Admin	View analytics dashboard	I can monitor business performance
US-16	Admin	Manage customer accounts	I can help customers
US-17	Admin	Create coupon codes	I can run promotions
US-18	Admin	View low stock alerts	I can restock before items run out
US-19	Admin	Export order / customer data	I can generate reports
US-20	Admin	Manage product categories	I can organize the catalog

7. Feature Prioritization

MoSCoW Matrix

Must Have (v1.0)

- User authentication (register, login, logout, reset password)
- Product listing, filtering, search
- Product detail page
- Shopping cart
- Checkout (COD)
- Order placement & email confirmation
- Order tracking (status)
- Admin: product CRUD
- Admin: order management
- Admin: dashboard overview
- Cloudinary image upload
- JWT authentication for admin

Should Have (v1.0)

- Wishlist
- Customer reviews & ratings
- Coupon codes

- Admin: customer management
- Admin: category management
- Admin: basic analytics
- User profile management
- Saved addresses

Could Have (v1.0)

- Online payment gateway (Razorpay / Stripe)
- Product variants (size, color)
- Admin: export to CSV
- Admin: report charts
- Social share on product pages

Won't Have (v1.0) — Future Releases

- Multi-vendor marketplace
- Mobile app
- Loyalty rewards system
- AI product recommendations
- Live chat support
- Multi-language support
- Multi-currency support
- Subscription products
- Marketing automation

8. Out of Scope (v1.0)

The following features are explicitly **not included** in version 1.0:

- Payment gateway integration (COD only in v1.0; online payment in v1.1)
- Mobile application (iOS / Android)
- Multi-vendor / seller support
- Multi-language / multi-currency
- AI-based recommendations
- Live chat
- Loyalty / rewards program
- Social login (Google, Facebook)
- Advanced subscription model
- Marketing automation tools

9. Acceptance Criteria

Website

- [] User can register, login, and logout successfully
- [] User can browse products with working filters and search
- [] User can add/remove items from cart and wishlist
- [] User can complete checkout and receive confirmation email
- [] User can view order history and current order status
- [] All pages are fully responsive on mobile and desktop
- [] All pages score ≥ 90 on Lighthouse SEO

Admin Dashboard

- [] Admin can login and access protected routes only
- [] Admin can perform full CRUD on products and categories
- [] Admin can update order status with confirmation
- [] Admin dashboard displays accurate real-time statistics
- [] Admin can search and manage customers

Backend API

- [] All API endpoints return correct HTTP status codes
- [] JWT authentication is enforced on all protected routes
- [] Input validation rejects invalid data with clear error messages
- [] File uploads work correctly via Cloudinary
- [] Password reset email is delivered within 60 seconds

10. Glossary

Term	Definition
PRD	Product Requirements Document — this document
TRD	Technical Requirements Document — Phase 2
LCP	Largest Contentful Paint — a Core Web Vital metric
SSR	Server-Side Rendering — Next.js renders HTML on server
SSG	Static Site Generation — pages pre-built at build time
JWT	JSON Web Token — used for stateless authentication
CRUD	Create, Read, Update, Delete — standard data operations
COD	Cash on Delivery — payment method
MoSCoW	Must/Should/Could/Won't — feature prioritization method
CDN	Content Delivery Network — for fast asset delivery
CORS	Cross-Origin Resource Sharing — security policy

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial PRD created

Next Phase → [Phase 2: Technical Requirements Document \(TRD\)](#)

Section 3: Phase 2: Technical Requirements Document (TRD)

Technical Requirements Document (TRD)

Project: Production-Ready E-Commerce Platform

Version: 1.0

Status: Planning

Date: July 2026

Phase: 2 of 9

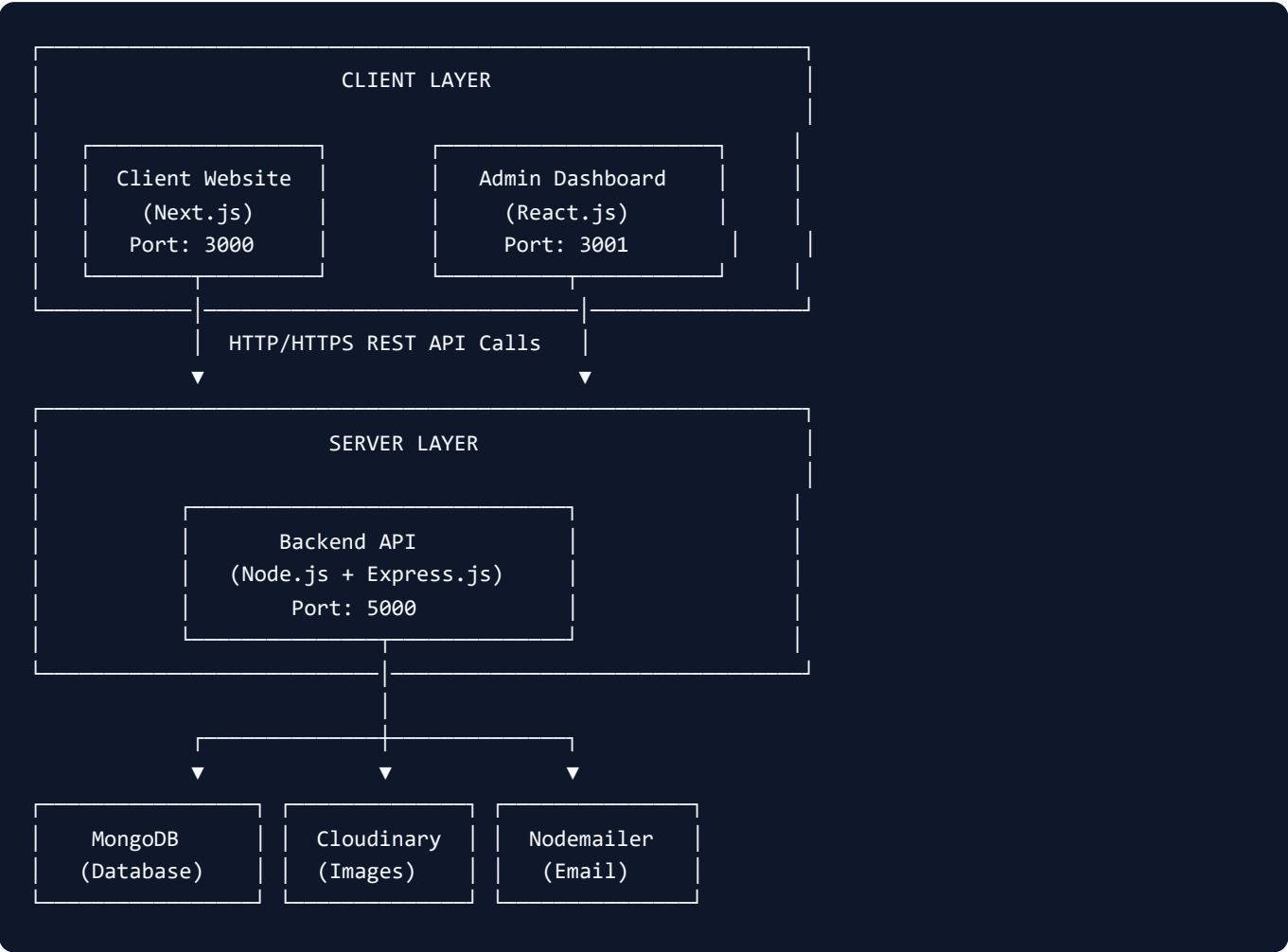
Table of Contents

- 1. System Architecture
- 2. Technology Stack — Detailed
- 3. Project Folder Structure
- 4. Database Design
- 5. API Design Standards
- 6. Authentication & Authorization
- 7. File Upload Strategy
- 8. Email Service
- 9. Error Handling Standards
- 10. Environment Configuration
- 11. Security Requirements
- 12. Performance Requirements
- 13. Deployment Architecture
- 14. Version Control Strategy
- 15. Coding Standards

1. System Architecture

1.1 Architecture Overview

The platform follows a **three-tier, decoupled architecture**:



1.2 Communication Pattern

From	To	Protocol	Format
Client Website	Backend API	HTTPS	JSON
Admin Dashboard	Backend API	HTTPS	JSON
Backend API	MongoDB	MongoDB Driver	BSON
Backend API	Cloudinary	HTTPS SDK	Multipart/JSON
Backend API	Email Service	SMTP	MIME

1.3 Architecture Principles

- **Separation of Concerns** — Frontend and Backend are fully independent
- **Stateless API** — No server-side session; JWT-based auth

- **Single Responsibility** — Each module handles one domain
- **DRY (Don't Repeat Yourself)** — Shared utilities and middleware
- **RESTful Design** — Standard HTTP verbs and status codes

2. Technology Stack — Detailed

2.1 Client Website (Next.js)

Technology	Version	Purpose
Next.js	14.x	React framework with SSR/SSG/ISR
React	18.x	UI component library
Tailwind CSS	3.x	Utility-first CSS framework
Zustand	4.x	Lightweight global state management
Axios	1.x	HTTP client for API calls
React Hook Form	7.x	Form handling and validation
Zod	3.x	Schema validation
next-seo	6.x	SEO meta tags management
Swiper.js	11.x	Product image slider
Lucide React	latest	Icon library

2.2 Admin Dashboard (React.js)

Technology	Version	Purpose
React	18.x	UI library
React Router DOM	6.x	Client-side routing
Redux Toolkit	2.x	Global state (auth, data)
RTK Query	2.x	Data fetching & caching
Tailwind CSS	3.x	Styling
React Hook Form	7.x	Form management
Recharts	2.x	Analytics charts
React Table (TanStack)	8.x	Data tables
Lucide React	latest	Icons
React Hot Toast	2.x	Notifications
date-fns	3.x	Date formatting

2.3 Backend API (Node.js)

Technology	Version	Purpose
Node.js	20.x LTS	Runtime environment
Express.js	4.x	Web framework
MongoDB	7.x	NoSQL database
Mongoose	8.x	MongoDB ODM
JSON Web Token	9.x	Authentication tokens
bcryptjs	2.x	Password hashing
Cloudinary SDK	2.x	Image upload & management
Nodemailer	6.x	Email sending
Multer	1.x	File upload middleware
Express Validator	7.x	Input validation
Helmet	7.x	HTTP security headers
CORS	2.x	Cross-origin policy
Morgan	1.x	HTTP request logging
dotenv	16.x	Environment variables
express-rate-limit	7.x	Rate limiting
cookie-parser	1.x	Cookie parsing
compression	1.x	Response compression

2.4 Development Tools

Tool	Purpose
Git + GitHub	Version control
VS Code	IDE
Postman	API testing
ESLint	Code linting
Prettier	Code formatting
Nodemon	Auto-restart dev server
concurrently	Run multiple scripts

3. Project Folder Structure

3.1 Root Structure

```

ecommerce-project/
├── backend/           # Node.js API
├── dashboard/        # React.js Admin
├── website/          # Next.js Client
├── documentation/    # All phase docs
│   ├── phase-1-PRD.md
│   ├── phase-2-TRD.md
│   ├── phase-3-AppFlow.md
│   ├── phase-4-UIUXBrief.md
│   ├── phase-5-Schema.md
│   ├── phase-6-APIDocs.md
│   ├── phase-7-ImplementationPlan.md
│   ├── phase-8-DeploymentGuide.md
│   └── phase-9-MaintenanceGuide.md
├── shared/           # Shared assets/types
│   └── assets/
└── README.md

```

3.2 Backend Structure

```

backend/
├── src/
│   ├── config/
│   │   ├── db.js           # MongoDB connection
│   │   ├── cloudinary.js   # Cloudinary config
│   │   └── email.js        # Nodemailer config
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   ├── user.controller.js
│   │   ├── product.controller.js
│   │   ├── category.controller.js
│   │   ├── cart.controller.js
│   │   ├── wishlist.controller.js
│   │   ├── order.controller.js
│   │   ├── review.controller.js
│   │   ├── coupon.controller.js
│   │   ├── upload.controller.js
│   │   └── analytics.controller.js
│   ├── middleware/
│   │   ├── auth.middleware.js # JWT verify
│   │   ├── admin.middleware.js # Role check
│   │   ├── error.middleware.js # Global error handler
│   │   ├── validate.middleware.js
│   │   └── upload.middleware.js # Multer config
│   ├── models/
│   │   ├── User.model.js
│   │   ├── Product.model.js
│   │   ├── Category.model.js
│   │   ├── Cart.model.js
│   │   ├── Wishlist.model.js
│   │   ├── Order.model.js
│   │   ├── Review.model.js
│   │   └── Coupon.model.js
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── user.routes.js
│   │   ├── product.routes.js
│   │   ├── category.routes.js
│   │   ├── cart.routes.js
│   │   ├── wishlist.routes.js
│   │   ├── order.routes.js
│   │   ├── review.routes.js
│   │   ├── coupon.routes.js
│   │   ├── upload.routes.js
│   │   └── analytics.routes.js
│   ├── services/
│   │   ├── email.service.js # Email templates & sending
│   │   └── cloudinary.service.js
│   └── utils/
│       ├── ApiError.js      # Custom error class
│       ├── ApiResponse.js   # Standard response wrapper
│       ├── asyncHandler.js  # Try-catch wrapper
│       ├── generateToken.js # JWT generation
│       └── validators/

```

```
| | | | auth.validator.js
| | | | product.validator.js
| | | | order.validator.js
| | └─ app.js # Express app setup
| └─ .env
| └─ .env.example
| └─ .eslintrc.js
| └─ .prettierrc
| └─ .gitignore
| └─ package.json
| └─ server.js # Entry point
```

3.3 Dashboard Structure (React.js)

```
dashboard/
├─ public/
├─ src/
│   ├─ app/
│   │   ├─ store.js          # Redux store
│   │   └─ rootReducer.js
│   ├─ assets/
│   │   └─ images/
│   ├─ components/
│   │   └─ common/
│   │       ├─ Button.jsx
│   │       ├─ Input.jsx
│   │       ├─ Modal.jsx
│   │       ├─ Table.jsx
│   │       ├─ Badge.jsx
│   │       ├─ Spinner.jsx
│   │       ├─ ConfirmDialog.jsx
│   │       └─ Pagination.jsx
│   │   └─ layout/
│   │       ├─ Sidebar.jsx
│   │       ├─ Header.jsx
│   │       ├─ DashboardLayout.jsx
│   │       └─ AuthLayout.jsx
│   │   └─ charts/
│   │       ├─ RevenueChart.jsx
│   │       └─ OrderChart.jsx
│   └─ features/
│       ├─ auth/
│       │   ├─ authSlice.js
│       │   └─ authApi.js
│       ├─ products/
│       │   ├─ productsSlice.js
│       │   └─ productsApi.js
│       ├─ orders/
│       │   ├─ ordersSlice.js
│       │   └─ ordersApi.js
│       ├─ customers/
│       │   └─ customersApi.js
│       ├─ categories/
│       │   └─ categoriesApi.js
│       ├─ coupons/
│       │   └─ couponsApi.js
│       └─ analytics/
│           └─ analyticsApi.js
│   └─ hooks/
│       ├─ useAuth.js
│       └─ useDebounce.js
│   └─ pages/
│       ├─ auth/
│       │   └─ LoginPage.jsx
│       ├─ dashboard/
│       │   └─ DashboardPage.jsx
│       ├─ products/
│       │   └─ ProductsPage.jsx
```

```
├── AddProductPage.jsx
├── EditProductPage.jsx
├── orders/
│   ├── OrdersPage.jsx
│   └── OrderDetailPage.jsx
├── customers/
│   ├── CustomersPage.jsx
│   └── CustomerDetailPage.jsx
├── categories/
│   └── CategoriesPage.jsx
├── coupons/
│   └── CouponsPage.jsx
├── analytics/
│   └── AnalyticsPage.jsx
├── settings/
│   └── SettingsPage.jsx
├── routes/
│   ├── AppRoutes.jsx
│   └── ProtectedRoute.jsx
├── services/
│   └── api.js # Axios instance
├── utils/
│   ├── formatCurrency.js
│   ├── formatDate.js
│   └── constants.js
├── index.css
├── main.jsx
├── .env
├── .env.example
├── index.html
├── tailwind.config.js
├── vite.config.js
└── package.json
```

3.4 Website Structure (Next.js)


```

website/
├── public/
│   ├── favicon.ico
│   └── robots.txt
├── src/
│   ├── app/
│   │   ├── layout.jsx           # Root layout
│   │   ├── page.jsx            # Home page
│   │   ├── (auth)/
│   │   │   ├── login/
│   │   │   │   └── page.jsx
│   │   │   ├── register/
│   │   │   │   └── page.jsx
│   │   │   └── forgot-password/
│   │   │       └── page.jsx
│   │   ├── products/
│   │   │   ├── page.jsx         # Product listing
│   │   │   └── [slug]/
│   │   │       └── page.jsx     # Product detail
│   │   ├── category/
│   │   │   └── [slug]/
│   │   │       └── page.jsx
│   │   ├── search/
│   │   │   └── page.jsx
│   │   ├── cart/
│   │   │   └── page.jsx
│   │   ├── checkout/
│   │   │   └── page.jsx
│   │   ├── profile/
│   │   │   ├── page.jsx
│   │   │   ├── orders/
│   │   │   │   ├── page.jsx
│   │   │   │   └── [id]/
│   │   │   │       └── page.jsx
│   │   │   ├── wishlist/
│   │   │   │   └── page.jsx
│   │   │   └── addresses/
│   │   │       └── page.jsx
│   │   └── order-success/
│   │       └── page.jsx
│   ├── components/
│   │   ├── common/
│   │   │   ├── Button.jsx
│   │   │   ├── Input.jsx
│   │   │   ├── Spinner.jsx
│   │   │   ├── StarRating.jsx
│   │   │   └── Pagination.jsx
│   │   ├── layout/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Footer.jsx
│   │   │   ├── MobileMenu.jsx
│   │   │   └── SearchBar.jsx
│   │   ├── product/
│   │   │   └── ProductCard.jsx

```

```

├── ProductGrid.jsx
├── ProductFilters.jsx
├── ProductImageGallery.jsx
├── ReviewCard.jsx
├── cart/
│   ├── CartItem.jsx
│   └── CartSummary.jsx
├── checkout/
│   ├── AddressForm.jsx
│   └── OrderSummary.jsx
├── home/
│   ├── HeroBanner.jsx
│   ├── FeaturedCategories.jsx
│   └── FeaturedProducts.jsx
├── context/
│   ├── AuthContext.jsx
│   └── CartContext.jsx
├── hooks/
│   ├── useAuth.js
│   ├── useCart.js
│   ├── useWishlist.js
│   └── useDebounce.js
├── lib/
│   └── axios.js          # Axios instance
├── services/
│   ├── auth.service.js
│   ├── product.service.js
│   ├── cart.service.js
│   ├── order.service.js
│   └── review.service.js
├── utils/
│   ├── formatCurrency.js
│   ├── formatDate.js
│   └── constants.js
├── .env.local
├── .env.example
├── next.config.js
├── tailwind.config.js
└── package.json

```

4. Database Design

4.1 Collections Overview

Collection	Description
users	Customer and admin accounts
products	Product catalog
categories	Product categories
carts	Shopping carts (per user)
wishlists	Wishlists (per user)
orders	Placed orders
reviews	Product reviews
coupons	Discount coupons

4.2 User Schema

```
{
  _id: ObjectId,
  name: String (required, trim),
  email: String (required, unique, lowercase),
  password: String (required, select: false),
  phone: String,
  avatar: {
    url: String,
    publicId: String
  },
  role: String (enum: ['customer', 'admin', 'superadmin'], default: 'customer'),
  addresses: [
    {
      _id: ObjectId,
      label: String,           // "Home", "Work"
      street: String,
      city: String,
      state: String,
      pincode: String,
      country: String (default: 'India'),
      isDefault: Boolean
    }
  ],
  isBlocked: Boolean (default: false),
  resetPasswordToken: String,
  resetPasswordExpire: Date,
  createdAt: Date,
  updatedAt: Date
}
```

4.3 Product Schema

```
{
  _id: ObjectId,
  name: String (required, trim),
  slug: String (unique, auto-generated),
  description: String (required),
  shortDescription: String,
  price: Number (required, min: 0),
  originalPrice: Number,
  discount: Number (0-100),          // percentage
  category: ObjectId (ref: 'Category', required),
  brand: String,
  images: [
    {
      url: String,
      publicId: String,
      isMain: Boolean
    }
  ],
  variants: [
    {
      name: String,                  // "Size", "Color"
      options: [String]             // ["S","M","L"] or ["Red","Blue"]
    }
  ],
  stock: Number (required, min: 0),
  sku: String (unique),
  tags: [String],
  specifications: [
    {
      key: String,
      value: String
    }
  ],
  isActive: Boolean (default: true),
  isFeatured: Boolean (default: false),
  ratings: {
    average: Number (default: 0),
    count: Number (default: 0)
  },
  createdBy: ObjectId (ref: 'User'),
  createdAt: Date,
  updatedAt: Date
}
```

4.4 Category Schema

```
{
  _id: ObjectId,
  name: String (required, trim, unique),
  slug: String (unique),
  description: String,
  image: {
    url: String,
    publicId: String
  },
  parent: ObjectId (ref: 'Category', nullable), // for sub-categories
  isActive: Boolean (default: true),
  createdAt: Date,
  updatedAt: Date
}
```

4.5 Cart Schema

```
{
  _id: ObjectId,
  user: ObjectId (ref: 'User', required, unique),
  items: [
    {
      product: ObjectId (ref: 'Product'),
      name: String,
      image: String,
      price: Number,
      quantity: Number (min: 1),
      selectedVariants: Object // { size: "M", color: "Red" }
    }
  ],
  coupon: ObjectId (ref: 'Coupon', nullable),
  discountAmount: Number (default: 0),
  updatedAt: Date
}
```

4.6 Wishlist Schema

```
{
  _id: ObjectId,
  user: ObjectId (ref: 'User', required, unique),
  products: [ObjectId (ref: 'Product')],
  updatedAt: Date
}
```

4.7 Order Schema

```

{
  _id: ObjectId,
  orderNumber: String (unique, auto-generated), // ORD-20260701-XXXX
  user: ObjectId (ref: 'User', required),
  items: [
    {
      product: ObjectId (ref: 'Product'),
      name: String,
      image: String,
      price: Number,
      quantity: Number,
      selectedVariants: Object
    }
  ],
  shippingAddress: {
    name: String,
    phone: String,
    street: String,
    city: String,
    state: String,
    pincode: String,
    country: String
  },
  paymentMethod: String (enum: ['COD', 'ONLINE']),
  paymentStatus: String (enum: ['pending', 'paid', 'failed', 'refunded'], default: 'pending'),
  orderStatus: String (
    enum: ['pending', 'processing', 'shipped', 'delivered', 'cancelled'],
    default: 'pending'
  ),
  statusHistory: [
    {
      status: String,
      updatedAt: Date,
      note: String
    }
  ],
  subtotal: Number,
  shippingCharge: Number (default: 0),
  taxAmount: Number,
  discountAmount: Number (default: 0),
  totalAmount: Number,
  coupon: ObjectId (ref: 'Coupon', nullable),
  estimatedDelivery: Date,
  deliveredAt: Date,
  cancelledAt: Date,
  cancelReason: String,
  createdAt: Date,
  updatedAt: Date
}

```

4.8 Review Schema

```

{
  _id: ObjectId,
  product: ObjectId (ref: 'Product', required),
  user: ObjectId (ref: 'User', required),
  rating: Number (required, min: 1, max: 5),
  title: String,
  comment: String (required),
  isVerifiedPurchase: Boolean (default: false),
  createdAt: Date,
  updatedAt: Date
}
// Compound unique index: { product: 1, user: 1 }

```

4.9 Coupon Schema

```

{
  _id: ObjectId,
  code: String (required, unique, uppercase),
  discountType: String (enum: ['percentage', 'flat']),
  discountValue: Number (required),
  minOrderAmount: Number (default: 0),
  maxDiscount: Number,           // cap for percentage coupons
  usageLimit: Number,           // total times can be used
  usedCount: Number (default: 0),
  perUserLimit: Number (default: 1),
  usedBy: [ObjectId (ref: 'User')],
  isActive: Boolean (default: true),
  expiresAt: Date,
  createdAt: Date,
  updatedAt: Date
}

```

4.10 Database Indexes

```
// Users
db.users.createIndex({ email: 1 }, { unique: true })

// Products
db.products.createIndex({ slug: 1 }, { unique: true })
db.products.createIndex({ category: 1 })
db.products.createIndex({ isActive: 1, isFeatured: 1 })
db.products.createIndex({ name: 'text', description: 'text', tags: 'text' }) // full-text search

// Orders
db.orders.createIndex({ orderNumber: 1 }, { unique: true })
db.orders.createIndex({ user: 1, createdAt: -1 })
db.orders.createIndex({ orderStatus: 1 })

// Reviews
db.reviews.createIndex({ product: 1, user: 1 }, { unique: true })
db.reviews.createIndex({ product: 1 })

// Coupons
db.coupons.createIndex({ code: 1 }, { unique: true })
```

5. API Design Standards

5.1 Base URL

```
Development: http://localhost:5000/api
Production:  https://api.yourdomain.com/api
```

5.2 HTTP Methods

Method	Usage
GET	Retrieve data
POST	Create resource
PUT	Full update
PATCH	Partial update
DELETE	Delete resource

5.3 Standard Response Format

Success Response:


```
{
  "success": true,
  "message": "Products fetched successfully",
  "data": { ... },
  "pagination": {
    "currentPage": 1,
    "totalPages": 5,
    "totalItems": 50,
    "itemsPerPage": 10
  }
}
```

Error Response:

```
{
  "success": false,
  "message": "Product not found",
  "errors": []
}
```

5.4 HTTP Status Codes

Code	Usage
200	OK — GET, PUT success
201	Created — POST success
204	No Content — DELETE success
400	Bad Request — validation error
401	Unauthorized — not logged in
403	Forbidden — not enough permissions
404	Not Found — resource doesn't exist
409	Conflict — duplicate (email, slug)
422	Unprocessable Entity — invalid data
429	Too Many Requests — rate limited
500	Internal Server Error

5.5 Pagination Query Params

```
GET /api/products?page=1&limit=12&sort=price&order=asc&category=electronics&minPrice=500&maxPrice=500
```

6. Authentication & Authorization

6.1 Strategy: JWT with HTTP-only Cookies

Login Flow:

1. User sends email + password
2. Server validates credentials
3. Server generates Access Token (15 min) + Refresh Token (7 days)
4. Tokens sent via HTTP-only, Secure, SameSite=Strict cookies
5. Client makes requests – cookie sent automatically
6. Server verifies token on each protected request
7. If Access Token expires → use Refresh Token to get new one

6.2 Token Configuration

Token	Expiry	Cookie Flag
Access Token	15 minutes	HTTP-only, Secure, SameSite=Strict
Refresh Token	7 days	HTTP-only, Secure, SameSite=Strict

6.3 Role-Based Access Control (RBAC)

Role	Access
customer	Own profile, cart, wishlist, orders, reviews
admin	All customer access + product/order/customer management
superadmin	All admin access + user management, settings

6.4 Middleware Chain

```
Request → authMiddleware (verify JWT) → adminMiddleware (check role) → Controller
```

6.5 Password Reset Flow

1. User submits email → POST /api/auth/forgot-password
2. Server generates crypto random token (32 bytes)
3. Token hashed and stored in DB with 10-min expiry
4. Reset link emailed: `https://website.com/reset-password?token=<rawToken>`
5. User submits new password → POST /api/auth/reset-password
6. Server finds user by hashed token, checks expiry
7. Password updated, token cleared

7. File Upload Strategy

7.1 Cloudinary Configuration

Storage: Cloudinary CDN
SDK: cloudinary v2 (Node.js)
Process: Multer (memoryStorage) → Cloudinary upload_stream

7.2 Upload Folders

Content	Cloudinary Folder
Product Images	ecommerce/products
Category Images	ecommerce/categories
User Avatars	ecommerce/avatars

7.3 Image Transformations

- **Product thumbnails:** `w_400,h_400,c_fill,f_webp,q_auto`
- **Product full:** `w_800,h_800,c_fit,f_webp,q_auto`
- **Category:** `w_600,h_400,c_fill,f_webp,q_auto`
- **Avatar:** `w_150,h_150,c_fill,g_face,r_max,f_webp`

7.4 Upload Limits

Setting	Value
Max file size	5 MB
Allowed types	image/jpeg, image/png, image/webp
Max images per product	6

8. Email Service

8.1 Provider

- **Service:** Gmail SMTP (dev) / SendGrid (production)
- **Library:** Nodemailer

8.2 Email Templates

Template	Trigger
Welcome Email	New user registration
Order Confirmation	Order placed
Order Status Update	Status changed by admin
Password Reset	Forgot password request
Order Cancellation	Order cancelled

8.3 Email Template Structure

```
HTML template with:
- Company logo & branding
- Clear heading
- Main content (order details, link, etc.)
- Call-to-action button
- Footer with contact info
```

9. Error Handling Standards

9.1 Custom ApiError Class

```
class ApiError extends Error {
  constructor(statusCode, message, errors = []) {
    super(message)
    this.statusCode = statusCode
    this.success = false
    this.errors = errors
  }
}
```

9.2 asyncHandler Wrapper

```
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next)
}
```

9.3 Global Error Middleware

```
app.use((err, req, res, next) => {
  const statusCode = err.statusCode || 500
  res.status(statusCode).json({
    success: false,
    message: err.message || 'Internal Server Error',
    errors: err.errors || []
  })
})
```

10. Environment Configuration

10.1 Backend (.env)

```
# Server
NODE_ENV=development
PORT=5000

# Database
MONGODB_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/ecommerce

# JWT
JWT_ACCESS_SECRET=<strong_random_secret>
JWT_REFRESH_SECRET=<strong_random_secret>
JWT_ACCESS_EXPIRE=15m
JWT_REFRESH_EXPIRE=7d

# Cloudinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# Email
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=
EMAIL_PASS=

# Frontend URLs (CORS)
CLIENT_URL=http://localhost:3000
DASHBOARD_URL=http://localhost:3001

# Cookie
COOKIE_SECURE=false    # true in production
```

10.2 Website (.env.local)

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
NEXT_PUBLIC_SITE_URL=http://localhost:3000
```

10.3 Dashboard (.env)

```
VITE_API_URL=http://localhost:5000/api
```

11. Security Requirements

Security Measure	Implementation
Password Hashing	bcrypt, saltRounds = 12
Token Storage	HTTP-only cookies (no localStorage)
Rate Limiting	100 req/min per IP (express-rate-limit)
CORS	Whitelist only CLIENT_URL and DASHBOARD_URL
HTTP Headers	Helmet.js (XSS, HSTS, CSP, etc.)
Input Sanitization	express-validator on all inputs
NoSQL Injection	Mongoose sanitize query (\$where blocked)
File Upload Safety	Type + size validation before Cloudinary
Sensitive Fields	<code>password</code> field: <code>select: false</code> in schema
Admin Routes	Double middleware: authMiddleware + adminMiddleware
Error Messages	Never expose stack traces in production

12. Performance Requirements

12.1 Backend

Optimization	Method
Response compression	compression middleware (gzip)
Database queries	Mongoose <code>.lean()</code> for read-only queries
Pagination	All list endpoints paginated
Image CDN	Cloudinary serves images (global CDN)
Indexes	Compound indexes on frequently queried fields

12.2 Next.js Website

Optimization	Method
SSG for static pages	Home, category pages pre-rendered
SSR for dynamic pages	Product detail with fresh data
ISR	Product listing (revalidate: 60 seconds)
Image Optimization	next/image (auto WebP, lazy load, responsive)
Code Splitting	Automatic per-route by Next.js
Font Optimization	next/font (Google Fonts, no FOUT)

12.3 React Dashboard

Optimization	Method
RTK Query caching	Auto cache + invalidation
Table virtualization	TanStack Table (virtual rows for large lists)
Lazy loading routes	React.lazy + Suspense
Memoization	React.memo for heavy components

13. Deployment Architecture

13.1 Target Platforms

Service	Platform
Backend API	Railway / Render
Client Website	Vercel
Admin Dashboard	Vercel / Netlify
Database	MongoDB Atlas (M0 free → M10 production)
Images	Cloudinary (free tier → paid)
Email	Gmail SMTP → SendGrid (production)

13.2 Environment Tiers

Tier	Purpose
Development	Local machines, <code>localhost</code>
Staging	Pre-production testing
Production	Live, public-facing

14. Version Control Strategy

14.1 Branching Model

```
main          ← production-ready code only
├── develop ← integration branch
│   ├── feature/product-listing
│   ├── feature/checkout-flow
│   ├── fix/cart-calculation-bug
│   └── chore/update-dependencies
```

14.2 Commit Message Convention

```
<type>(<scope>): <short description>
```

Types: feat | fix | docs | style | refactor | test | chore

Examples:

feat(auth): add JWT refresh token logic

fix(cart): resolve quantity calculation error

docs(readme): update setup instructions

14.3 PR Rules

- Every feature must go through a Pull Request
- At least 1 review before merging to `develop`
- `main` branch is protected — no direct pushes
- Squash commits before merging

15. Coding Standards

15.1 General

- **Language:** JavaScript (ES2022+)
- **Module system:** ES Modules (`import/export`) everywhere
- **Async:** `async/await` — no callbacks or `.then()` chains
- **Naming:** camelCase for variables/functions, PascalCase for components/classes, UPPER_SNAKE_CASE for constants

15.2 Backend

```
// ✅ Good
export const getProducts = asyncHandler(async (req, res) => {
  const { page = 1, limit = 12 } = req.query
  const products = await Product.find({ isActive: true })
    .lean()
    .limit(limit)
    .skip((page - 1) * limit)
  res.status(200).json(new ApiResponse(200, products, 'Products fetched'))
})

// ❌ Bad
app.get('/products', async (req, res) => {
  try {
    const p = await Product.find()
    res.json(p)
  } catch(e) {
    res.status(500).json({ error: e.message })
  }
})
```

15.3 React / Next.js

```
// ✅ Good — functional component, named export
export const ProductCard = ({ product }) => {
  const { name, price, images } = product
  return (
    <div className="product-card">
      <img src={images[0]?.url} alt={name} />
      <h3>{name}</h3>
      <p>₹{price.toLocaleString()}</p>
    </div>
  )
}

// ❌ Bad — default export, unclear naming
export default function Card(props) { ... }
```

15.4 ESLint + Prettier Config

```
// .prettierrc
{
  "semi": false,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100
}
```

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial TRD created

Previous Phase → [Phase 1: Product Requirements Document \(PRD\)](#)

Next Phase → [Phase 3: Application Flow](#)

Section 4: Phase 3: Application Flow & Architecture

Application Flow Document

Project: Production-Ready E-Commerce Platform

Version: 1.0

Status: Planning

Date: July 2026

Phase: 3 of 9

Table of Contents

- 1. Overview
- 2. Client Website Flows
 - 2.1 Guest User Flow
 - 2.2 Authentication Flow
 - 2.3 Home Page Flow
 - 2.4 Product Discovery Flow
 - 2.5 Product Detail Flow
 - 2.6 Cart Flow
 - 2.7 Checkout Flow
 - 2.8 Order Success Flow
 - 2.9 User Profile Flow
 - 2.10 Wishlist Flow
 - 2.11 Order Tracking Flow
 - 2.12 Review Flow
 - 2.13 Password Reset Flow
- 3. Admin Dashboard Flows
 - 3.1 Admin Login Flow
 - 3.2 Dashboard Overview Flow
 - 3.3 Product Management Flow
 - 3.4 Order Management Flow
 - 3.5 Customer Management Flow
 - 3.6 Category Management Flow

- 3.7 Coupon Management Flow
 - 3.8 Analytics Flow
 - 4. Backend API Flows
 - 4.1 Request Lifecycle
 - 4.2 JWT Auth Flow
 - 4.3 Image Upload Flow
 - 4.4 Order Processing Flow
 - 5. Navigation Maps
 - 5.1 Website Navigation Map
 - 5.2 Dashboard Navigation Map
 - 6. Page States
 - 7. Error States & Fallbacks
-

1. Overview

This document maps every user journey, screen transition, and data flow across the three applications. It defines how users move through the platform, what happens at each step, and how data travels between the frontend and backend.

Applications Covered

App	Users	Entry Point
Client Website (Next.js)	Customers (guest + logged in)	<code>website.com/</code>
Admin Dashboard (React.js)	Admin, Super Admin	<code>admin.website.com/login</code>
Backend API (Node.js)	Both apps via HTTP	<code>api.website.com/api</code>

2. Client Website Flows

2.1 Guest User Flow

Visit website.com



[Home Page]

- Hero Banner
- Featured Categories
- Featured Products
- Promotional Banners



- Browse Products (no login required)
- View Product Detail (no login required)
- Search Products (no login required)
- Add to Cart (guest cart – saved in cookie/localStorage)
- Checkout



[Login Required]

Redirect to /login?redirect=/checkout

Guest Cart Behavior:

- Items stored in `localStorage` (key: `guestCart`)
- On login → guest cart merged with server cart
- Cart count shown in Navbar for both guest and logged-in users

2.2 Authentication Flow

Registration

```
/register
|
▼
Fill Form [Name, Email, Password, Confirm Password]
|
▼
Submit → POST /api/auth/register
|
├── Validation Error → Show inline field errors
├── Success
|   |
|   ▼
|   JWT cookies set (access + refresh)
|   |
|   ▼
|   Welcome email sent
|   |
|   ▼
|   Redirect to / (Home Page)
```

Login

```
/login
|
▼
Fill Form [Email, Password]
|
▼
Submit → POST /api/auth/login
|
├── Invalid credentials → Show error toast
├── Account blocked → Show blocked message
├── Success
|   |
|   ▼
|   JWT cookies set
|   |
|   ▼
|   Guest cart merged with server cart
|   |
|   ▼
|   Redirect to previous page OR /
```

Logout

Click "Logout" in Navbar/Profile menu



POST /api/auth/logout



JWT cookies cleared on server



Local state cleared (user, cart)



Redirect to /

2.3 Home Page Flow

[Home Page] - /



Click Category Card



/category/[slug]



Click Featured Product Card



/products/[slug]



Click "Shop Now" (Hero Banner)



/products



Click "Add to Wishlist" on Product Card



└ Not logged in → /login?redirect=back



└ Logged in → POST /api/wishlist/add → Toast notification



Click "Add to Cart" on Product Card



└ Guest → add to localStorage cart



└ Logged in → POST /api/cart/add → Cart count updated

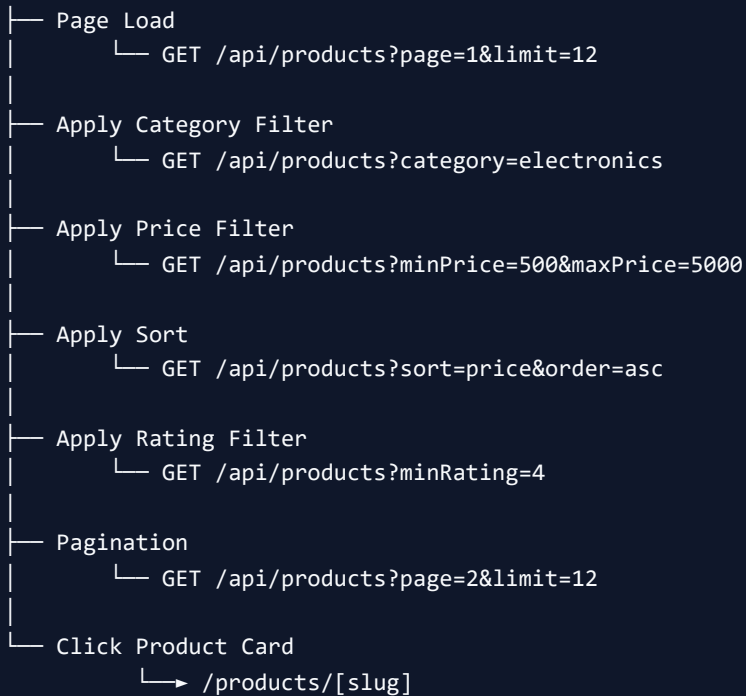


Newsletter Signup → POST /api/newsletter → Success toast

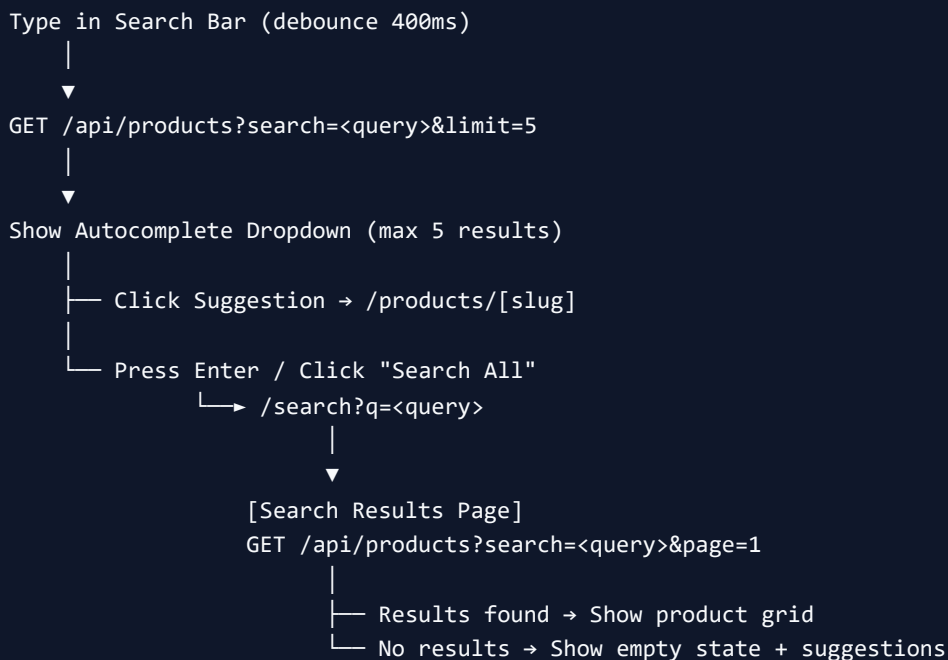
2.4 Product Discovery Flow

Product Listing

[Product Listing] – /products



Search Flow



2.5 Product Detail Flow

[Product Detail] – /products/[slug]

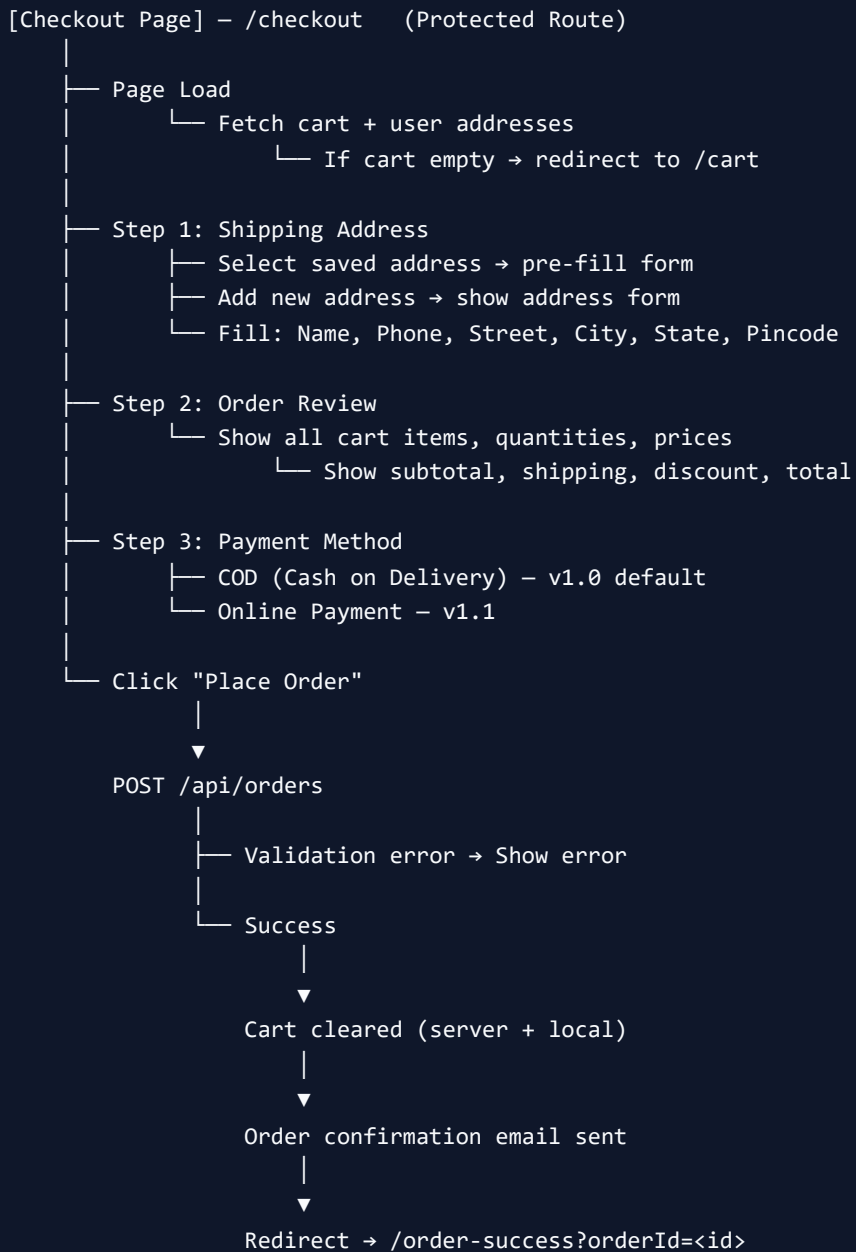
- Page Load (SSR)
 - └ GET /api/products/[slug]
 - └ GET /api/reviews/[productId]
- Image Gallery
 - └ Click thumbnail → Switch main image
 - └ Click main image → Open zoom modal
- Select Variant (Size/Color)
 - └ Click option → Update selected variant state
- Quantity Selector
 - └ +/- buttons → Update quantity (min:1, max:stock)
- Add to Cart
 - └ Guest → save to localStorage
 - └ Logged in → POST /api/cart/add
 - └ Out of stock → Button disabled, show "Out of Stock"
- Add to Wishlist
 - └ Not logged in → redirect to /login
 - └ Logged in → POST /api/wishlist/add → icon fills (red heart)
- Write Review (logged in + purchased only)
 - └ See Review Flow (2.12)
- Related Products Section
 - └ Click → /products/[slug]

2.6 Cart Flow

[Cart Page] – /cart

- Page Load
 - Guest → read localStorage
 - Logged in → GET /api/cart
- Update Quantity
 - Guest → update localStorage
 - Logged in → PUT /api/cart/update
- Remove Item
 - Guest → remove from localStorage
 - Logged in → DELETE /api/cart/remove/:productId
- Apply Coupon
 - POST /api/coupons/validate
 - Valid → apply discount, show savings
 - Invalid → show error (expired / not found / min order)
- Empty Cart State
 - Show illustration + "Continue Shopping" button → /products
- Click "Proceed to Checkout"
 - Guest → /login?redirect=/checkout
 - Logged in → /checkout

2.7 Checkout Flow



2.8 Order Success Flow

[Order Success Page] – /order-success

- Display
 - ☒ Success animation / icon
 - Order Number (e.g., ORD-20260701-4821)
 - Estimated delivery date
 - Summary of items ordered
- CTA Buttons
 - "Track My Order" → /profile/orders/[id]
 - "Continue Shopping" → /products
- Email sent to customer with order details

2.9 User Profile Flow

[Profile] – /profile (Protected Route)

- View/Edit Profile
 - PUT /api/users/:id → update name, phone, avatar
- Change Password
 - PUT /api/users/:id/password
 - Wrong current password → error
 - Success → toast + logout (re-login required)
- Manage Addresses – /profile/addresses
 - Add Address → POST /api/users/:id/addresses
 - Edit Address → PUT /api/users/:id/addresses/:addressId
 - Delete Address → DELETE /api/users/:id/addresses/:addressId
 - Set Default → PATCH /api/users/:id/addresses/:addressId/default
- Order History – /profile/orders
 - GET /api/orders/my-orders (paginated)
 - Click Order → /profile/orders/[id]
- Wishlist – /profile/wishlist
 - See Wishlist Flow (2.10)

2.10 Wishlist Flow

```

[Wishlist] - /profile/wishlist (Protected Route)
├── Page Load → GET /api/wishlist
├── Empty State → Show "No items in wishlist" + Browse button
├── Wishlist Item Actions
│   ├── Remove from Wishlist → DELETE /api/wishlist/remove/:productId
│   └── Move to Cart
│       └── POST /api/cart/add
│           └── DELETE /api/wishlist/remove/:productId
└── Click Product → /products/[slug]

```

2.11 Order Tracking Flow

```

[Order Detail] - /profile/orders/[id]
├── Page Load → GET /api/orders/:id
├── Display
│   ├── Order Number, Date, Payment Method
│   ├── Status Timeline
│   │   └── pending → processing → shipped → delivered
│   │       (current status highlighted)
│   ├── Estimated Delivery Date
│   ├── Shipping Address
│   ├── Items Ordered (name, qty, price)
│   └── Price Breakdown (subtotal, shipping, discount, total)
└── Cancel Order Button (only if status = "pending")
    └── ▼
        Confirm Dialog: "Are you sure you want to cancel?"
            ├── No → dismiss
            └── Yes → PUT /api/orders/:id/cancel
                    └── Status updated → Toast → Page refresh

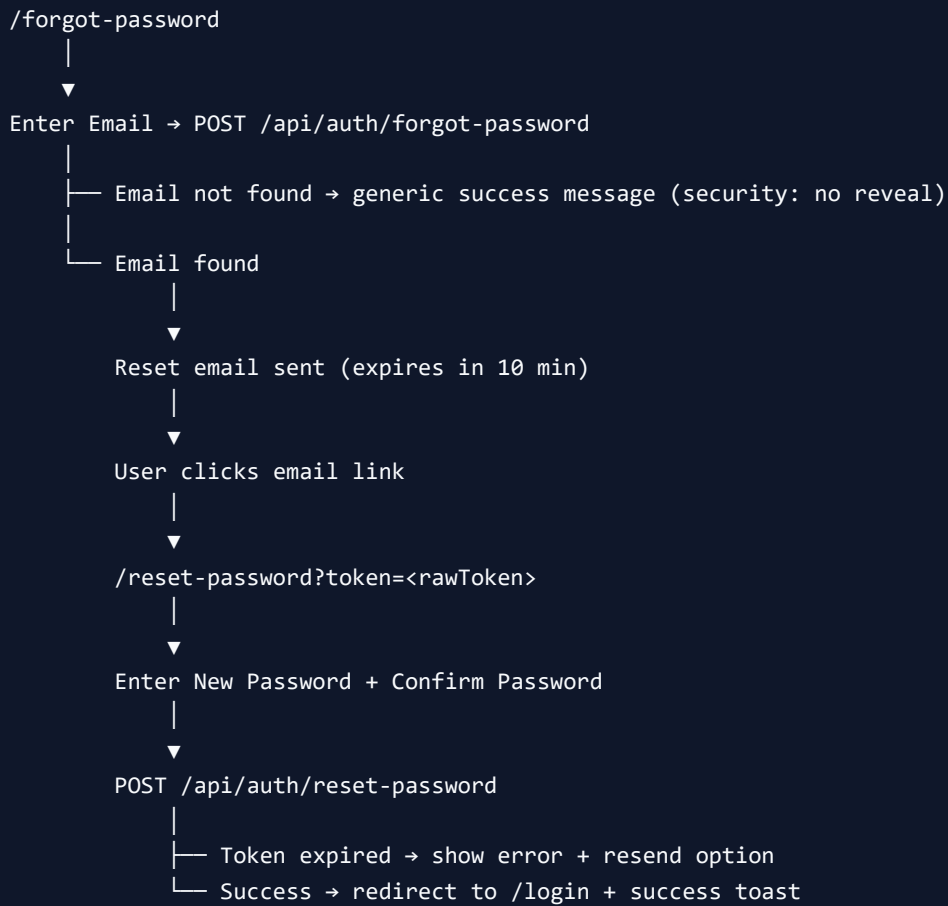
```

2.12 Review Flow

[Product Detail Page – Reviews Section]

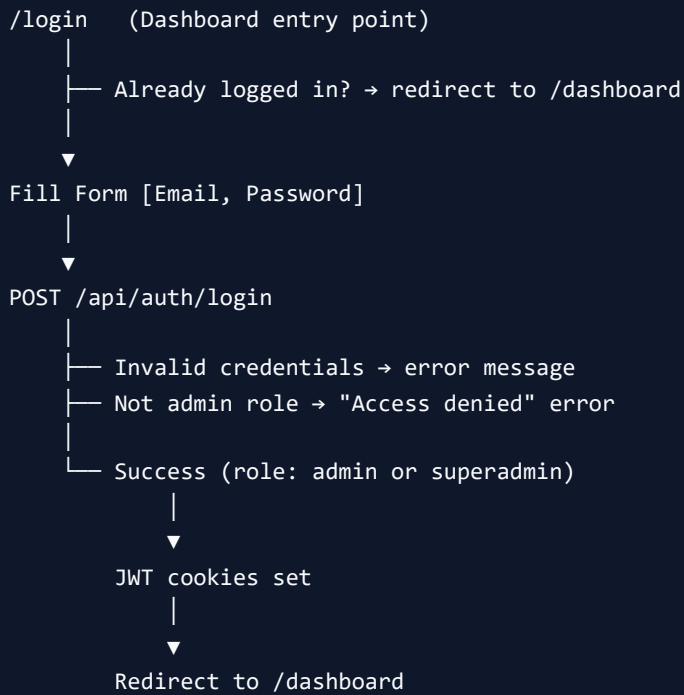
- └ View Reviews
 - └ GET /api/reviews/:productId
 - └ Show star ratings, review text, verified badge
- └ Write Review (only if: logged in + purchased product)
 - └ ▼
 - [Review Form]
 - └ Select Star Rating (1-5)
 - └ Write Title (optional)
 - └ Write Comment (required)
 - └ Submit → POST /api/reviews/:productId
 - └ Already reviewed → show "Edit" instead
 - └ Success → review appears immediately, ratings updated
- └ Edit Review
 - └ PUT /api/reviews/:reviewId
- └ Delete Review
 - └ DELETE /api/reviews/:reviewId
 - └ Confirm → removed from list

2.13 Password Reset Flow

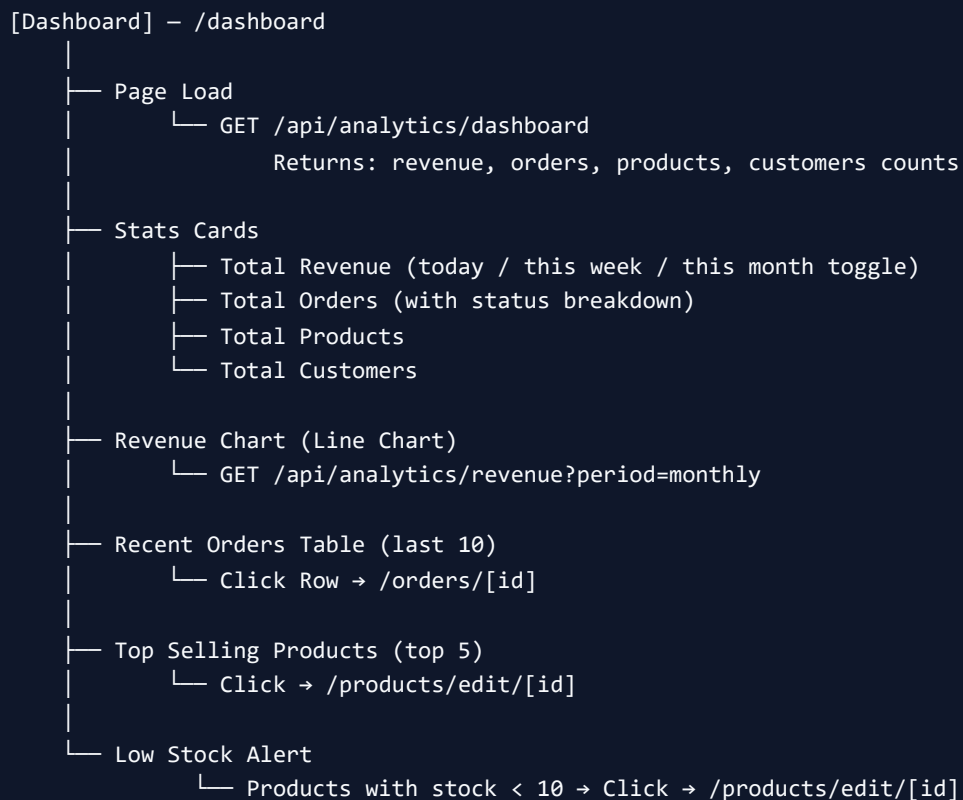


3. Admin Dashboard Flows

3.1 Admin Login Flow



3.2 Dashboard Overview Flow



3.3 Product Management Flow

Product List

```
[Products] - /products
├── Page Load → GET /api/products?page=1&limit=20 (admin view: all)
├── Search → GET /api/products?search=<query>
├── Filter by Category → GET /api/products?category=<id>
├── Filter by Status → GET /api/products?isActive=true/false
├── Table Actions per Row
│   ├── Edit → /products/edit/[id]
│   ├── Toggle Active/Inactive → PATCH /api/products/:id/status
│   └── Delete → Confirm Dialog → DELETE /api/products/:id
├── Bulk Actions (select multiple)
│   ├── Bulk Delete → DELETE /api/products/bulk
│   └── Bulk Status Change
└── "Add Product" Button → /products/add
```

Add/Edit Product

```
[Add Product] - /products/add
[Edit Product] - /products/edit/[id]
├── Form Fields
│   ├── Basic Info: Name, Slug (auto), Short Desc, Full Desc
│   ├── Category (dropdown from GET /api/categories)
│   ├── Brand, SKU
│   ├── Pricing: Price, Original Price, Discount %
│   ├── Stock Quantity
│   ├── Tags (multi-input)
│   ├── Variants (add variant groups with options)
│   ├── Specifications (key-value pairs)
│   ├── Image Upload (drag & drop, max 6)
│   │   ├── POST /api/upload/image → Cloudinary
│   │   └── Preview shown immediately
│   ├── Status Toggle (Active/Inactive)
│   └── Featured Toggle
└── Submit
    ├── POST /api/products (Add)
    ├── PUT /api/products/:id (Edit)
    └── Success → redirect to /products with toast
```

3.4 Order Management Flow

Order List

```
[Orders] - /orders
├── Page Load → GET /api/orders?page=1&limit=20
├── Filters
│   ├── By Status (pending/processing/shipped/delivered/cancelled)
│   ├── By Date Range (date picker)
│   └── By Customer (search by name/email)
├── Table Columns
│   Order# | Customer | Items | Total | Status | Date | Actions
├── Actions per Row
│   └── View → /orders/[id]
└── Export to CSV Button → download orders.csv
```

Order Detail

```
[Order Detail] - /orders/[id]
├── Page Load → GET /api/orders/:id
├── Display
│   ├── Order Info (number, date, payment method, payment status)
│   ├── Customer Info (name, email, phone) → click → /customers/[id]
│   ├── Shipping Address
│   ├── Items Table (product, qty, price, total)
│   ├── Price Summary (subtotal, shipping, discount, total)
│   └── Status History Timeline
├── Update Order Status
│   ├── Dropdown: pending → processing → shipped → delivered → cancelled
│   ├── Optional Note field
│   └── Submit → PUT /api/orders/:id/status
│       └── Email sent to customer on status change
└── View Invoice Button → generate/print order invoice
```

3.5 Customer Management Flow

```

[Customers] - /customers
├── Page Load → GET /api/users?role=customer&page=1
├── Search by name/email
├── Filter by status (active/blocked)
├── Table Columns
│   ├── Name | Email | Phone | Orders | Joined | Status | Actions
├── Actions per Row
│   ├── View → /customers/[id]
│   └── Block/Unblock → PUT /api/users/:id/block
└── Export Customers CSV

[Customer Detail] - /customers/[id]
├── Profile Info (avatar, name, email, phone, join date)
├── Address List
├── Order History Table (GET /api/orders?user=:id)
│   └── Click Order → /orders/[orderId]
└── Block/Unblock Toggle

```

3.6 Category Management Flow

```

[Categories] - /categories
├── Page Load → GET /api/categories
├── Category Tree View (parent → children)
├── Actions
│   ├── Add Category → Modal Form
│   │   ├── Name, Slug (auto), Description
│   │   ├── Parent Category (optional dropdown)
│   │   ├── Image Upload → Cloudinary
│   │   └── POST /api/categories
│   ├── Edit Category → pre-filled Modal
│   │   └── PUT /api/categories/:id
│   └── Delete Category → Confirm Dialog
│       ├── Has products? → "Reassign products first" warning
│       └── No products → DELETE /api/categories/:id
└── Toggle Active/Inactive → PATCH /api/categories/:id/status

```

3.7 Coupon Management Flow

```
[Coupons] - /coupons
├── Page Load → GET /api/coupons
├── Coupon Table Columns
│   ├── Code | Type | Value | Min Order | Usage | Expiry | Status | Actions
├── Create Coupon → Modal Form
│   ├── Code (auto-generate or manual, uppercase)
│   ├── Discount Type: Percentage / Flat Amount
│   ├── Discount Value
│   ├── Minimum Order Amount
│   ├── Max Discount Cap (for percentage type)
│   ├── Usage Limit (total / per user)
│   ├── Expiry Date
│   └── POST /api/coupons
├── Edit Coupon → pre-filled Modal → PUT /api/coupons/:id
├── Toggle Active/Inactive → PATCH /api/coupons/:id/status
└── Delete Coupon → DELETE /api/coupons/:id
```

3.8 Analytics Flow

```
[Analytics] - /analytics
├── Page Load → GET /api/analytics/dashboard
├── Revenue Report
│   ├── Period Toggle: Daily | Weekly | Monthly | Yearly
│   └── GET /api/analytics/revenue?period=monthly&year=2026
│       └── Line Chart (Recharts)
├── Orders Report
│   └── GET /api/analytics/orders?period=monthly
│       └── Bar Chart (orders by status)
├── Top Products
│   └── GET /api/analytics/top-products?limit=10
│       └── Table with product name, units sold, revenue
└── Export Report
    └── Download as CSV / PDF
```

4. Backend API Flows

4.1 Request Lifecycle



4.2 JWT Auth Flow



4.3 Image Upload Flow

```
Admin uploads image (drag & drop in dashboard)
|
▼
Multer (memoryStorage) receives file buffer
|
▼
Validate: type (jpeg/png/webp) + size (< 5MB)
|
├── Invalid → 400 error
|
▼
Upload to Cloudinary via upload_stream
{ folder: 'ecommerce/products', transformation: [...] }
|
▼
Cloudinary returns { url, public_id }
|
▼
Store { url, publicId } in product.images[]
|
▼
Return url to frontend → preview shown

Delete Flow:
DELETE /api/upload/image/:publicId
|
▼
cloudinary.uploader.destroy(publicId)
|
▼
Remove from DB document
```

4.4 Order Processing Flow


```

Customer clicks "Place Order"
|
▼
POST /api/orders
|
▼
Validate:
|— Cart not empty
|— All items in stock
|— Shipping address valid
|— Payment method valid
|
|— Validation fails → 400 error
|
▼
Calculate:
  subtotal = Σ (item.price × item.quantity)
  discountAmount = (coupon applied ? calculate : 0)
  taxAmount = subtotal × TAX_RATE (e.g., 18% GST)
  shippingCharge = (subtotal > FREE_SHIPPING_THRESHOLD ? 0 : 99)
  totalAmount = subtotal - discountAmount + taxAmount + shippingCharge
|
▼
Create Order document (orderNumber: ORD-YYYYMMDD-XXXX)
|
▼
Update product stock (decrement each item's stock)
|
▼
Update coupon usedCount + add user to usedBy[]
|
▼
Clear user's cart
|
▼
Send Order Confirmation Email (async, non-blocking)
|
▼
Return order data to client
|
▼
Client redirects to /order-success

```

5. Navigation Maps

5.1 Website Navigation Map

```

website.com/
├─ /                               ← Home
├─ /products                       ← All Products
│   └─ /products/[slug]          ← Product Detail
├─ /category/[slug]              ← Category Listing
├─ /search?q=                     ← Search Results
├─ /cart                           ← Cart
├─ /checkout                      🔒 ← Checkout (auth required)
├─ /order-success                 🔒 ← Order Success
├─ /login                         ← Login
├─ /register                      ← Register
├─ /forgot-password              ← Forgot Password
├─ /reset-password?token=        ← Reset Password
└─ /profile                       🔒 ← User Profile (auth required)
    ├─ /profile                  ← Profile Info
    ├─ /profile/orders           ← Order History
    │   └─ /profile/orders/[id] ← Order Detail
    ├─ /profile/wishlist         ← Wishlist
    └─ /profile/addresses        ← Saved Addresses

```

5.2 Dashboard Navigation Map

```

admin.website.com/
├─ /login                         ← Admin Login
└─ /dashboard                     🔒 ← (all below are protected)
    ├─ /dashboard                 ← Overview
    ├─ /products                  ← Product List
    │   ├─ /products/add          ← Add Product
    │   └─ /products/edit/[id]   ← Edit Product
    ├─ /orders                    ← Order List
    │   └─ /orders/[id]          ← Order Detail
    ├─ /customers                 ← Customer List
    │   └─ /customers/[id]       ← Customer Detail
    ├─ /categories                ← Category Management
    ├─ /coupons                  ← Coupon Management
    ├─ /analytics                 ← Analytics & Reports
    └─ /settings                  ← Settings

```

6. Page States

Every page must handle these states:

Loading State

- Skeleton loaders (not spinners) for content-heavy pages
- Spinner for action buttons (submit, delete)
- Disabled state for all interactive elements while loading

Empty State

- Illustrated empty state (SVG icon + message + CTA)

Examples:

- Cart empty → "Your cart is empty" + "Browse Products" button
- No orders → "No orders yet" + "Shop Now" button
- No wishlist → "Your wishlist is empty" + "Explore Products" button
- No search results → "No results for '<query>'" + suggestions

Error State

- API error → Error toast (top-right, auto-dismiss 4s)
- 404 Page → Custom 404 with "Go Home" button
- 500 / Network error → "Something went wrong, please try again"
- Unauthorized → redirect to login

Success State

- Form submit → Toast notification ("Product added successfully")
- Order placed → Full Order Success page
- Profile update → Inline success message

7. Error States & Fallbacks

Scenario	User Sees	Action
API is down	"Something went wrong. Please try again."	Retry button
Product out of stock (during checkout)	"Item out of stock, removed from cart"	Cart updated
Coupon expired	"This coupon has expired"	Coupon removed
Session expired	"Session expired, please login again"	Redirect to /login
Invalid product URL	Custom 404 page	"Go Home" button
Payment failure (v1.1)	"Payment failed. Try again or use COD"	Retry / change method
Image upload fails	"Image upload failed. Please try again."	Re-upload option
Network timeout	"No internet connection. Check your network."	Auto-retry on reconnect
Blocked account	"Your account has been suspended. Contact support."	Contact email shown
Admin: duplicate product SKU	"SKU already exists. Please use a unique SKU."	Highlight SKU field

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial Application Flow Document

Previous Phase → [Phase 2: Technical Requirements Document \(TRD\)](#)

Next Phase → [Phase 4: UI/UX Brief](#)

Section 5: Phase 4: UI/UX Brief & Design System

UI/UX Brief

Project: Production-Ready E-Commerce Platform
Version: 1.0
Status: Planning
Date: July 2026
Phase: 4 of 9

Table of Contents

- 1. [Design Philosophy](#)
- 2. [Brand Identity](#)
- 3. [Color System](#)
- 4. [Typography](#)
- 5. [Spacing & Layout Grid](#)
- 6. [Iconography](#)
- 7. [Component Library](#)
- 8. [Client Website — Page-by-Page UI Brief](#)
- 9. [Admin Dashboard — Page-by-Page UI Brief](#)
- 10. [Responsive Breakpoints](#)
- 11. [Animation & Micro-interactions](#)
- 12. [Accessibility Guidelines](#)
- 13. [Image & Media Guidelines](#)
- 14. [Design Tokens \(CSS Variables\)](#)

1. Design Philosophy

Core Principles

Principle	Description
Clean & Modern	Ample whitespace, clear hierarchy, no clutter
Mobile First	Design starts at 375px, scales up
Speed Feels Fast	Skeleton loaders, instant feedback, optimistic UI
Trust & Safety	Clear CTAs, secure badge indicators, transparent pricing
Accessible	WCAG 2.1 AA compliant — usable by everyone

Design Mood

- **Website:** Bright, fresh, inviting — makes users want to shop
- **Dashboard:** Professional, data-dense, calm — helps admins focus

2. Brand Identity

Brand Name

ShopEase (*placeholder — replace with actual brand name*)

Brand Voice

- Friendly but professional
- Clear and direct (no jargon)
- Helpful and reassuring during purchase flow

Logo Usage

Primary Logo: [Icon] + "ShopEase" wordmark — for Navbar, emails
 Icon Only: Favicon, mobile app icon
 Wordmark Only: Footer, loading screen

Brand Tagline

"Everything you need, delivered."

3. Color System

3.1 Website — Primary Palette

```
/* Primary — Indigo Blue */
--color-primary-50: #eef2ff;
--color-primary-100: #e0e7ff;
--color-primary-200: #c7d2fe;
--color-primary-300: #a5b4fc;
--color-primary-400: #818cf8;
--color-primary-500: #6366f1; /* Main brand color */
--color-primary-600: #4f46e5; /* Hover state */
--color-primary-700: #4338ca; /* Active/pressed */
--color-primary-800: #3730a3;
--color-primary-900: #312e81;

/* Accent — Amber (Sale badges, highlights) */
--color-accent-400: #fbbf24;
--color-accent-500: #f59e0b;
--color-accent-600: #d97706;

/* Success — Green */
--color-success-100: #dcfce7;
--color-success-500: #22c55e;
--color-success-700: #15803d;

/* Error — Red */
--color-error-100: #fee2e2;
--color-error-500: #ef4444;
--color-error-700: #b91c1c;

/* Warning — Orange */
--color-warning-100: #ffedd5;
--color-warning-500: #f97316;

/* Neutral — Gray scale */
--color-gray-50: #f9fafb;
--color-gray-100: #f3f4f6;
--color-gray-200: #e5e7eb;
--color-gray-300: #d1d5db;
--color-gray-400: #9ca3af;
--color-gray-500: #6b7280;
--color-gray-600: #4b5563;
--color-gray-700: #374151;
--color-gray-800: #1f2937;
--color-gray-900: #111827;

/* Base */
--color-white: #ffffff;
--color-black: #000000;
--color-background: #f9fafb; /* Page background */
--color-surface: #ffffff; /* Cards, modals */
```

3.2 Dashboard — Dark Professional Palette

```
/* Dashboard background — Deep Dark Blue */
--dash-bg:          #0f172a; /* Main background */
--dash-surface:      #1e293b; /* Cards, sidebar */
--dash-surface-2:    #334155; /* Hover, elevated cards */
--dash-border:       #475569; /* Borders, dividers */

/* Dashboard Primary — Blue */
--dash-primary:      #3b82f6; /* Main action color */
--dash-primary-h:    #2563eb; /* Hover */

/* Dashboard text */
--dash-text:         #f1f5f9; /* Primary text */
--dash-text-muted:   #94a3b8; /* Secondary text */
--dash-text-dim:     #64748b; /* Disabled, placeholder */

/* Status Colors */
--dash-success:      #10b981;
--dash-warning:      #f59e0b;
--dash-error:        #ef4444;
--dash-info:         #06b6d4;
```

3.3 Semantic Color Usage

Use Case	Token
Primary CTA buttons	<code>--color-primary-500</code>
Hover on CTA	<code>--color-primary-600</code>
Destructive actions	<code>--color-error-500</code>
Success toasts/badges	<code>--color-success-500</code>
Sale / discount badge	<code>--color-accent-500</code>
Star ratings	<code>#facc15</code> (yellow-400)
"In Stock" label	<code>--color-success-500</code>
"Out of Stock" label	<code>--color-error-500</code>
"Low Stock" label	<code>--color-warning-500</code>
Order: Pending	<code>--color-warning-500</code>
Order: Processing	<code>--color-primary-500</code>
Order: Shipped	<code>--color-info</code>
Order: Delivered	<code>--color-success-500</code>
Order: Cancelled	<code>--color-error-500</code>

4. Typography

4.1 Font Families

```

/* Heading Font */
--font-heading: 'Plus Jakarta Sans', sans-serif;

/* Body Font */
--font-body: 'Inter', sans-serif;

/* Monospace (for codes, SKUs, order numbers) */
--font-mono: 'JetBrains Mono', monospace;

```

Import from Google Fonts: `Plus Jakarta Sans` (600, 700, 800) + `Inter` (400, 500, 600)

4.2 Type Scale

Token	Size	Weight	Line Height	Usage
<code>--text-xs</code>	12px	400	1.4	Labels, captions
<code>--text-sm</code>	14px	400	1.5	Body small, helpers
<code>--text-base</code>	16px	400	1.6	Body text
<code>--text-lg</code>	18px	500	1.5	Subheadings
<code>--text-xl</code>	20px	600	1.4	Section titles
<code>--text-2xl</code>	24px	700	1.3	Page headings
<code>--text-3xl</code>	30px	700	1.2	Hero subheadings
<code>--text-4xl</code>	36px	800	1.1	Hero headings
<code>--text-5xl</code>	48px	800	1.0	Large hero (desktop)

4.3 Typography Rules

- **Headings:** Plus Jakarta Sans, Bold/ExtraBold
- **Body:** Inter, Regular/Medium
- **Prices:** Inter, Bold, slightly larger than surrounding text
- **Discounts:** Strikethrough + red color for original price
- **Order numbers:** JetBrains Mono for readability

5. Spacing & Layout Grid

5.1 Spacing Scale (4px base)

```
--space-1: 4px
--space-2: 8px
--space-3: 12px
--space-4: 16px
--space-5: 20px
--space-6: 24px
--space-8: 32px
--space-10: 40px
--space-12: 48px
--space-16: 64px
--space-20: 80px
--space-24: 96px
```

5.2 Website Layout Grid

```
Desktop (≥1280px): 12-column grid, max-width: 1280px, gap: 24px
Laptop (≥1024px): 12-column grid, max-width: 1024px, gap: 20px
Tablet (≥768px): 8-column grid, gap: 16px
Mobile (<768px): 4-column grid, gap: 12px
```

5.3 Container

```
.container {
  width: 100%;
  max-width: 1280px;
  margin: 0 auto;
  padding: 0 24px; /* desktop */
  /* padding: 0 16px; */ /* mobile */
}
```

5.4 Dashboard Layout

```
Sidebar: 240px fixed width (collapsible to 64px)
Main Content: flex-1, padding: 24px
Header: 64px height, full width minus sidebar
```

6. Iconography

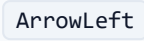
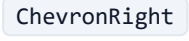
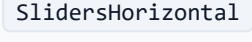
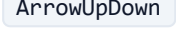
Icon Library: Lucide React

- **Style:** Outline (default), Filled for active states
- **Sizes:** 16px (inline), 20px (buttons), 24px (standalone), 32px (feature icons)

- **Color:** Inherits from parent or uses semantic color tokens

Common Icon Mappings

Action / Concept	Lucide Icon
Cart	ShoppingCart
Wishlist (empty)	Heart
Wishlist (filled)	HeartFill
Search	Search
User / Profile	User
Menu (mobile)	Menu
Close	X
Edit	Pencil
Delete	Trash2
Add	Plus
Settings	Settings
Orders	Package
Dashboard	LayoutDashboard
Analytics	BarChart2
Customers	Users
Categories	Grid3x3
Coupons	Tag
Upload	Upload
Download	Download
Eye (view)	Eye
Star (rating)	Star
Truck (shipping)	Truck
Check	CheckCircle2
Alert	AlertCircle

Action / Concept	Lucide Icon
Info	
Arrow back	
Chevron	
Filter	
Sort	

7. Component Library

7.1 Buttons

Variants:

- Primary – filled indigo, white text
- Secondary – white bg, indigo border + text
- Danger – filled red
- Ghost – transparent bg, text only
- Link – underline style

Sizes:

- sm – h-8, px-3, text-sm
- md – h-10, px-4, text-base (default)
- lg – h-12, px-6, text-lg

States:

- Default | Hover (darker shade) | Active (pressed)
- Loading (spinner inside, disabled) | Disabled (opacity-50)

Rules:

- Always have min-width to prevent layout shift on loading state
- Icon buttons: square, same height variants
- Full-width on mobile checkout forms

7.2 Input Fields

Structure:

Label (above)
Input field
Helper text OR Error message (below)

Variants:

Default – gray border
Focus – indigo border + ring
Error – red border + red helper text
Disabled – gray bg, not-allowed cursor

Types handled:

text | email | password | number | search | textarea | select

7.3 Cards

Product Card (Website):

[Product Image]	→ aspect-ratio: 1/1, object-cover
[Wishlist icon]	→ top-right overlay on hover
Category (small)	
Product Name	→ 2-line clamp
★★★★☆ (4.2) 128	
₹1,299 ₹1,999	→ price + strikethrough original
[Add to Cart btn]	

Hover effect: card lifts (translateY -4px + shadow deepens)

Dashboard Stat Card:

Icon (colored bg)	
Label	
Value (large bold)	
↑ 12% vs last month	→ trend indicator

7.4 Badges & Tags

Order Status Badges:

Pending – amber bg, amber text
Processing – blue bg, blue text
Shipped – cyan bg, cyan text
Delivered – green bg, green text
Cancelled – red bg, red text

Product Badges:

NEW – indigo bg
SALE – red bg (e.g., "-30%")
OUT OF STOCK – gray bg
FEATURED – gold border

All badges: rounded-full, px-2, py-0.5, text-xs, font-semibold

7.5 Modals & Dialogs

Backdrop: black/50 overlay, blurred

Modal Panel: white, rounded-xl, shadow-2xl

Max-width: 480px (sm), 640px (md), 768px (lg)

Animation: scale 0.95 → 1 + fade in (150ms ease-out)

Confirm Dialog: smaller variant (max-w-sm)

Title | Description | Cancel + Confirm buttons

7.6 Toast Notifications

Position: top-right (desktop), top-center (mobile)

Auto-dismiss: 4 seconds

Types: success (green) | error (red) | warning (amber) | info (blue)

Animation: slide-in from right + fade out

Structure:

[Icon] [Message] [X close]

7.7 Tables (Dashboard)

Header: gray-50 bg, uppercase, text-xs, letter-spacing

Rows: white bg, hover: gray-50

Borders: bottom border only (gray-200)

Actions: icon buttons on each row (edit, delete, view)

Mobile: horizontal scroll on small screens

Sorting: clickable column headers with sort icon

Pagination: bottom of table, "Previous | 1 2 3 | Next"

7.8 Skeleton Loaders

Product Card Skeleton:

- Gray animated pulse box (image area)
- 3 gray lines (name + price + button)

Table Skeleton:

- Header row normal
- 5 rows of gray animated lines

Dashboard Card Skeleton:

- Gray box (icon area)
- Two gray lines (label + value)

Animation: CSS keyframe pulse (opacity 0.5 → 1 → 0.5, 1.5s infinite)

7.9 Pagination

Style: numbered buttons

← Prev | 1 2 [3] 4 5 ... 12 | Next →

Active page: filled indigo button

Hover: light indigo bg

Disabled: gray, not-allowed cursor

7.10 Star Rating Component

Display mode: 5 stars, partial fill supported (e.g., 4.3 stars)

Input mode: clickable stars (hover highlights up to cursor)

Color: #facc15 (yellow-400) filled, gray-300 empty

Sizes: sm (12px) | md (16px) | lg (20px)

8. Client Website — Page-by-Page UI Brief

8.1 Navbar

Desktop Layout:

```
| [Logo] [Categories ▾] [Search Bar ———] [♥] [🛒 3] [Account ▾] |
```

Mobile Layout (< 768px):

```
| [≡] [Logo] [🔍] [🛒 3] |
```

[Mobile Drawer slides in from left]

Sticky on scroll, adds drop shadow after 50px scroll.

Category dropdown: mega-menu with category grid + featured image.

Cart icon shows badge count (red pill).

Account menu: Login / Register (guest) or Profile / Orders / Logout (user).

8.2 Home Page

Section 1 – Hero Banner

Full-width slider (Swiper.js), 3-5 slides

Each slide: background image + overlay + heading + subtext + CTA button

Autoplay: 4s, fade transition

Section 2 – Category Showcase

Horizontal scrollable cards on mobile

Grid 6 cols on desktop

Each card: circular image + category name below

Section 3 – Featured Products

Heading: "Featured Products"

Grid: 4 cols (desktop), 2 cols (mobile)

Product Cards (see 7.3)

"View All" button → /products

Section 4 – Promotional Banner

Full-width split layout: left (text + CTA), right (product image)

Eye-catching background gradient

Section 5 – Newsletter

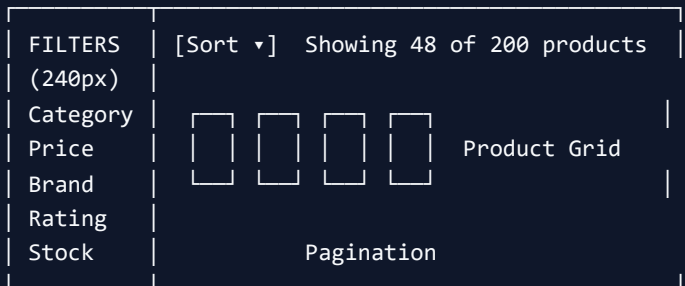
Full-width section, indigo gradient bg

Email input + Subscribe button (inline)

"Join 10,000+ happy customers" social proof

8.3 Product Listing Page

Layout:



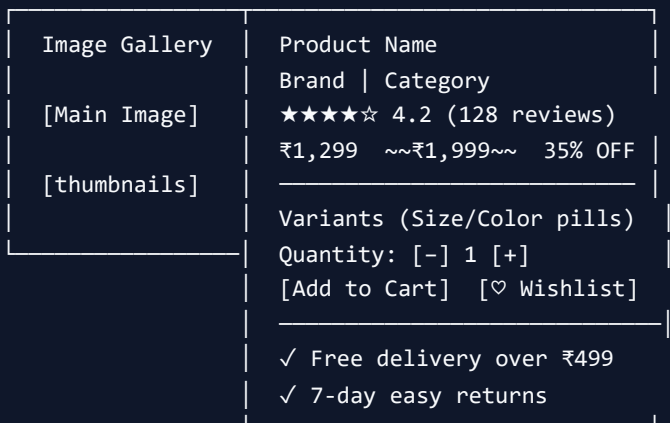
Mobile: Filters behind slide-in drawer (filter icon top-right)

Active filters shown as dismissible chips below search bar

Grid: 4 cols desktop → 3 cols laptop → 2 cols tablet → 2 cols mobile

8.4 Product Detail Page

Layout (Desktop):



Below fold:

Tabs: Description | Specifications | Reviews (128)

Related Products grid

8.5 Cart Page

Layout (Desktop):

Cart Items (left, 65%)	Order Summary (right, 35%)												
<table><tr><td>[img]</td><td>Name</td><td>Qty</td><td>Price</td></tr><tr><td></td><td>Variant</td><td>[-][1][+]</td><td></td></tr><tr><td>[Remove]</td><td>₹1,299</td><td></td><td></td></tr></table>	[img]	Name	Qty	Price		Variant	[-][1][+]		[Remove]	₹1,299			Subtotal
[img]	Name	Qty	Price										
	Variant	[-][1][+]											
[Remove]	₹1,299												
[Apply Coupon code —] [Apply]	Discount												
	Shipping												
	Total												
	[Checkout]												

Mobile: stacked layout (summary at bottom)

8.6 Checkout Page

Multi-step layout (3 steps):

Step indicator: [1 Address] — [2 Review] — [3 Payment]

Step 1 - Shipping Address:

Saved addresses as selectable cards

"Add New Address" expandable form

[Continue →]

Step 2 - Order Review:

Items list (read-only)

Price breakdown

[← Back] [Continue →]

Step 3 - Payment:

Method: ☒ Cash on Delivery ☐ Online (coming soon)

Terms checkbox

[← Back] [Place Order] ← large indigo button

8.7 Profile Pages

Sidebar navigation (desktop), tabs (mobile):

- My Profile (avatar + edit form)
- My Orders (table with status badges)
- Wishlist (product grid)
- My Addresses (address cards with edit/delete/default)

9. Admin Dashboard — Page-by-Page UI Brief

9.1 Dashboard Layout Shell

HEADER: [≡ collapse] [ShopEase Admin][👤 Admin ▼]

SIDE BAR
240px

MAIN CONTENT AREA

Breadcrumb: Dashboard > Products > Add

[icon] Dashboard

[icon] Products

[icon] Orders

[icon] Customers

[icon] Categories

[icon] Coupons

[icon] Analytics

[icon] Settings

Sidebar: dark (--dash-surface), collapses to icon-only mode
Active item: indigo left border + indigo tinted bg
Header: slightly lighter dark bg, has notification bell

9.2 Dashboard Overview Page

Row 1 – Stat Cards (4 cols):
[Total Revenue] [Total Orders] [Total Products] [Total Customers]
Each: icon (colored circle) + value + trend arrow

Row 2 – Charts (2 cols):
Left (70%): Revenue Line Chart (12 months)
Right (30%): Orders by Status Donut Chart

Row 3 – Tables (2 cols):
Left (60%): Recent Orders table (last 10)
Right (40%): Top Products list (top 5 with mini bar)

Row 4 – Alerts:
Low Stock Products list (stock < 10)

9.3 Add/Edit Product Page

Two-column form layout:

Left Column (65%) – Main Info:

- Product Name *
- Slug (auto-generated, editable)
- Short Description
- Full Description (rich text)
- Category * (dropdown)
- Brand, SKU
- Tags (multi-tag input)
- Specifications (dynamic key-value rows)

Right Column (35%) – Media & Settings:

- Image Upload Box (drag & drop, 6 max)
 - shows preview grid after upload
- Price *
- Original Price
- Discount % (auto-calculated)
- Stock *
- Variants (add variant group: e.g., Size → S,M,L,XL)
- Toggle: Active / Inactive
- Toggle: Featured

Bottom: [Cancel] [Save Draft] [Publish Product]

10. Responsive Breakpoints

```
/* Tailwind CSS breakpoints */
xs:   < 480px   (small phones)
sm:   ≥ 640px   (large phones)
md:   ≥ 768px   (tablets)
lg:   ≥ 1024px  (laptops)
xl:   ≥ 1280px  (desktops)
2xl:  ≥ 1536px  (large screens)
```

Responsive Behavior

Element	Mobile	Tablet	Desktop
Product grid	2 cols	3 cols	4 cols
Navbar	hamburger	hamburger	full
Cart layout	stacked	stacked	2-col
Checkout steps	vertical	vertical	horizontal
Dashboard sidebar	hidden (drawer)	icon-only	full
Dashboard stat cards	2×2 grid	2×2 grid	1×4 row
Filters (website)	drawer	sidebar	sidebar

11. Animation & Micro-interactions

Principles

- Duration: 150ms (instant feedback) to 300ms (transitions)
- Easing: `ease-out` for appearing, `ease-in` for disappearing
- Never animate layout-affecting properties (width, height) — use transform/opacity only

Animation Catalog

Element	Animation
Page transition	fade-in (opacity 0→1, 200ms)
Product card hover	translateY(-4px) + shadow, 200ms ease-out
Button hover	background darken + scale(1.02), 150ms
Button click	scale(0.97), 100ms
Modal open	scale(0.95→1) + opacity(0→1), 150ms ease-out
Modal close	opacity(1→0), 100ms ease-in
Toast appear	translateX(100%→0) + opacity, 300ms ease-out
Toast dismiss	opacity(1→0) + translateY(-8px), 200ms
Dropdown open	translateY(-8px→0) + opacity, 150ms
Skeleton loader	pulse keyframe (opacity 0.5↔1, 1.5s infinite)
Sidebar collapse	width 240px→64px, 250ms ease-in-out
Cart badge update	scale(1→1.3→1) bounce, 300ms
Star rating hover	fill color sweeps left→right
Wishlist toggle	heart scale(1→1.4→1) + color change
Image gallery switch	fade cross-dissolve, 200ms
Checkout step change	slide left/right, 250ms ease-out
Success checkmark	draw SVG stroke animation, 400ms

CSS Keyframes


```

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.5; }
}

@keyframes slideInRight {
  from { transform: translateX(100%); opacity: 0; }
  to { transform: translateX(0); opacity: 1; }
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes scaleIn {
  from { transform: scale(0.95); opacity: 0; }
  to { transform: scale(1); opacity: 1; }
}

@keyframes bounceScale {
  0% { transform: scale(1); }
  50% { transform: scale(1.35); }
  100% { transform: scale(1); }
}

```

12. Accessibility Guidelines

WCAG 2.1 AA Requirements

Requirement	Implementation
Color contrast	Minimum 4.5:1 for normal text, 3:1 for large text
Focus visible	Custom focus ring: 2px indigo outline, 2px offset
Keyboard nav	All interactive elements reachable via Tab key
Screen reader	<code>aria-label</code> on icon buttons, <code>alt</code> on images
Form errors	<code>aria-describedby</code> linking input to error message
Loading states	<code>aria-busy="true"</code> + <code>aria-label="Loading..."</code>
Modals	Focus trap inside modal, <code>role="dialog"</code> , <code>aria-modal</code>
Skip links	"Skip to main content" at top of page

Interactive Element Rules

Buttons:

min 44×44px touch target

Links:

descriptive text (no "click here")

Images:

alt="" for decorative, meaningful alt for content images

Videos:

captions (future)

Forms:

every input has associated <label>

13. Image & Media Guidelines

Product Images

Usage	Dimensions	Format	Max Size
Product thumbnail	400×400px	WebP	80KB
Product detail	800×800px	WebP	200KB
Cart / order item	80×80px	WebP	20KB
Hero banner	1440×560px	WebP	300KB
Category card	600×400px	WebP	150KB
User avatar	150×150px	WebP	30KB

Image Rules

- All product images: **square aspect ratio (1:1)**
- Always include `alt` text with product name
- Use `next/image` for automatic optimization in website
- Cloudinary transformations handle format conversion

Placeholder Images

- While loading: gray skeleton (no broken image icon)
- If image fails: product icon on gray background
- Default avatar: initials on colored circle (e.g., "BK" on indigo)

14. Design Tokens (CSS Variables)

```

/* === WEBSITE tokens (globals.css) === */

:root {
  /* Colors */
  --color-primary:      #6366f1;
  --color-primary-dark: #4f46e5;
  --color-accent:       #f59e0b;
  --color-success:      #22c55e;
  --color-error:        #ef4444;
  --color-warning:      #f97316;
  --color-bg:           #f9fafb;
  --color-surface:      #ffffff;
  --color-border:       #e5e7eb;
  --color-text:         #111827;
  --color-text-muted:   #6b7280;

  /* Typography */
  --font-heading: 'Plus Jakarta Sans', sans-serif;
  --font-body:    'Inter', sans-serif;
  --font-mono:    'JetBrains Mono', monospace;

  /* Spacing */
  --space-1: 4px;   --space-2: 8px;
  --space-3: 12px;  --space-4: 16px;
  --space-5: 20px;  --space-6: 24px;
  --space-8: 32px;  --space-10: 40px;
  --space-12: 48px; --space-16: 64px;

  /* Border Radius */
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-xl: 16px;
  --radius-full: 9999px;

  /* Shadows */
  --shadow-sm: 0 1px 2px rgba(0,0,0,0.05);
  --shadow-md: 0 4px 6px rgba(0,0,0,0.07);
  --shadow-lg: 0 10px 15px rgba(0,0,0,0.10);
  --shadow-xl: 0 20px 25px rgba(0,0,0,0.10);
  --shadow-card: 0 2px 8px rgba(99,102,241,0.08);

  /* Transitions */
  --transition-fast: all 150ms ease-out;
  --transition-base: all 200ms ease-out;
  --transition-slow: all 300ms ease-out;

  /* Z-index scale */
  --z-dropdown: 100;
  --z-sticky: 200;
  --z-modal: 300;
  --z-toast: 400;
  --z-tooltip: 500;
}

```

```
/* === DASHBOARD tokens === */
.dashboard {
  --dash-bg:          #0f172a;
  --dash-surface:     #1e293b;
  --dash-surface-2:   #334155;
  --dash-border:      #475569;
  --dash-primary:     #3b82f6;
  --dash-text:        #f1f5f9;
  --dash-text-muted:  #94a3b8;
}
```

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial UI/UX Brief created

Previous Phase → [Phase 3: Application Flow](#)

Next Phase → [Phase 5: Backend Schema](#)

Section 6: Phase 5: Database Schema & Data Models

Backend Schema Document

Project: Production-Ready E-Commerce Platform
Version: 1.0
Status: Planning
Date: July 2026
Phase: 5 of 9

Table of Contents

- 1. [Overview](#)
- 2. [Database Configuration](#)
- 3. [Schema Conventions](#)
- 4. [User Schema](#)
- 5. [Product Schema](#)
- 6. [Category Schema](#)
- 7. [Cart Schema](#)
- 8. [Wishlist Schema](#)
- 9. [Order Schema](#)
- 10. [Review Schema](#)
- 11. [Coupon Schema](#)
- 12. [Database Indexes](#)
- 13. [Schema Relationships](#)
- 14. [Mongoose Model Files](#)
- 15. [Seed Data](#)

1. Overview

The platform uses **MongoDB** (NoSQL) with **Mongoose** as the ODM. All collections are designed for:

- Fast reads via proper indexing
- Minimal data duplication (denormalized only where performance demands)
- Scalability — no joins, references used via `ObjectId`

Collections Summary

Collection	Model Name	Primary Purpose
<code>users</code>	<code>User</code>	Customers + Admins
<code>products</code>	<code>Product</code>	Product catalog
<code>categories</code>	<code>Category</code>	Product categories
<code>carts</code>	<code>Cart</code>	Per-user shopping cart
<code>wishlists</code>	<code>Wishlist</code>	Per-user wishlist
<code>orders</code>	<code>Order</code>	Placed orders
<code>reviews</code>	<code>Review</code>	Product reviews
<code>coupons</code>	<code>Coupon</code>	Discount coupons

2. Database Configuration

2.1 Connection Setup

```
// src/config/db.js
import mongoose from 'mongoose'

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI, {
      dbName: 'ecommerce',
    })
    console.log(`MongoDB Connected: ${conn.connection.host}`)
  } catch (error) {
    console.error(`DB Connection Error: ${error.message}`)
    process.exit(1)
  }
}

export default connectDB
```

2.2 Mongoose Global Settings

```
// In app.js – before connecting
mongoose.set('strictQuery', true) // reject unknown fields
mongoose.set('toJSON', {
  virtuals: true,
  versionKey: false,
  transform: (_, obj) => {
    delete obj.__v
    return obj
  }
})
```

3. Schema Conventions

Convention	Rule
Field names	<code>camelCase</code>
Required fields	Marked with <code>required: [true, 'Error message']</code>
String trim	All string fields use <code>trim: true</code>
Timestamps	All schemas use <code>{ timestamps: true }</code>
Soft delete	<code>isActive</code> boolean flag — never hard delete products/categories
Slug generation	Auto-generated from <code>name</code> using <code>slugify</code>
Version key	Disabled globally (<code>__v</code> removed)
Passwords	Never returned — <code>select: false</code>
Enums	Array of allowed values, <code>lowercase</code> or <code>uppercase</code> enforced

4. User Schema

4.1 Full Schema Definition

```

// src/models/User.model.js
import mongoose from 'mongoose'
import bcrypt from 'bcryptjs'
import crypto from 'crypto'

const addressSchema = new mongoose.Schema(
  {
    label: { type: String, trim: true }, // "Home", "Work", "Other"
    fullName: { type: String, required: true, trim: true },
    phone: { type: String, required: true },
    street: { type: String, required: true, trim: true },
    city: { type: String, required: true, trim: true },
    state: { type: String, required: true, trim: true },
    pincode: { type: String, required: true },
    country: { type: String, default: 'India' },
    isDefault: { type: Boolean, default: false },
  },
  { _id: true }
)

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      minLength: [2, 'Name must be at least 2 characters'],
      maxLength: [50, 'Name cannot exceed 50 characters'],
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email'],
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minLength: [8, 'Password must be at least 8 characters'],
      select: false, // never returned in queries
    },
    phone: {
      type: String,
      trim: true,
    },
    avatar: {
      url: { type: String, default: '' },
      publicId: { type: String, default: '' },
    },
    role: {
      type: String,
      enum: ['customer', 'admin', 'superadmin'],
    }
  }
)

```



```

    default: 'customer',
  },
  addresses: [addressSchema],
  isBlocked: {
    type: Boolean,
    default: false,
  },
  resetPasswordToken: { type: String, select: false },
  resetPasswordExpire: { type: Date, select: false },
},
{ timestamps: true }
)

/* — Pre-save hook: hash password — */
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next()
  this.password = await bcrypt.hash(this.password, 12)
  next()
})

/* — Instance method: compare password — */
userSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password)
}

/* — Instance method: generate password reset token — */
userSchema.methods.getResetPasswordToken = function () {
  const rawToken = crypto.randomBytes(32).toString('hex')
  this.resetPasswordToken = crypto.createHash('sha256').update(rawToken).digest('hex')
  this.resetPasswordExpire = Date.now() + 10 * 60 * 1000 // 10 minutes
  return rawToken
}

/* — Virtual: full avatar URL fallback — */
userSchema.virtual('avatarUrl').get(function () {
  return this.avatar?.url || `https://api.dicebear.com/7.x/initials/svg?seed=${this.name}`
})

const User = mongoose.model('User', userSchema)
export default User

```

4.2 Field Reference

Field	Type	Required	Notes
name	String	✓	2–50 chars
email	String	✓	Unique, lowercase
password	String	✓	Hashed, never selected
phone	String	✗	Optional
avatar.url	String	✗	Cloudinary URL
avatar.publicId	String	✗	For deletion
role	Enum	✗	customer / admin / superadmin
addresses	Array	✗	Embedded sub-documents
isBlocked	Boolean	✗	Default: false
resetPasswordToken	String	✗	SHA-256 hashed, select: false
resetPasswordExpire	Date	✗	10-min TTL

5. Product Schema

5.1 Full Schema Definition

```

// src/models/Product.model.js
import mongoose from 'mongoose'
import slugify from 'slugify'

const variantSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },          // "Size", "Color"
    options: [{ type: String }],                    // ["S","M","L"] or ["Red","Blue"]
  },
  { _id: false }
)

const specificationSchema = new mongoose.Schema(
  {
    key: { type: String, required: true },          // "Material"
    value: { type: String, required: true },        // "100% Cotton"
  },
  { _id: false }
)

const imageSchema = new mongoose.Schema(
  {
    url: { type: String, required: true },
    publicId: { type: String, required: true },
    isMain: { type: Boolean, default: false },
  },
  { _id: true }
)

const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Product name is required'],
      trim: true,
      minLength: [3, 'Name must be at least 3 characters'],
      maxLength: [200, 'Name cannot exceed 200 characters'],
    },
    slug: {
      type: String,
      unique: true,
      index: true,
    },
    description: {
      type: String,
      required: [true, 'Product description is required'],
    },
    shortDescription: {
      type: String,
      maxLength: [300, 'Short description cannot exceed 300 characters'],
    },
    price: {
      type: Number,
      required: [true, 'Price is required'],
    },
  },
  { _id: true }
)

```

```

    min:      [0, 'Price cannot be negative'],
  },
  originalPrice: {
    type: Number,
    min: [0, 'Original price cannot be negative'],
  },
  discount: {
    type: Number,
    min: 0,
    max: 100,
    default: 0,
  },
  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Category',
    required: [true, 'Category is required'],
    index: true,
  },
  brand: {
    type: String,
    trim: true,
  },
  sku: {
    type: String,
    unique: true,
    sparse: true, // allows multiple nulls
    trim: true,
    uppercase: true,
  },
  images: [imageSchema],
  variants: [variantSchema],
  specifications: [specificationSchema],
  tags: [{ type: String, lowercase: true, trim: true }],
  stock: {
    type: Number,
    required: [true, 'Stock quantity is required'],
    min: [0, 'Stock cannot be negative'],
    default: 0,
  },
  isActive: {
    type: Boolean,
    default: true,
    index: true,
  },
  isFeatured: {
    type: Boolean,
    default: false,
  },
  ratings: {
    average: { type: Number, default: 0, min: 0, max: 5 },
    count: { type: Number, default: 0, min: 0 },
  },
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
  },

```

```

    },
  },
  { timestamps: true }
)

/* — Pre-save: auto-generate slug — */
productSchema.pre('save', function (next) {
  if (this.isModified('name')) {
    this.slug = slugify(this.name, { lower: true, strict: true })
  }
  next()
})

/* — Pre-save: auto-calculate discount — */
productSchema.pre('save', function (next) {
  if (this.originalPrice && this.price && this.originalPrice > this.price) {
    this.discount = Math.round(((this.originalPrice - this.price) / this.originalPrice) * 100)
  }
  next()
})

/* — Virtual: stock status — */
productSchema.virtual('stockStatus').get(function () {
  if (this.stock === 0) return 'out_of_stock'
  if (this.stock < 10) return 'low_stock'
  return 'in_stock'
})

/* — Text search index — */
productSchema.index({ name: 'text', description: 'text', tags: 'text', brand: 'text' })

const Product = mongoose.model('Product', productSchema)
export default Product

```

5.2 Field Reference

Field	Type	Required	Notes
name	String	✓	3–200 chars
slug	String	auto	From name, unique
description	String	✓	Full HTML allowed
shortDescription	String	✗	Max 300 chars
price	Number	✓	Current selling price
originalPrice	Number	✗	MRP for strike-through
discount	Number	auto	Auto-calculated %
category	ObjectId	✓	Ref: Category
brand	String	✗	—
sku	String	✗	Auto uppercase, unique
images	Array	✗	Max 6, Cloudinary
variants	Array	✗	Size, Color, etc.
specifications	Array	✗	Key-value pairs
tags	Array	✗	Lowercase strings
stock	Number	✓	Min: 0
isActive	Boolean	✗	Soft delete flag
isFeatured	Boolean	✗	Show on home page
ratings.average	Number	auto	Updated on review save
ratings.count	Number	auto	Updated on review save

6. Category Schema

6.1 Full Schema Definition

```

// src/models/Category.model.js
import mongoose from 'mongoose'
import slugify from 'slugify'

const categorySchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Category name is required'],
      unique: true,
      trim: true,
      minLength: [2, 'Name must be at least 2 characters'],
      maxLength: [50, 'Name cannot exceed 50 characters'],
    },
    slug: {
      type: String,
      unique: true,
    },
    description: {
      type: String,
      trim: true,
    },
    image: {
      url: { type: String, default: '' },
      publicId: { type: String, default: '' },
    },
    parent: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Category',
      default: null, // null = top-level category
    },
    isActive: {
      type: Boolean,
      default: true,
    },
  },
  { timestamps: true }
)

/* — Auto-generate slug — */
categorySchema.pre('save', function (next) {
  if (this.isModified('name')) {
    this.slug = slugify(this.name, { lower: true, strict: true })
  }
  next()
})

/* — Virtual: isSubcategory — */
categorySchema.virtual('isSubcategory').get(function () {
  return this.parent !== null
})

```

```
const Category = mongoose.model('Category', categorySchema)
export default Category
```

6.2 Field Reference

Field	Type	Required	Notes
name	String	✓	Unique, 2–50 chars
slug	String	auto	From name
description	String	✗	—
image.url	String	✗	Cloudinary URL
image.publicId	String	✗	For deletion
parent	ObjectId	✗	Ref: Category (null = top-level)
isActive	Boolean	✗	Default: true

7. Cart Schema

7.1 Full Schema Definition


```
// src/models/Cart.model.js
import mongoose from 'mongoose'

const cartItemSchema = new mongoose.Schema(
  {
    product: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Product',
      required: true,
    },
    name: { type: String, required: true }, // Snapshot at add time
    image: { type: String, default: '' }, // Snapshot at add time
    price: { type: Number, required: true }, // Snapshot at add time
    quantity: {
      type: Number,
      required: true,
      min: [1, 'Quantity must be at least 1'],
      default: 1,
    },
    selectedVariants: {
      type: Map,
      of: String, // { size: "M", color: "Red" }
    },
  },
  { _id: true }
)

const cartSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
      unique: true, // One cart per user
    },
    items: [cartItemSchema],
    coupon: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Coupon',
      default: null,
    },
    discountAmount: {
      type: Number,
      default: 0,
    },
  },
  { timestamps: true }
)

/* — Virtual: item count — */
cartSchema.virtual('itemCount').get(function () {
  return this.items.reduce((total, item) => total + item.quantity, 0)
})
```

```
/* — Virtual: subtotal — */
cartSchema.virtual('subtotal').get(function () {
  return this.items.reduce((total, item) => total + item.price * item.quantity, 0)
})

const Cart = mongoose.model('Cart', cartSchema)
export default Cart
```

7.2 Design Decisions

Why snapshot name/image/price?

Product data can change after item is added to cart. We store a snapshot so the cart always shows the price the user saw when they added it. At checkout, we re-verify current price and stock.

8. Wishlist Schema

8.1 Full Schema Definition

```
// src/models/Wishlist.model.js
import mongoose from 'mongoose'

const wishlistSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
      unique: true, // One wishlist per user
    },
    products: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Product',
      }
    ],
  },
  { timestamps: true }
)

/* — Virtual: product count — */
wishlistSchema.virtual('count').get(function () {
  return this.products.length
})

const Wishlist = mongoose.model('Wishlist', wishlistSchema)
export default Wishlist
```

9. Order Schema

9.1 Full Schema Definition

```

// src/models/Order.model.js
import mongoose from 'mongoose'

const orderItemSchema = new mongoose.Schema(
  {
    product: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Product',
    },
    name: { type: String, required: true }, // Snapshot
    image: { type: String, default: '' }, // Snapshot
    price: { type: Number, required: true }, // Price at purchase
    quantity: { type: Number, required: true, min: 1 },
    selectedVariants: {
      type: Map,
      of: String,
    },
  },
  { _id: true }
)

const shippingAddressSchema = new mongoose.Schema(
  {
    fullName: { type: String, required: true },
    phone: { type: String, required: true },
    street: { type: String, required: true },
    city: { type: String, required: true },
    state: { type: String, required: true },
    pincode: { type: String, required: true },
    country: { type: String, default: 'India' },
  },
  { _id: false }
)

const statusHistorySchema = new mongoose.Schema(
  {
    status: { type: String, required: true },
    note: { type: String, default: '' },
    updatedAt: { type: Date, default: Date.now },
    updatedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  },
  { _id: false }
)

const orderSchema = new mongoose.Schema(
  {
    orderNumber: {
      type: String,
      unique: true,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
  },
  { _id: true }
)

```

```

    index:    true,
  },
  items: [orderItemSchema],

  shippingAddress: shippingAddressSchema,

  paymentMethod: {
    type:      String,
    enum:      ['COD', 'ONLINE'],
    required: true,
  },
  paymentStatus: {
    type:      String,
    enum:      ['pending', 'paid', 'failed', 'refunded'],
    default: 'pending',
  },
  orderStatus: {
    type:      String,
    enum:      ['pending', 'processing', 'shipped', 'delivered', 'cancelled'],
    default: 'pending',
    index:    true,
  },
  statusHistory: [statusHistorySchema],

  coupon: {
    type:      mongoose.Schema.Types.ObjectId,
    ref:       'Coupon',
    default: null,
  },

  /* — Financials — */
  subtotal:      { type: Number, required: true },
  shippingCharge: { type: Number, default: 0 },
  taxAmount:      { type: Number, default: 0 },
  discountAmount: { type: Number, default: 0 },
  totalAmount:    { type: Number, required: true },

  /* — Dates — */
  estimatedDelivery: { type: Date },
  deliveredAt:        { type: Date },
  cancelledAt:         { type: Date },
  cancelReason:        { type: String, trim: true },
},
{ timestamps: true }
)

/* — Pre-save: generate order number — */
orderSchema.pre('save', async function (next) {
  if (!this.orderNumber) {
    const date   = new Date()
    const ymd    = `${date.getFullYear()}${String(date.getMonth()+1).padStart(2, '0')}${String(date.ge
    const count  = await mongoose.model('Order').countDocuments() + 1
    const serial = String(count).padStart(4, '0')
    this.orderNumber = `ORD-${ymd}-${serial}`
  }
}

```

```

    next()
  })

  /* — Pre-save: push to status history on status change — */
  orderSchema.pre('save', function (next) {
    if (this.isModified('orderStatus')) {
      this.statusHistory.push({ status: this.orderStatus })
    }
    next()
  })

  /* — Virtual: item count — */
  orderSchema.virtual('itemCount').get(function () {
    return this.items.reduce((sum, item) => sum + item.quantity, 0)
  })

  const Order = mongoose.model('Order', orderSchema)
  export default Order

```

9.2 Order Number Format

ORD-YYYYMMDD-XXXX

Examples:

ORD-20260701-0001 (first order)
 ORD-20260701-0127 (127th order)

9.3 Financial Calculation Logic

```

// In order.controller.js – before creating order

const TAX_RATE          = 0.18           // 18% GST
const FREE_SHIPPING_THRESHOLD = 499       // Free shipping above ₹499

const subtotal          = items.reduce((sum, i) => sum + i.price * i.quantity, 0)
const discountAmount    = couponApplied ? calculateDiscount(coupon, subtotal) : 0
const taxableAmount     = subtotal - discountAmount
const taxAmount         = parseFloat((taxableAmount * TAX_RATE).toFixed(2))
const shippingCharge    = taxableAmount >= FREE_SHIPPING_THRESHOLD ? 0 : 99
const totalAmount       = taxableAmount + taxAmount + shippingCharge

```

10. Review Schema

10.1 Full Schema Definition

```

// src/models/Review.model.js
import mongoose from 'mongoose'

const reviewSchema = new mongoose.Schema(
  {
    product: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Product',
      required: true,
      index: true,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    rating: {
      type: Number,
      required: [true, 'Rating is required'],
      min: [1, 'Rating must be at least 1'],
      max: [5, 'Rating cannot exceed 5'],
    },
    title: {
      type: String,
      trim: true,
      maxLength: [100, 'Title cannot exceed 100 characters'],
    },
    comment: {
      type: String,
      required: [true, 'Review comment is required'],
      trim: true,
      minLength: [10, 'Comment must be at least 10 characters'],
      maxLength: [1000, 'Comment cannot exceed 1000 characters'],
    },
    isVerifiedPurchase: {
      type: Boolean,
      default: false,
    },
  },
  { timestamps: true }
)

/* — Compound unique: one review per user per product — */
reviewSchema.index({ product: 1, user: 1 }, { unique: true })

/* — Post-save: update product ratings — */
reviewSchema.post('save', async function () {
  await updateProductRatings(this.product)
})

/* — Post-remove: update product ratings — */
reviewSchema.post('findOneAndDelete', async function (doc) {
  if (doc) await updateProductRatings(doc.product)
})

```

```

/* — Helper: recalculate average rating — */
async function updateProductRatings(productId) {
  const stats = await mongoose.model('Review').aggregate([
    { $match: { product: productId } },
    {
      $group: {
        _id: '$product',
        average: { $avg: '$rating' },
        count: { $sum: 1 },
      }
    }
  ])

  await mongoose.model('Product').findByIdAndUpdate(productId, {
    'ratings.average': stats.length > 0 ? Math.round(stats[0].average * 10) / 10 : 0,
    'ratings.count': stats.length > 0 ? stats[0].count : 0,
  })
}

const Review = mongoose.model('Review', reviewSchema)
export default Review

```

10.2 Verified Purchase Logic

```

// In review.controller.js – before saving review

const hasPurchased = await Order.findOne({
  user: req.user._id,
  'items.product': productId,
  orderStatus: 'delivered',
})

review.isVerifiedPurchase = !!hasPurchased

```

11. Coupon Schema

11.1 Full Schema Definition


```
// src/models/Coupon.model.js
import mongoose from 'mongoose'

const couponSchema = new mongoose.Schema(
  {
    code: {
      type: String,
      required: [true, 'Coupon code is required'],
      unique: true,
      uppercase: true,
      trim: true,
      minLength: [3, 'Code must be at least 3 characters'],
      maxLength: [20, 'Code cannot exceed 20 characters'],
    },
    discountType: {
      type: String,
      enum: ['percentage', 'flat'],
      required: [true, 'Discount type is required'],
    },
    discountValue: {
      type: Number,
      required: [true, 'Discount value is required'],
      min: [1, 'Discount value must be at least 1'],
    },
    minOrderAmount: {
      type: Number,
      default: 0,
    },
    maxDiscount: {
      type: Number, // Cap for percentage discounts
      default: null,
    },
    usagelimit: {
      type: Number,
      default: null, // null = unlimited
    },
    usedCount: {
      type: Number,
      default: 0,
    },
    perUserLimit: {
      type: Number,
      default: 1,
    },
    usedBy: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
      }
    ],
    isActive: {
      type: Boolean,
      default: true,
    },
  },

```

```

    expiresAt: {
      type: Date,
      required: [true, 'Expiry date is required'],
    },
  },
  { timestamps: true }
)

/* — Virtual: isExpired — */
couponSchema.virtual('isExpired').get(function () {
  return new Date() > this.expiresAt
})

/* — Virtual: isUsageLimitReached — */
couponSchema.virtual('isUsageLimitReached').get(function () {
  return this.usageLimit !== null && this.usedCount >= this.usageLimit
})

/* — Instance method: validate coupon for a user and order — */
couponSchema.methods.validateForUser = function (userId, orderAmount) {
  const errors = []

  if (!this.isActive) errors.push('This coupon is inactive')
  if (this.isExpired) errors.push('This coupon has expired')
  if (this.isUsageLimitReached) errors.push('This coupon usage limit has been reached')
  if (orderAmount < this.minOrderAmount)
    errors.push(`Minimum order amount of ₹${this.minOrderAmount} required`)

  const userUsageCount = this.usedBy.filter(id => id.toString() === userId.toString()).length
  if (userUsageCount >= this.perUserLimit)
    errors.push('You have already used this coupon')

  return errors
}

/* — Instance method: calculate discount amount — */
couponSchema.methods.calculateDiscount = function (orderAmount) {
  let discount = 0

  if (this.discountType === 'flat') {
    discount = Math.min(this.discountValue, orderAmount)
  } else {
    discount = (orderAmount * this.discountValue) / 100
    if (this.maxDiscount) {
      discount = Math.min(discount, this.maxDiscount)
    }
  }

  return Math.round(discount * 100) / 100
}

const Coupon = mongoose.model('Coupon', couponSchema)
export default Coupon

```

11.2 Coupon Examples

Code	Type	Value	Min Order	Max Discount	Per User
SAVE10	percentage	10%	₹500	₹200	1
FLAT50	flat	₹50	₹300	—	2
WELCOME20	percentage	20%	₹0	₹500	1
FREESHIP	flat	₹99	₹200	—	3

12. Database Indexes

```

// All indexes to be created (auto via schema + manual)

// — users —
db.users.createIndex({ email: 1 }, { unique: true })
db.users.createIndex({ role: 1 })
db.users.createIndex({ isBlocked: 1 })

// — products —
db.products.createIndex({ slug: 1 }, { unique: true })
db.products.createIndex({ category: 1 })
db.products.createIndex({ isActive: 1, isFeatured: 1 })
db.products.createIndex({ isActive: 1, price: 1 })
db.products.createIndex({ 'ratings.average': -1 })
db.products.createIndex({ stock: 1 })
db.products.createIndex({ name: 'text', description: 'text', tags: 'text', brand: 'text' })

// — categories —
db.categories.createIndex({ slug: 1 }, { unique: true })
db.categories.createIndex({ parent: 1 })
db.categories.createIndex({ isActive: 1 })

// — orders —
db.orders.createIndex({ orderNumber: 1 }, { unique: true })
db.orders.createIndex({ user: 1, createdAt: -1 })
db.orders.createIndex({ orderStatus: 1 })
db.orders.createIndex({ createdAt: -1 })

// — reviews —
db.reviews.createIndex({ product: 1, user: 1 }, { unique: true })
db.reviews.createIndex({ product: 1, createdAt: -1 })

// — coupons —
db.coupons.createIndex({ code: 1 }, { unique: true })
db.coupons.createIndex({ isActive: 1, expiresAt: 1 })

// — carts —
db.carts.createIndex({ user: 1 }, { unique: true })

// — wishlists —
db.wishlists.createIndex({ user: 1 }, { unique: true })

```

13. Schema Relationships

```

users (1) ————— (1) carts
users (1) ————— (1) wishlists
users (1) ————— (N) orders
users (1) ————— (N) reviews

products (1) ————— (N) reviews
products (N) ————— (1) categories
products (N) ————— (N) carts    [via cartItem.product]
products (N) ————— (N) wishlists [via wishlist.products[]]
products (N) ————— (N) orders    [via orderItem.product]

categories (1) ————— (N) products
categories (1) ————— (N) categories [self-ref: parent]

coupons (1) ————— (N) orders    [via order.coupon]
coupons (1) ————— (N) carts      [via cart.coupon]
coupons (N) ————— (N) users      [via coupon.usedBy[]]

```

Populate Patterns

```

// Product with category
Product.findById(id).populate('category', 'name slug image')

// Order with user + items.product
Order.findById(id)
  .populate('user', 'name email phone')
  .populate('items.product', 'name slug images')

// Review with user
Review.find({ product: id })
  .populate('user', 'name avatarUrl')
  .sort({ createdAt: -1 })

// Cart with product details
Cart.findOne({ user: userId })
  .populate('items.product', 'name price stock isActive images')
  .populate('coupon', 'code discountType discountValue')

```

14. Mongoose Model Files

File Structure Summary

```
src/models/
├─ User.model.js      ← users collection
├─ Product.model.js   ← products collection
├─ Category.model.js  ← categories collection
├─ Cart.model.js      ← carts collection
├─ Wishlist.model.js  ← wishlists collection
├─ Order.model.js     ← orders collection
├─ Review.model.js    ← reviews collection
└─ Coupon.model.js    ← coupons collection
```

Model Export Pattern

```
// Every model file ends with:
const ModelName = mongoose.model('ModelName', modelNameSchema)
export default ModelName

// Import in controllers:
import User      from '../models/User.model.js'
import Product   from '../models/Product.model.js'
import Order     from '../models/Order.model.js'
```

15. Seed Data

15.1 Seed Script Location

```
backend/
├─ src/
│   └─ utils/
│       └─ seed.js      ← Run: node src/utils/seed.js
```

15.2 Seed Categories

```
const categories = [
  { name: 'Electronics', slug: 'electronics' },
  { name: 'Fashion', slug: 'fashion' },
  { name: 'Home & Kitchen', slug: 'home-kitchen' },
  { name: 'Books', slug: 'books' },
  { name: 'Sports', slug: 'sports' },
  { name: 'Beauty', slug: 'beauty' },

  // Sub-categories
  { name: 'Mobiles', slug: 'mobiles', parent: 'electronics' },
  { name: 'Laptops', slug: 'laptops', parent: 'electronics' },
  { name: 'Men\'s Wear', slug: 'mens-wear', parent: 'fashion' },
  { name: 'Women\'s Wear', slug: 'womens-wear', parent: 'fashion' },
]
```

15.3 Seed Admin User

```
const adminUser = {
  name: 'Super Admin',
  email: 'admin@shopease.com',
  password: 'Admin@123456', // Hashed by pre-save hook
  role: 'superadmin',
}
```

15.4 Seed Products (Sample)

```
const sampleProducts = [
  {
    name: 'Wireless Bluetooth Headphones',
    price: 1299,
    originalPrice: 1999,
    stock: 50,
    isFeatured: true,
    // ... other fields
  },
  {
    name: 'Running Shoes',
    price: 2499,
    originalPrice: 3499,
    stock: 30,
    isFeatured: true,
  },
  // ... 8 more sample products
]
```

15.5 Seed Coupons

```
const seedCoupons = [
  {
    code: 'WELCOME20',
    discountType: 'percentage',
    discountValue: 20,
    maxDiscount: 500,
    perUserLimit: 1,
    expiresAt: new Date('2027-01-01'),
  },
  {
    code: 'FLAT100',
    discountType: 'flat',
    discountValue: 100,
    minOrderAmount: 500,
    expiresAt: new Date('2027-01-01'),
  },
]
```

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial Backend Schema Document

Previous Phase → [Phase 4: UI/UX Brief](#)

Next Phase → [Phase 6: API Documentation](#)

Section 7: Phase 6: API Documentation & Endpoints

API Documentation

Project: Production-Ready E-Commerce Platform

Version: 1.0

Status: Planning

Date: July 2026

Phase: 6 of 9

Table of Contents

- 1. [API Overview](#)
- 2. [Authentication](#)
- 3. [Auth Endpoints](#)
- 4. [User Endpoints](#)
- 5. [Product Endpoints](#)
- 6. [Category Endpoints](#)
- 7. [Cart Endpoints](#)
- 8. [Wishlist Endpoints](#)
- 9. [Order Endpoints](#)
- 10. [Review Endpoints](#)
- 11. [Coupon Endpoints](#)
- 12. [Upload Endpoints](#)
- 13. [Analytics Endpoints](#)
- 14. [Error Reference](#)
- 15. [Postman Collection](#)

1. API Overview

Base URL

```
Development: http://localhost:5000/api
Production:  https://api.shopease.com/api
```

Request Headers

```
Content-Type: application/json
Cookie:      accessToken=<JWT> (auto-sent by browser)
```

Standard Response Envelope

Success:

```
{
  "success": true,
  "message": "Description of result",
  "data": { },
  "pagination": {
    "currentPage": 1,
    "totalPages": 10,
    "totalItems": 98,
    "itemsPerPage": 10
  }
}
```

Error:

```
{
  "success": false,
  "message": "Human-readable error",
  "errors": [
    { "field": "email", "message": "Email is required" }
  ]
}
```

Access Levels

Symbol	Meaning
	Public — no auth required
	Customer — must be logged in
	Admin — must have admin/superadmin role
	Super Admin only

2. Authentication

All protected routes require a valid JWT `accessToken` sent as an HTTP-only cookie.

Cookie Names

Cookie	Expiry	Purpose
<code>accessToken</code>	15 minutes	API authorization
<code>refreshToken</code>	7 days	Get new access token

Token Refresh

If an `accessToken` expires, the middleware automatically uses `refreshToken` to issue a new one. If `refreshToken` is also invalid/expired, the user must re-login.

3. Auth Endpoints

POST `/auth/register` 

Register a new customer account.

Request Body:

```
{
  "name": "Brijesh Kumar",
  "email": "brijesh@example.com",
  "password": "Password@123",
  "confirmPassword": "Password@123"
}
```

Validation Rules:

Field	Rule
name	Required, 2–50 chars
email	Required, valid email format
password	Required, min 8 chars, must contain uppercase + number
confirmPassword	Must match password

Success Response — 201:

```
{
  "success": true,
  "message": "Account created successfully",
  "data": {
    "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
    "name": "Brijesh Kumar",
    "email": "brijesh@example.com",
    "role": "customer",
    "createdAt": "2026-07-01T04:30:00.000Z"
  }
}
```

Sets `accessToken` + `refreshToken` HTTP-only cookies.

Error Responses:

Code	Message
400	Validation errors
409	Email already registered

POST `/auth/login`

Login with email and password.

Request Body:

```
{
  "email": "brijesh@example.com",
  "password": "Password@123"
}
```

Success Response — 200:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
    "name": "Brijesh Kumar",
    "email": "brijesh@example.com",
    "role": "customer",
    "avatar": { "url": "", "publicId": "" }
  }
}
```

Sets `accessToken` + `refreshToken` HTTP-only cookies.

Error Responses:

Code	Message
400	Invalid credentials
403	Account is blocked

POST `/auth/logout`

Logout and clear auth cookies.

Success Response — 200:

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

Clears `accessToken` + `refreshToken` cookies.

GET `/auth/me`

Get currently logged-in user profile.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
    "name": "Brijesh Kumar",
    "email": "brijesh@example.com",
    "phone": "9876543210",
    "role": "customer",
    "avatar": { "url": "https://res.cloudinary.com/...", "publicId": "ecommerce/avatars/abc" },
    "addresses": [ ],
    "createdAt": "2026-07-01T04:30:00.000Z"
  }
}
```

POST `/auth/forgot-password`

Send password reset email.

Request Body:

```
{ "email": "brijesh@example.com" }
```

Success Response — 200:

```
{
  "success": true,
  "message": "If this email is registered, a reset link has been sent."
}
```

Always returns 200 (security: does not reveal if email exists).

POST `/auth/reset-password`

Reset password using token from email link.

Request Body:

```
{
  "token": "a3f7b2c1d4e5...",
  "password": "NewPassword@123",
  "confirmPassword": "NewPassword@123"
}
```

Success Response — 200:

```
{
  "success": true,
  "message": "Password reset successful. Please login."
}
```

Error Responses:

Code	Message
400	Token is invalid or has expired
400	Passwords do not match

POST `/auth/refresh-token`

Get a new access token using refresh token cookie.

Success Response — 200:

```
{
  "success": true,
  "message": "Token refreshed"
}
```

Sets new `accessToken` cookie.

4. User Endpoints

GET `/users`

Get all users (admin only). Supports pagination + filters.

Query Parameters:

Param	Type	Description
<code>page</code>	Number	Page number (default: 1)
<code>limit</code>	Number	Items per page (default: 20)
<code>role</code>	String	Filter: customer / admin
<code>isBlocked</code>	Boolean	Filter by block status
<code>search</code>	String	Search by name or email

Success Response — 200:

```
{
  "success": true,
  "data": [
    {
      "_id": "64f1...",
      "name": "Brijesh Kumar",
      "email": "brijesh@example.com",
      "phone": "9876543210",
      "role": "customer",
      "isBlocked": false,
      "createdAt": "2026-07-01T04:30:00.000Z"
    }
  ],
  "pagination": { "currentPage": 1, "totalPages": 5, "totalItems": 98, "itemsPerPage": 20 }
}
```

GET `/users/:id` 

Get a single user. Customers can only get their own profile. Admins can get any.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "_id": "64f1...",
    "name": "Brijesh Kumar",
    "email": "brijesh@example.com",
    "phone": "9876543210",
    "avatar": { "url": "...", "publicId": "..." },
    "addresses": [
      {
        "_id": "65a2...",
        "label": "Home",
        "fullName": "Brijesh Kumar",
        "phone": "9876543210",
        "street": "123 Main St",
        "city": "Ahmedabad",
        "state": "Gujarat",
        "pincode": "380001",
        "country": "India",
        "isDefault": true
      }
    ]
  },
  "createdAt": "2026-07-01T04:30:00.000Z"
}
```


PUT `/users/:id`

Update user profile. Customers update own profile only.

Request Body:

```
{
  "name": "Brijesh K.",
  "phone": "9876543210"
}
```

Success Response — 200:

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "_id": "...", "name": "Brijesh K.", ... }
}
```

PUT `/users/:id/password`

Change user password.

Request Body:

```
{
  "currentPassword": "OldPassword@123",
  "newPassword": "NewPassword@456",
  "confirmPassword": "NewPassword@456"
}
```

Success Response — 200:

```
{ "success": true, "message": "Password changed successfully" }
```

PUT `/users/:id/block`

Block or unblock a customer account.

Request Body:

```
{ "isBlocked": true }
```

Success Response — 200:

```
{ "success": true, "message": "User has been blocked" }
```

POST `/users/:id/addresses`

Add a new address.

Request Body:

```
{
  "label": "Home",
  "fullName": "Brijesh Kumar",
  "phone": "9876543210",
  "street": "123 Main Street",
  "city": "Ahmedabad",
  "state": "Gujarat",
  "pincode": "380001",
  "isDefault": true
}
```

Success Response — 201:

```
{
  "success": true,
  "message": "Address added successfully",
  "data": { "_id": "...", "label": "Home", ... }
}
```

PUT `/users/:id/addresses/:addressId`

Update an existing address.

Request Body: *(same as POST, all fields optional)*

Success Response — 200:

```
{ "success": true, "message": "Address updated successfully" }
```

DELETE `/users/:id/addresses/:addressId`

Delete an address.

Success Response — 200:

```
{ "success": true, "message": "Address deleted successfully" }
```

PATCH `/users/:id/addresses/:addressId/default`

Set an address as default.

Success Response — 200:

```
{ "success": true, "message": "Default address updated" }
```

5. Product Endpoints

GET `/products`

Get all active products with filters, sorting, and pagination.

Query Parameters:

Param	Type	Example	Description
----- ----- ----- -----			
page	Number	1	Page number
limit	Number	12	Items per page
search	String	phone	Full-text search
category	String	64f1...	Category ObjectId
minPrice	Number	500	Minimum price
maxPrice	Number	5000	Maximum price
brand	String	Samsung	Filter by brand
minRating	Number	4	Minimum average rating
inStock	Boolean	true	Only in-stock items
isFeatured	Boolean	true	Only featured items
sort	String	price	Sort field
order	String	asc asc or desc	
tags	String	wireless	Filter by tag

Success Response — 200:

```
{
  "success": true,
  "data": [
    {
      "_id": "64f1a2...",
      "name": "Wireless Bluetooth Headphones",
      "slug": "wireless-bluetooth-headphones",
      "price": 1299,
      "originalPrice": 1999,
      "discount": 35,
      "category": { "_id": "...", "name": "Electronics", "slug": "electronics" },
      "brand": "SoundMax",
      "images": [{ "url": "https://res.cloudinary.com/...", "isMain": true }],
      "stock": 50,
      "stockStatus": "in_stock",
      "ratings": { "average": 4.3, "count": 128 },
      "isFeatured": true,
      "createdAt": "2026-07-01T04:30:00.000Z"
    }
  ],
  "pagination": { "currentPage": 1, "totalPages": 8, "totalItems": 96, "itemsPerPage": 12 }
}
```

GET `/products/:slug` 

Get a single product by slug.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "_id": "64f1a2...",
    "name": "Wireless Bluetooth Headphones",
    "slug": "wireless-bluetooth-headphones",
    "description": "<p>Full product description...</p>",
    "shortDescription": "Premium sound with 30hr battery life.",
    "price": 1299,
    "originalPrice": 1999,
    "discount": 35,
    "category": { "_id": "...", "name": "Electronics", "slug": "electronics" },
    "brand": "SoundMax",
    "sku": "SMAX-BT-001",
    "images": [
      { "_id": "...", "url": "https://...", "publicId": "ecommerce/products/abc", "isMain": true },
      { "_id": "...", "url": "https://...", "publicId": "ecommerce/products/def", "isMain": false }
    ],
    "variants": [
      { "name": "Color", "options": ["Black", "White", "Navy"] }
    ],
    "specifications": [
      { "key": "Driver Size", "value": "40mm" },
      { "key": "Battery", "value": "30 hours" },
      { "key": "Connectivity", "value": "Bluetooth 5.0" }
    ],
    "tags": ["wireless", "headphones", "bluetooth"],
    "stock": 50,
    "stockStatus": "in_stock",
    "ratings": { "average": 4.3, "count": 128 },
    "isFeatured": true,
    "isActive": true,
    "createdAt": "2026-07-01T04:30:00.000Z"
  }
}
```

Error Responses:

| Code | Message |

|-----|-----|

| 404 | Product not found |

POST `/products` 

Create a new product.

Request Body:

```
{
  "name": "Wireless Bluetooth Headphones",
  "description": "<p>Full description...</p>",
  "shortDescription": "Premium sound with 30hr battery.",
  "price": 1299,
  "originalPrice": 1999,
  "category": "64f1a2b3c4d5e6f7a8b9c0d1",
  "brand": "SoundMax",
  "sku": "SMAX-BT-001",
  "stock": 50,
  "isFeatured": true,
  "tags": ["wireless", "headphones"],
  "variants": [
    { "name": "Color", "options": ["Black", "White"] }
  ],
  "specifications": [
    { "key": "Battery", "value": "30 hours" }
  ],
  "images": [
    { "url": "https://res.cloudinary.com/...", "publicId": "ecommerce/products/abc", "isMain": true }
  ]
}
```

Success Response — 201:

```
{
  "success": true,
  "message": "Product created successfully",
  "data": { "_id": "...", "name": "Wireless Bluetooth Headphones", "slug": "wireless-bluetooth-headph"
}
```

PUT `/products/:id`

Update a product. Send only fields to update.

Request Body: *(any product fields, all optional)*

Success Response — 200:

```
{
  "success": true,
  "message": "Product updated successfully",
  "data": { ... }
}
```

DELETE `/products/:id`

Soft-delete a product (sets `isActive: false`).

Success Response — 200:

```
{ "success": true, "message": "Product deleted successfully" }
```

PATCH `/products/:id/status`

Toggle product active/inactive status.

Request Body:

```
{ "isActive": false }
```

Success Response — 200:

```
{ "success": true, "message": "Product status updated" }
```

GET `/products/featured`

Get featured products for home page (max 8).

Success Response — 200:

```
{  
  "success": true,  
  "data": [ ... ]  
}
```

6. Category Endpoints

GET `/categories`

Get all active categories (with sub-categories nested).

Success Response — 200:

```
{
  "success": true,
  "data": [
    {
      "_id": "64f1...",
      "name": "Electronics",
      "slug": "electronics",
      "description": "All electronic items",
      "image": { "url": "https://...", "publicId": "..." },
      "parent": null,
      "children": [
        { "_id": "65a2...", "name": "Mobiles", "slug": "mobiles", "parent": "64f1..." },
        { "_id": "65a3...", "name": "Laptops", "slug": "laptops", "parent": "64f1..." }
      ]
    }
  ]
}
```

GET `/categories/:slug`

Get single category by slug.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "_id": "64f1...",
    "name": "Electronics",
    "slug": "electronics",
    "description": "...",
    "image": { "url": "...", "publicId": "..." },
    "parent": null,
    "isActive": true
  }
}
```

POST `/categories`

Create a new category.

Request Body:


```
{
  "name": "Electronics",
  "description": "All electronic gadgets",
  "image": { "url": "https://...", "publicId": "ecommerce/categories/abc" },
  "parent": null
}
```

Success Response — 201:

```
{
  "success": true,
  "message": "Category created successfully",
  "data": { "_id": "...", "name": "Electronics", "slug": "electronics", ... }
}
```

PUT `/categories/:id`

Update a category.

Success Response — 200:

```
{ "success": true, "message": "Category updated successfully", "data": { ... } }
```

DELETE `/categories/:id`

Delete a category (only if no products are assigned).

Success Response — 200:

```
{ "success": true, "message": "Category deleted successfully" }
```

Error Responses:

Code	Message
400	Cannot delete: category has products assigned

7. Cart Endpoints

GET `/cart`

Get the current user's cart.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "_id": "64f1...",
    "user": "64f0...",
    "items": [
      {
        "_id": "65a1...",
        "product": { "_id": "...", "name": "Headphones", "slug": "...", "stock": 50, "isActive": true },
        "name": "Wireless Bluetooth Headphones",
        "image": "https://...",
        "price": 1299,
        "quantity": 2,
        "selectedVariants": { "color": "Black" }
      }
    ],
    "coupon": null,
    "discountAmount": 0,
    "itemCount": 2,
    "subtotal": 2598
  }
}
```

POST `/cart/add`

Add a product to cart (or increase quantity if already exists).

Request Body:

```
{
  "productId": "64f1a2b3c4d5e6f7a8b9c0d1",
  "quantity": 1,
  "selectedVariants": { "color": "Black" }
}
```

Success Response — 200:

```
{
  "success": true,
  "message": "Item added to cart",
  "data": { "itemCount": 3, "subtotal": 3897 }
}
```

Error Responses:

| Code | Message |

|-----|-----|

| 404 | Product not found |

| 400 | Product is out of stock |

| 400 | Requested quantity exceeds available stock |

PUT `/cart/update`

Update item quantity in cart.

Request Body:

```
{
  "cartItemId": "65a1...",
  "quantity": 3
}
```

Success Response — 200:

```
{
  "success": true,
  "message": "Cart updated",
  "data": { "itemCount": 3, "subtotal": 3897 }
}
```

DELETE `/cart/remove/:cartItemId`

Remove a specific item from cart.

Success Response — 200:

```
{ "success": true, "message": "Item removed from cart", "data": { "itemCount": 1, "subtotal": 1299 } }
```

DELETE `/cart/clear`

Clear all items from cart.

Success Response — 200:

```
{ "success": true, "message": "Cart cleared" }
```

POST `/cart/merge`

Merge guest cart (localStorage) with server cart on login.

Request Body:

```
{
  "guestCart": [
    { "productId": "64f1...", "quantity": 2, "selectedVariants": { "color": "Black" } }
  ]
}
```

Success Response — 200:

```
{ "success": true, "message": "Cart merged successfully", "data": { "itemCount": 3, "subtotal": 3897
```

8. Wishlist Endpoints

GET `/wishlist`

Get the current user's wishlist with product details.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "_id": "64f1...",
    "user": "64f0...",
    "products": [
      {
        "_id": "65a1...",
        "name": "Wireless Headphones",
        "slug": "wireless-headphones",
        "price": 1299,
        "originalPrice": 1999,
        "discount": 35,
        "images": [{ "url": "https://...", "isMain": true }],
        "ratings": { "average": 4.3, "count": 128 },
        "stock": 50,
        "stockStatus": "in_stock"
      }
    ],
    "count": 1
  }
}
```

POST `/wishlist/add`

Add a product to wishlist.

Request Body:

```
{ "productId": "64f1a2b3c4d5e6f7a8b9c0d1" }
```

Success Response — 200:

```
{ "success": true, "message": "Added to wishlist", "data": { "count": 3 } }
```

DELETE `/wishlist/remove/:productId`

Remove a product from wishlist.

Success Response — 200:

```
{ "success": true, "message": "Removed from wishlist", "data": { "count": 2 } }
```

9. Order Endpoints

POST `/orders`

Place a new order.

Request Body:

```
{
  "shippingAddress": {
    "fullName": "Brijesh Kumar",
    "phone": "9876543210",
    "street": "123 Main Street",
    "city": "Ahmedabad",
    "state": "Gujarat",
    "pincode": "380001",
    "country": "India"
  },
  "paymentMethod": "COD",
  "couponCode": "SAVE10"
}
```

(Items are taken from user's server cart)

Success Response — 201:

```
{
  "success": true,
  "message": "Order placed successfully",
  "data": {
    "_id": "64f1...",
    "orderNumber": "ORD-20260701-0001",
    "orderStatus": "pending",
    "paymentMethod": "COD",
    "paymentStatus": "pending",
    "items": [ ... ],
    "shippingAddress": { ... },
    "subtotal": 2598,
    "shippingCharge": 0,
    "taxAmount": 467.64,
    "discountAmount": 259.8,
    "totalAmount": 2805.84,
    "estimatedDelivery": "2026-07-08T00:00:00.000Z",
    "createdAt": "2026-07-01T04:30:00.000Z"
  }
}
```

GET `/orders`

Get all orders (admin). Supports filters and pagination.

Query Parameters:

Param	Type	Description
----- ----- -----		
page	Number	Page number
limit	Number	Items per page (default: 20)
status	String	Filter by orderStatus
paymentStatus	String	Filter by paymentStatus
startDate	Date	From date
endDate	Date	To date
search	String	Search by orderNumber or customer name

Success Response — 200:

```
{
  "success": true,
  "data": [
    {
      "_id": "64f1...",
      "orderNumber": "ORD-20260701-0001",
      "user": { "_id": "...", "name": "Brijesh Kumar", "email": "brijesh@example.com" },
      "itemCount": 2,
      "totalAmount": 2805.84,
      "orderStatus": "pending",
      "paymentMethod": "COD",
      "createdAt": "2026-07-01T04:30:00.000Z"
    }
  ],
  "pagination": { ... }
}
```

GET `/orders/my-orders`

Get logged-in customer's own orders.

Query Parameters: `page`, `limit`, `status`

Success Response — 200:

```
{
  "success": true,
  "data": [ ... ],
  "pagination": { ... }
}
```

GET `/orders/:id`

Get full order details. Customers can only view own orders.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "_id": "64f1...",
    "orderNumber": "ORD-20260701-0001",
    "user": { "_id": "...", "name": "Brijesh Kumar", "email": "...", "phone": "..." },
    "items": [
      {
        "_id": "...",
        "product": "65a1...",
        "name": "Wireless Bluetooth Headphones",
        "image": "https://...",
        "price": 1299,
        "quantity": 2,
        "selectedVariants": { "color": "Black" }
      }
    ],
    "shippingAddress": { ... },
    "paymentMethod": "COD",
    "paymentStatus": "pending",
    "orderStatus": "pending",
    "statusHistory": [
      { "status": "pending", "note": "", "updatedAt": "2026-07-01T04:30:00.000Z" }
    ],
    "subtotal": 2598,
    "shippingCharge": 0,
    "taxAmount": 467.64,
    "discountAmount": 259.8,
    "totalAmount": 2805.84,
    "estimatedDelivery": "2026-07-08T00:00:00.000Z",
    "createdAt": "2026-07-01T04:30:00.000Z"
  }
}
```

PUT `/orders/:id/status`

Update order status (admin only).

Request Body:

```
{
  "orderStatus": "shipped",
  "note": "Dispatched via BlueDart, tracking: BD123456"
}
```


Success Response — 200:

```
{ "success": true, "message": "Order status updated to shipped" }
```

Sends status update email to customer.

PUT `/orders/:id/cancel`

Customer cancels their own order (only if status is `pending`).

Request Body:

```
{ "cancelReason": "Changed my mind" }
```

Success Response — 200:

```
{ "success": true, "message": "Order cancelled successfully" }
```

Error Responses:

Code	Message
400	Order cannot be cancelled after processing has begun
403	Not authorized to cancel this order

10. Review Endpoints

GET `/reviews/:productId`

Get all reviews for a product.

Query Parameters: `page`, `limit`, `sort` (newest / highest / lowest)

Success Response — 200:

```
{
  "success": true,
  "data": {
    "reviews": [
      {
        "_id": "64f1...",
        "user": { "_id": "...", "name": "Brijesh Kumar", "avatarUrl": "https://..." },
        "rating": 5,
        "title": "Excellent sound quality!",
        "comment": "Best headphones I have ever used. The bass is incredible.",
        "isVerifiedPurchase": true,
        "createdAt": "2026-07-01T04:30:00.000Z"
      }
    ],
    "summary": {
      "average": 4.3,
      "count": 128,
      "breakdown": { "5": 72, "4": 32, "3": 14, "2": 6, "1": 4 }
    }
  },
  "pagination": { ... }
}
```

POST `/reviews/:productId`

Submit a review for a product.

Request Body:

```
{
  "rating": 5,
  "title": "Excellent sound quality!",
  "comment": "Best headphones I have ever used. The bass is incredible."
}
```

Success Response — 201:

```
{
  "success": true,
  "message": "Review submitted successfully",
  "data": { "_id": "...", "rating": 5, "isVerifiedPurchase": true, ... }
}
```

Error Responses:

| Code | Message |

|-----|-----|

| 409 | You have already reviewed this product |
| 400 | Only customers who purchased the product can review |

PUT `/reviews/:reviewId`

Edit own review.

Request Body:

```
{
  "rating": 4,
  "title": "Updated: Still great!",
  "comment": "Updated my review after 3 months of use..."
}
```

Success Response — 200:

```
{ "success": true, "message": "Review updated successfully", "data": { ... } }
```

DELETE `/reviews/:reviewId`

Delete own review. Admins can delete any review.

Success Response — 200:

```
{ "success": true, "message": "Review deleted successfully" }
```

11. Coupon Endpoints

POST `/coupons/validate`

Validate a coupon code for the current user and cart total.

Request Body:

```
{
  "code": "SAVE10",
  "orderAmount": 2598
}
```

Success Response — 200:

```
{
  "success": true,
  "message": "Coupon applied successfully",
  "data": {
    "code": "SAVE10",
    "discountType": "percentage",
    "discountValue": 10,
    "discountAmount": 259.8,
    "maxDiscount": 500,
    "finalAmount": 2338.2
  }
}
```

Error Responses:

Code	Message
------	---------

-----	-----
-------	-------

404	Coupon not found
-----	------------------

400	This coupon has expired
-----	-------------------------

400	Minimum order amount of ₹500 required
-----	---------------------------------------

400	You have already used this coupon
-----	-----------------------------------

400	Coupon usage limit reached
-----	----------------------------

GET `/coupons` 

Get all coupons (admin only).

Success Response — 200:

```
{
  "success": true,
  "data": [
    {
      "_id": "64f1...",
      "code": "SAVE10",
      "discountType": "percentage",
      "discountValue": 10,
      "minOrderAmount": 500,
      "maxDiscount": 200,
      "usageLimit": 100,
      "usedCount": 45,
      "perUserLimit": 1,
      "isActive": true,
      "expiresAt": "2027-01-01T00:00:00.000Z",
      "isExpired": false,
      "isUsageLimitReached": false
    }
  ]
}
```

POST `/coupons`

Create a new coupon.

Request Body:

```
{
  "code": "MONSOON30",
  "discountType": "percentage",
  "discountValue": 30,
  "minOrderAmount": 1000,
  "maxDiscount": 500,
  "usageLimit": 200,
  "perUserLimit": 1,
  "expiresAt": "2026-09-30T23:59:59.000Z"
}
```

Success Response — 201:

```
{ "success": true, "message": "Coupon created successfully", "data": { ... } }
```

PUT `/coupons/:id`

Update an existing coupon.

Success Response — 200:

```
{ "success": true, "message": "Coupon updated successfully", "data": { ... } }
```

PATCH `/coupons/:id/status`

Toggle coupon active/inactive.

Request Body:

```
{ "isActive": false }
```

Success Response — 200:

```
{ "success": true, "message": "Coupon deactivated" }
```

DELETE `/coupons/:id`

Delete a coupon.

Success Response — 200:

```
{ "success": true, "message": "Coupon deleted successfully" }
```

12. Upload Endpoints

POST `/upload/image`

Upload an image to Cloudinary.

Request: `multipart/form-data`

Field	Type	Description
<code>image</code>	File	Image file (jpeg/png/webp, max 5MB)
<code>folder</code>	String	<code>products</code> / <code>categories</code> / <code>avatars</code>

Success Response — 200:

```
{
  "success": true,
  "message": "Image uploaded successfully",
  "data": {
    "url": "https://res.cloudinary.com/shopease/image/upload/v1234567890/ecommerce/products/abc123.webp",
    "publicId": "ecommerce/products/abc123"
  }
}
```

Error Responses:

| Code | Message |

|-----|-----|

| 400 | No image file provided |

| 400 | File type not allowed. Use jpeg, png, or webp |

| 400 | File size exceeds 5MB limit |

POST `/upload/avatar`

Upload user avatar (customers can upload own).

Request: `multipart/form-data`

Field	Type
<code>image</code>	File

Success Response — 200:

```
{
  "success": true,
  "data": {
    "url": "https://res.cloudinary.com/.../avatars/user123.webp",
    "publicId": "ecommerce/avatars/user123"
  }
}
```

DELETE `/upload/image/:publicId`

Delete an image from Cloudinary.

Note: `publicId` must be URL-encoded (replace `/` with `%2F`)

Success Response — 200:

```
{ "success": true, "message": "Image deleted successfully" }
```

13. Analytics Endpoints

GET `/analytics/dashboard` 

Get dashboard summary stats.

Success Response — 200:

```
{
  "success": true,
  "data": {
    "revenue": {
      "today": 12450,
      "thisWeek": 84320,
      "thisMonth": 342100,
      "trend": "+12.5%"
    },
    "orders": {
      "total": 1284,
      "pending": 43,
      "processing": 28,
      "shipped": 67,
      "delivered": 1102,
      "cancelled": 44,
      "trend": "+8.2%"
    },
    "products": {
      "total": 156,
      "active": 148,
      "outOfStock": 8
    },
    "customers": {
      "total": 892,
      "newToday": 12,
      "trend": "+5.1%"
    },
    "lowStockProducts": [
      { "_id": "...", "name": "USB-C Cable", "stock": 3 },
      { "_id": "...", "name": "Phone Stand", "stock": 7 }
    ],
    "recentOrders": [ ... ]
  }
}
```


GET `/analytics/revenue`

Get revenue data for charting.

Query Parameters:

Param	Type	Description
----- ----- -----		
<code>period</code>	String	<code>daily</code> / <code>weekly</code> / <code>monthly</code> / <code>yearly</code>
<code>year</code>	Number	Year (default: current year)
<code>month</code>	Number	Month 1–12 (for daily period)

Success Response — 200:

```
{
  "success": true,
  "data": [
    { "label": "Jan", "revenue": 284500, "orders": 312 },
    { "label": "Feb", "revenue": 312400, "orders": 341 },
    { "label": "Mar", "revenue": 298700, "orders": 328 },
    { "label": "Apr", "revenue": 342100, "orders": 378 }
  ]
}
```

GET `/analytics/top-products`

Get top-selling products.

Query Parameters: `limit` (default: 10), `period` (last 30 / 90 / 365 days)

Success Response — 200:

```
{
  "success": true,
  "data": [
    {
      "_id": "64f1...",
      "name": "Wireless Bluetooth Headphones",
      "image": "https://...",
      "unitsSold": 284,
      "revenue": 368916,
      "category": "Electronics"
    }
  ]
}
```

GET `/analytics/orders`

Get order analytics by period.

Query Parameters: `period`, `year`, `month`

Success Response — 200:

```
{
  "success": true,
  "data": [
    { "label": "Jan", "total": 312, "delivered": 280, "cancelled": 18, "pending": 14 }
  ]
}
```

14. Error Reference

HTTP Status Codes

Code	When Used
200	Successful GET, PUT, PATCH, DELETE
201	Successful POST (resource created)
400	Bad request — validation failed, business rule violation
401	Unauthorized — no valid token / token expired
403	Forbidden — valid token but insufficient permissions
404	Resource not found
409	Conflict — duplicate email, duplicate review, etc.
429	Too many requests — rate limit exceeded
500	Internal server error

Common Error Messages

Error	Status	Message
Not logged in	401	Authentication required. Please login.
Token expired	401	Your session has expired. Please login again.
Not admin	403	Access denied. Admin privileges required.
Own data only	403	You are not authorized to access this resource.
Not found	404	[Resource] not found.
Duplicate email	409	An account with this email already exists.
Duplicate review	409	You have already reviewed this product.
Rate limit	429	Too many requests. Please try again after 1 minute.

15. Postman Collection

Environment Variables

```
{
  "baseUrl": "http://localhost:5000/api",
  "productId": "",
  "orderId": "",
  "categoryId": "",
  "couponId": "",
  "reviewId": ""
}
```

Collection Structure

```
ShopEase API
├── Auth
│   ├── Register
│   ├── Login
│   ├── Get Me
│   ├── Logout
│   ├── Forgot Password
│   └── Reset Password
├── Users
│   ├── Get All Users (Admin)
│   ├── Get User by ID
│   ├── Update Profile
│   ├── Change Password
│   ├── Block User (Admin)
│   ├── Add Address
│   ├── Update Address
│   ├── Delete Address
│   └── Set Default Address
├── Products
│   ├── Get All Products
│   ├── Get Product by Slug
│   ├── Get Featured Products
│   ├── Create Product (Admin)
│   ├── Update Product (Admin)
│   ├── Delete Product (Admin)
│   └── Toggle Status (Admin)
├── Categories
│   ├── Get All Categories
│   ├── Get Category by Slug
│   ├── Create Category (Admin)
│   ├── Update Category (Admin)
│   └── Delete Category (Admin)
├── Cart
│   ├── Get Cart
│   ├── Add to Cart
│   ├── Update Cart Item
│   ├── Remove Cart Item
│   ├── Clear Cart
│   └── Merge Guest Cart
├── Wishlist
│   ├── Get Wishlist
│   ├── Add to Wishlist
│   └── Remove from Wishlist
├── Orders
│   ├── Place Order
│   ├── Get All Orders (Admin)
│   ├── Get My Orders
│   ├── Get Order by ID
│   ├── Update Order Status (Admin)
│   └── Cancel Order
├── Reviews
│   ├── Get Product Reviews
│   ├── Submit Review
│   └── Edit Review
```

- └─ Delete Review
- └─  Coupons
 - └─ Validate Coupon
 - └─ Get All Coupons (Admin)
 - └─ Create Coupon (Admin)
 - └─ Update Coupon (Admin)
 - └─ Toggle Status (Admin)
 - └─ Delete Coupon (Admin)
- └─  Upload
 - └─ Upload Product Image
 - └─ Upload Avatar
 - └─ Delete Image
- └─  Analytics (Admin)
 - └─ Dashboard Stats
 - └─ Revenue Chart
 - └─ Top Products
 - └─ Orders Chart

Total Endpoints: 54

Module	Count
Auth	6
Users	9
Products	7
Categories	5
Cart	6
Wishlist	3
Orders	6
Reviews	4
Coupons	6
Upload	3
Analytics	4
Total	59

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial API Documentation

Previous Phase → [Phase 5: Backend Schema](#)

Next Phase → [Phase 7: Implementation Plan](#)

Section 8: Phase 7: Implementation Plan & Roadmap

Implementation Plan

Project: Production-Ready E-Commerce Platform
Version: 1.0
Status: Planning
Date: July 2026
Phase: 7 of 9

Table of Contents

- 1. Overview
- 2. Development Environment Setup
- 3. Sprint Plan
- 4. Stage 1 — Project Setup
- 5. Stage 2 — Backend Development
- 6. Stage 3 — Admin Dashboard Development
- 7. Stage 4 — Client Website Development
- 8. Stage 5 — Integration & Testing
- 9. Task Checklist
- 10. Coding Order (Dependencies First)
- 11. Branch Strategy Per Stage
- 12. Definition of Done

1. Overview

Total Estimated Timeline: 18 Weeks

Stage	Description	Duration
Stage 1	Project Setup + Tooling	Week 1
Stage 2	Backend Development	Weeks 2–5
Stage 3	Admin Dashboard	Weeks 6–8
Stage 4	Client Website	Weeks 9–12
Stage 5	Integration + Testing	Weeks 13–14
Stage 6	Deployment	Week 15
Stage 7	Buffer + Bug Fixes	Weeks 16–18

Team Roles

Role	Responsibilities
Backend Dev	API, database, auth, email, uploads
Frontend Dev (Dashboard)	Admin React app
Frontend Dev (Website)	Customer Next.js app
Full Stack	Integration, testing, deployment

2. Development Environment Setup

2.1 Prerequisites

Node.js	v20.x LTS	→ https://nodejs.org
MongoDB	v7.x	→ Local: MongoDB Community / Cloud: MongoDB Atlas
Git	Latest	→ https://git-scm.com
VS Code	Latest	→ https://code.visualstudio.com
Postman	Latest	→ https://www.postman.com

2.2 VS Code Extensions (Required)

ESLint	→ dbaeumer.vscode-eslint
Prettier	→ esbenp.prettier-vscode
Tailwind CSS IntelliSense	→ bradlc.vscode-tailwindcss
ES7+ React Snippets	→ dsznajder.es7-react-js-snippets
Thunder Client	→ rangav.vscode-thunder-client
Gitlens	→ eamodio.vscode-gitlens
Auto Rename Tag	→ formulahendry.auto-rename-tag
Path IntelliSense	→ christian-kohler.path-intellisense
MongoDB for VS Code	→ mongodb.mongodb-vscode

2.3 VS Code Workspace Settings

```
// .vscode/settings.json (in each project root)
{
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  },
  "editor.tabSize": 2,
  "editor.wordWrap": "on",
  "tailwindCSS.includeLanguages": { "javascript": "javascript" },
  "files.associations": { "*.css": "tailwindcss" }
}
```

2.4 Global NPM Packages

```
npm install -g nodemon
npm install -g eslint
npm install -g prettier
```

3. Sprint Plan

Sprint Overview (2-week sprints)

```
Sprint 1 (Weeks 1-2): Project setup + Backend Auth + User module
Sprint 2 (Weeks 3-4): Backend Products + Categories + Cart + Wishlist
Sprint 3 (Weeks 5-6): Backend Orders + Reviews + Coupons + Analytics + Upload
Sprint 4 (Weeks 7-8): Admin Dashboard – Auth + Layout + Product Management
Sprint 5 (Weeks 9-10): Admin Dashboard – Orders + Customers + Categories + Coupons + Analytics
Sprint 6 (Weeks 11-12): Website – Auth + Home + Product Listing + Product Detail
Sprint 7 (Weeks 13-14): Website – Cart + Checkout + Profile + Wishlist + Order Tracking
Sprint 8 (Weeks 15-16): Integration testing + Bug fixes + Performance
Sprint 9 (Weeks 17-18): Deployment + Final QA + Go Live
```

4. Stage 1 — Project Setup

Week 1

4.1 Repository Setup

```
# 1. Create GitHub repository
#   Repo name: ecommerce-project
#   Visibility: Private
#   Add: .gitignore (Node), README

# 2. Clone locally
git clone https://github.com/<username>/ecommerce-project.git
cd ecommerce-project

# 3. Create main branch structure
git checkout -b develop
git push origin develop

# 4. Protect main branch in GitHub settings:
#   - Require PR before merging
#   - Require 1 approval
#   - No direct pushes
```

4.2 Initialize Backend

```

mkdir backend && cd backend

# Init package.json
npm init -y

# Update package.json → add "type": "module" for ES Modules

# Install production dependencies
npm install express mongoose bcryptjs jsonwebtoken cookie-parser \
  cloudinary multer nodemailer express-validator helmet cors \
  morgan dotenv express-rate-limit compression slugify crypto

# Install dev dependencies
npm install -D nodemon eslint prettier eslint-config-prettier \
  eslint-plugin-node

# Create folder structure
mkdir -p src/{config,controllers,middleware,models,routes,services,utils/validators}

# Create entry point
touch server.js src/app.js src/config/db.js

# Create .env and .env.example
touch .env .env.example .gitignore .eslintrc.js .prettierrc

```

4.3 Initialize Dashboard (React + Vite)

```

cd .. && mkdir dashboard && cd dashboard

# Create Vite + React app
npm create vite@latest . -- --template react

# Install dependencies
npm install react-router-dom @reduxjs/toolkit react-redux axios \
  react-hook-form @hookform/resolvers zod recharts \
  @tanstack/react-table react-hot-toast lucide-react date-fns

# Install dev dependencies
npm install -D tailwindcss postcss autoprefixer eslint \
  eslint-plugin-react eslint-plugin-react-hooks prettier

# Init Tailwind
npx tailwindcss init -p

# Create folder structure
mkdir -p src/{app,assets,components/{common,layout,charts},features,hooks,pages/{auth,dashboard,produ

```

4.4 Initialize Website (Next.js)

```

cd .. && mkdir website && cd website

# Create Next.js app
npx create-next-app@latest . --typescript=false --eslint --tailwind \
  --src-dir --app --import-alias "@/*" --use-npm

# Install additional dependencies
npm install axios react-hook-form @hookform/resolvers zod \
  zustand swiper lucide-react next-seo react-hot-toast date-fns

# Install dev dependencies
npm install -D prettier eslint-config-prettier

# Create folder structure
mkdir -p src/{components/{common,layout,product,card,checkout,home},context,hooks,lib,services,utils}

```

4.5 Root README + Scripts

```

# Back in root
cd ..

# Create root package.json for running all apps together
touch package.json README.md

```

```

// Root package.json
{
  "name": "ecommerce-platform",
  "version": "1.0.0",
  "scripts": {
    "dev:backend": "cd backend && npm run dev",
    "dev:dashboard": "cd dashboard && npm run dev",
    "dev:website": "cd website && npm run dev",
    "dev": "concurrently \"npm run dev:backend\" \"npm run dev:dashboard\" \"npm run dev:website\"",
  },
  "devDependencies": {
    "concurrently": "^8.0.0"
  }
}

```

5. Stage 2 — Backend Development

Weeks 2–5 | Build order: bottom-up (config → models → middleware → controllers → routes)

5.1 Week 2 — Foundation + Auth + Users

Step 1: Core Setup

```
Create: src/app.js
- Import express, helmet, cors, morgan, cookieParser, compression
- Mount rate limiter
- Mount all routers
- Mount global error handler

Create: server.js
- Import app + connectDB
- Call connectDB() then app.listen(PORT)

Create: src/config/db.js
- MongoDB connection with mongoose
- console.log on success, process.exit on failure

Create: src/config/cloudinary.js
- Configure cloudinary with env vars

Create: src/config/email.js
- Configure nodemailer transporter
```

Step 2: Utilities

```
Create: src/utils/ApiError.js      → Custom error class
Create: src/utils/ApiResponse.js   → Standard response wrapper
Create: src/utils/asyncHandler.js  → Try-catch HOF wrapper
Create: src/utils/generateToken.js → JWT sign access + refresh tokens
```

Step 3: User Model

```
Create: src/models/User.model.js
- Full schema (see Phase 5 Schema doc)
- Pre-save: bcrypt password
- Methods: comparePassword, getResetPasswordToken
- Virtual: avatarUrl
```

Step 4: Auth Middleware

```
Create: src/middleware/auth.middleware.js
- Verify accessToken from cookie
- If expired → check refreshToken → issue new accessToken
- Attach user to req.user

Create: src/middleware/admin.middleware.js
- Check req.user.role is 'admin' or 'superadmin'
- Throw 403 if not

Create: src/middleware/error.middleware.js
- Global error handler
- Handle Mongoose ValidationError, CastError, duplicate key

Create: src/middleware/validate.middleware.js
- Run express-validator validationResult
- Return 400 with errors array if invalid
```

Step 5: Auth Validators

```
Create: src/utils/validators/auth.validator.js
- registerValidator: name, email, password, confirmPassword
- loginValidator: email, password
- forgotPasswordValidator: email
- resetPasswordValidator: token, password, confirmPassword
```

Step 6: Auth Controller + Routes

```
Create: src/controllers/auth.controller.js
- register()      → hash pw, create user, set cookies, send welcome email
- login()         → verify credentials, check blocked, set cookies
- logout()        → clear cookies
- getMe()         → return req.user
- forgotPassword() → generate token, hash it, save, send email
- resetPassword() → find by hashed token, check expiry, update pw
- refreshToken()  → verify refresh token, issue new access token

Create: src/routes/auth.routes.js
- Map all auth endpoints

Create: src/services/email.service.js
- sendWelcomeEmail(user)
- sendPasswordResetEmail(user, resetUrl)
- sendOrderConfirmationEmail(user, order)
- sendOrderStatusEmail(user, order)
```

Step 7: User Controller + Routes

```
Create: src/controllers/user.controller.js
- getAllUsers()    → paginated, filtered list (admin)
- getUserById()    → single user
- updateUser()     → update name, phone, avatar
- changePassword()
- blockUser()
- addAddress()
- updateAddress()
- deleteAddress()
- setDefaultAddress()
```

```
Create: src/routes/user.routes.js
- Protected routes with authMiddleware
- Admin routes with adminMiddleware
```

Test:

```
✓ POST /api/auth/register
✓ POST /api/auth/login
✓ GET  /api/auth/me
✓ POST /api/auth/logout
✓ POST /api/auth/forgot-password
✓ POST /api/auth/reset-password
✓ GET  /api/users (admin)
✓ GET  /api/users/:id
✓ PUT  /api/users/:id
```

5.2 Week 3 — Products + Categories + Upload

Step 1: Models

```
Create: src/models/Category.model.js → (see Phase 5)
Create: src/models/Product.model.js  → (see Phase 5)
- Pre-save: auto slug, auto discount %
- Virtual: stockStatus
- Text index for search
```

Step 2: Upload Middleware + Controller

```

Create: src/middleware/upload.middleware.js
- Multer memoryStorage config
- File filter: jpeg/png/webp only
- Size limit: 5MB
- Fields: single image per request

Create: src/services/cloudinary.service.js
- uploadImage(buffer, folder) → stream upload to Cloudinary
- deleteImage(publicId) → destroy from Cloudinary

Create: src/controllers/upload.controller.js
- uploadImage() → multer → cloudinary → return {url, publicId}
- uploadAvatar() → apply face-crop transformation
- deleteImage() → cloudinary.destroy(publicId)

Create: src/routes/upload.routes.js

```

Step 3: Category Controller + Routes

```

Create: src/controllers/category.controller.js
- getAllCategories() → nested (parent + children), active only
- getCategoryBySlug()
- createCategory() → admin
- updateCategory() → admin
- deleteCategory() → check no products assigned, admin

Create: src/routes/category.routes.js

```

Step 4: Product Validators

```

Create: src/utils/validators/product.validator.js
- createProductValidator: name, price, stock, category required
- updateProductValidator: all optional

```

Step 5: Product Controller + Routes

```

Create: src/controllers/product.controller.js
- getAllProducts() → filters: category, price, brand, rating, inStock, featured
                        sort: price, rating, newest, name
                        pagination, text search
- getProductBySlug() → populate category
- getFeaturedProducts() → isActive + isFeatured, limit 8
- createProduct() → admin
- updateProduct() → admin
- deleteProduct() → soft delete (isActive: false)
- toggleStatus() → admin

Create: src/routes/product.routes.js

```


Test:

- ✓ GET /api/categories
- ✓ POST /api/categories (admin)
- ✓ GET /api/products
- ✓ GET /api/products/:slug
- ✓ POST /api/products (admin)
- ✓ POST /api/upload/image (admin)
- ✓ Full-text search: GET /api/products?search=headphones
- ✓ Price filter: GET /api/products?minPrice=500&maxPrice=2000

5.3 Week 4 — Cart + Wishlist + Orders

Step 1: Cart Model + Controller

Create: `src/models/Cart.model.js` → (see Phase 5)

Create: `src/controllers/cart.controller.js`

- `getCart()` → populate `items.product` (check `isActive` + `stock`)
- `addToCart()` → `upsert`: if product exists → increment `qty`
verify stock availability
- `updateCart()` → set new quantity, validate against stock
- `removeItem()` → pull `cartItem` by `_id`
- `clearCart()` → set `items: []`
- `mergeGuestCart()` → on login: merge `localStorage` cart with server cart

Create: `src/routes/cart.routes.js`

Step 2: Wishlist Model + Controller

Create: `src/models/Wishlist.model.js` → (see Phase 5)

Create: `src/controllers/wishlist.controller.js`

- `getWishlist()` → populate products with details
- `addToWishlist()` → `addToSet` (no duplicates)
- `removeFromWishlist()` → pull product from array

Create: `src/routes/wishlist.routes.js`

Step 3: Order Validators

Create: `src/utils/validators/order.validator.js`

- `placeOrderValidator`: `shippingAddress` fields, `paymentMethod`

Step 4: Order Controller + Routes

Create: src/models/Order.model.js → (see Phase 5)

Create: src/controllers/order.controller.js

- placeOrder()
 - validate cart not empty
 - re-verify stock for each item (not just cart snapshot)
 - apply coupon discount (if provided)
 - calculate: subtotal, tax, shipping, total
 - create Order document
 - decrement stock for each product
 - update coupon.usedCount + usedBy[]
 - clear user cart
 - send confirmation email (async)
 - return order
- getAllOrders() → paginated, filters: status, date range, search (admin)
- getMyOrders() → customer's own orders, paginated
- getOrderById() → full details, customer: own only
- updateOrderStatus() → admin: status + note, push statusHistory
send email notification to customer
- cancelOrder() → customer: only if status = 'pending'
set status = 'cancelled', restore stock

Create: src/routes/order.routes.js

Test:

- ✓ GET /api/cart
- ✓ POST /api/cart/add
- ✓ PUT /api/cart/update
- ✓ DELETE /api/cart/remove/:id
- ✓ GET /api/wishlist
- ✓ POST /api/wishlist/add
- ✓ POST /api/orders (place order)
- ✓ GET /api/orders/my-orders
- ✓ PUT /api/orders/:id/status (admin)
- ✓ PUT /api/orders/:id/cancel
- ✓ Confirm stock deducted after order
- ✓ Confirm cart cleared after order
- ✓ Confirm confirmation email received

5.4 Week 5 — Reviews + Coupons + Analytics

Step 1: Review Model + Controller

```
Create: src/models/Review.model.js    → (see Phase 5)
- Compound unique index: {product, user}
- Post-save: recalculate product ratings via aggregation
- Post-delete: recalculate product ratings

Create: src/controllers/review.controller.js
- getProductReviews() → paginated, sort options, rating breakdown summary
- submitReview()      → check not already reviewed
                        check isVerifiedPurchase (has delivered order)
                        save review → triggers rating recalculation
- editReview()        → own review only
- deleteReview()       → own review or admin

Create: src/routes/review.routes.js
```

Step 2: Coupon Model + Controller

```
Create: src/models/Coupon.model.js    → (see Phase 5)
- Methods: validateForUser, calculateDiscount
- Virtuals: isExpired, isUsageLimitReached

Create: src/controllers/coupon.controller.js
- validateCoupon() → check active, not expired, usage limits, min order
                  return discount amount
- getAllCoupons() → admin
- createCoupon()  → admin
- updateCoupon()  → admin
- toggleStatus()  → admin
- deleteCoupon()  → admin

Create: src/routes/coupon.routes.js
```

Step 3: Analytics Controller

```

Create: src/controllers/analytics.controller.js
- getDashboardStats()
  → MongoDB aggregation pipelines:
    - Revenue: sum of totalAmount where status=delivered
    - Orders: count by status
    - Products: count active/inactive/outOfStock
    - Customers: count + today's new customers
    - Low stock: stock < 10
    - Recent orders: last 10

- getRevenueChart()
  → Aggregate by period (daily/monthly/yearly)
  → Group by date label, sum revenue, count orders

- getTopProducts()
  → Aggregate from orderItems
  → Group by product, sum quantity and revenue
  → Populate product name, image, category

- getOrdersChart()
  → Aggregate orders by period and status

Create: src/routes/analytics.routes.js

```

Step 4: Backend Complete — Final Checks

- ✓ All 59 endpoints working and tested in Postman
- ✓ Rate limiting active (100 req/min)
- ✓ All protected routes require valid JWT
- ✓ Admin routes reject customer role
- ✓ All email templates working
- ✓ Cloudinary upload/delete working
- ✓ Error responses follow standard format
- ✓ .env.example has all required variables
- ✓ No secrets in codebase
- ✓ MongoDB indexes created

6. Stage 3 — Admin Dashboard Development

Weeks 6–8

6.1 Week 6 — Foundation + Auth + Layout + Products

Step 1: Setup Foundation

```

Configure: tailwind.config.js
- Extend: colors (dashboard palette)
- Extend: fontFamily (Inter)
- Content paths

Create: src/index.css
- Import Google Fonts (Inter)
- Dashboard CSS variables
- Base styles

Create: src/services/api.js
- Axios instance: baseURL from env
- withCredentials: true (send cookies)
- Request interceptor: (no token needed, cookie auto-sent)
- Response interceptor: handle 401 → redirect to login

Create: src/app/store.js + rootReducer.js
- Configure Redux store with RTK
- Combine auth slice + RTK Query APIs

Create: src/utils/constants.js → ORDER_STATUSES, ROLES, etc.
Create: src/utils/formatCurrency.js
Create: src/utils/formatDate.js

```

Step 2: Auth Feature

```

Create: src/features/auth/authSlice.js
- State: { user, isLoading, isAuthenticated }
- Actions: setCredentials, logout

Create: src/features/auth/authApi.js (RTK Query)
- login mutation
- logout mutation
- getMe query

Create: src/hooks/useAuth.js
- Return user, isAuthenticated, role from Redux

Create: src/pages/auth/LoginPage.jsx
- Email + password form
- Handle error toast
- Redirect to /dashboard on success

Create: src/routes/AppRoutes.jsx
Create: src/routes/ProtectedRoute.jsx
- Redirect to /login if not authenticated
- Redirect to /login if not admin role

```

Step 3: Layout Components

```
Create: src/components/layout/DashboardLayout.jsx
- Flex: Sidebar + main content area
- Renders <Sidebar /> + <Header /> + <Outlet />
```

```
Create: src/components/layout/Sidebar.jsx
- Logo + nav links (icon + label)
- Active link highlighting
- Collapse/expand button
- Logout button at bottom
```

```
Create: src/components/layout/Header.jsx
- Page title (from route)
- Admin name + avatar
- Dropdown: Profile / Logout
```

Step 4: Common Components

```
Create: src/components/common/Button.jsx
- variants: primary, secondary, danger, ghost
- sizes: sm, md, lg
- loading state with spinner
```

```
Create: src/components/common/Input.jsx
- label, error, helperText props
- forwardRef for react-hook-form
```

```
Create: src/components/common/Badge.jsx
- Order status badges with color mapping
```

```
Create: src/components/common/Modal.jsx
- Portal-based, backdrop, close on Escape
```

```
Create: src/components/common/ConfirmDialog.jsx
- Title, message, Cancel + Confirm buttons
```

```
Create: src/components/common/Spinner.jsx
```

```
Create: src/components/common/Pagination.jsx
```

```
Create: src/components/common/Table.jsx → TanStack Table wrapper
```

Step 5: Dashboard Overview Page

```
Create: src/features/analytics/analyticsApi.js
- getDashboardStats query
- getRevenueChart query
- getTopProducts query
- getOrdersChart query

Create: src/pages/dashboard/DashboardPage.jsx
- Stat cards (Revenue, Orders, Products, Customers)
- Revenue LineChart (Recharts)
- Orders DonutChart (Recharts)
- Recent orders table
- Top products list
- Low stock alerts
```

Step 6: Product Management

```
Create: src/features/products/productsApi.js (RTK Query)
- getProducts query (with filter params)
- getProductById query
- createProduct mutation
- updateProduct mutation
- deleteProduct mutation
- toggleProductStatus mutation

Create: src/pages/products/ProductsPage.jsx
- Table: Name, Category, Price, Stock, Status, Actions
- Search input + category filter + status filter
- Pagination
- Edit / Delete / Toggle actions per row
- "Add Product" button

Create: src/pages/products/AddProductPage.jsx
- Multi-section form (Basic Info, Media, Pricing, Inventory, Settings)
- Image drag & drop (up to 6)
- Variant builder (add/remove groups and options)
- Specification builder (key-value)
- Tag input
- Submit → POST /api/products

Create: src/pages/products/EditProductPage.jsx
- Pre-filled form (load by id)
- Submit → PUT /api/products/:id
```

6.2 Week 7 — Orders + Customers + Categories + Coupons

Step 1: Order Management

```
Create: src/features/orders/ordersApi.js
- getAllOrders query
- getOrderById query
- updateOrderStatus mutation

Create: src/pages/orders/OrdersPage.jsx
- Table: Order#, Customer, Items, Total, Status, Date, Actions
- Filters: status dropdown + date range picker
- Search by order number or customer
- Pagination + Export CSV button

Create: src/pages/orders/OrderDetailPage.jsx
- Order info header (number, date, payment)
- Customer info card (with link to customer profile)
- Items table
- Price breakdown
- Status timeline (visual stepper)
- Status update dropdown + note field + Submit
```

Step 2: Customer Management

```
Create: src/features/customers/customersApi.js
- getAllCustomers query
- getCustomerById query
- blockCustomer mutation

Create: src/pages/customers/CustomersPage.jsx
- Table: Avatar, Name, Email, Orders, Joined, Status, Actions
- Search + filter by status
- View / Block-Unblock actions

Create: src/pages/customers/CustomerDetailPage.jsx
- Profile info
- Address list
- Order history table
- Block/Unblock toggle
```

Step 3: Category Management

```
Create: src/features/categories/categoriesApi.js
- getCategories, createCategory, updateCategory, deleteCategory

Create: src/pages/categories/CategoriesPage.jsx
- Category list (tree view for parent/child)
- Add Category modal form (name, desc, image, parent)
- Edit Category modal (pre-filled)
- Delete with confirm dialog (check has products)
```

Step 4: Coupon Management


```
Create: src/features/coupons/couponsApi.js
- getCoupons, createCoupon, updateCoupon, toggleStatus, deleteCoupon
```

```
Create: src/pages/coupons/CouponsPage.jsx
- Table: Code, Type, Value, Min Order, Usage, Expiry, Status, Actions
- Create Coupon modal (all fields)
- Edit modal
- Toggle active / Delete
```

6.3 Week 8 — Analytics + Settings + Polish

Step 1: Analytics Page

```
Create: src/pages/analytics/AnalyticsPage.jsx
- Revenue chart (period toggle: daily/weekly/monthly/yearly)
- Orders by status chart (bar)
- Top products table (units sold + revenue)
- Export to CSV button
```

Step 2: Settings Page

```
Create: src/pages/settings/SettingsPage.jsx
- Admin profile tab (edit name, email, avatar, password)
- Store settings tab (name, logo, address, contact)
```

Step 3: Dashboard Polish

- ✓ All pages have loading skeletons
- ✓ All async actions show loading spinner in button
- ✓ All errors show toast notification
- ✓ All tables have empty states with illustration
- ✓ Sidebar active link matches current route
- ✓ Sidebar collapses on small screens
- ✓ Breadcrumb shows on all nested pages
- ✓ Confirm dialog before every delete
- ✓ All forms have validation error messages
- ✓ Dashboard is fully responsive (tablet + desktop)

7. Stage 4 — Client Website Development

Weeks 9–12

7.1 Week 9 — Foundation + Auth + Home Page

Step 1: Website Foundation

```
Configure: tailwind.config.js
- Extend colors (primary indigo palette)
- Extend fontFamily (Plus Jakarta Sans + Inter)

Create: src/app/globals.css
- Import Google Fonts
- CSS variables (design tokens from Phase 4)
- Base reset styles

Create: src/app/layout.jsx
- Root layout: <Navbar />, <main>{children}</main>, <Footer />
- Google Fonts <link> in <head>
- Toast provider

Create: src/lib/axios.js
- Axios instance with baseURL + withCredentials

Create: src/context/AuthContext.jsx
- State: user, isLoading, isAuthenticated
- Actions: login, logout, updateUser
- Fetch /auth/me on mount

Create: src/context/CartContext.jsx
- State: items, itemCount, subtotal
- Actions: addToCart, removeFromCart, updateQuantity, clearCart
- Guest cart from localStorage
- On login: merge guest cart
```

Step 2: Layout Components

```
Create: src/components/layout/Navbar.jsx
- Logo + Category mega-menu + Search bar + Wishlist + Cart + Account
- Sticky on scroll (add shadow after 50px)
- Mobile: hamburger + MobileMenu drawer
- Cart count badge (live from CartContext)
```

```
Create: src/components/layout/MobileMenu.jsx
- Slide-in drawer from left
- Category links + auth links
```

```
Create: src/components/layout/Footer.jsx
- Brand info, Quick links, Categories, Contact, Social icons
- Newsletter inline form
- Copyright row
```

```
Create: src/components/layout/SearchBar.jsx
- Input with debounce (400ms)
- Autocomplete dropdown (5 results)
- Keyboard nav (arrow keys + Enter)
```

Step 3: Auth Pages

```
Create: src/services/auth.service.js
- register(), login(), logout(), forgotPassword(), resetPassword(), getMe()
```

```
Create: src/app/(auth)/login/page.jsx
- Email + password form
- "Forgot password?" link
- "Register" link
- On success: merge cart → redirect
```

```
Create: src/app/(auth)/register/page.jsx
- Name, email, password, confirm password
- On success: redirect to home
```

```
Create: src/app/(auth)/forgot-password/page.jsx
- Email form → success message
```

```
Create: src/app/(auth)/reset-password/page.jsx
- Read token from URL params
- New password + confirm form
```

Step 4: Common Components

```
Create: src/components/common/Button.jsx
Create: src/components/common/Input.jsx
Create: src/components/common/Spinner.jsx
Create: src/components/common/StarRating.jsx → display + input modes
Create: src/components/common/Pagination.jsx
Create: src/components/common/Badge.jsx → order status, product badges
```

Step 5: Home Page

```
Create: src/services/product.service.js
- getProducts(), getProductBySlug(), getFeaturedProducts()

Create: src/components/home/HeroBanner.jsx
- Swiper.js slider (3-5 slides, autoplay, fade)
- Each slide: BG image + overlay + heading + CTA

Create: src/components/home/FeaturedCategories.jsx
- Horizontal scroll on mobile, grid on desktop
- Circular image + category name

Create: src/components/home/FeaturedProducts.jsx
- GET /api/products/featured
- 4-col product grid

Create: src/components/home/PromoBanner.jsx
- Split layout with gradient bg

Create: src/app/page.jsx
- Assemble: HeroBanner + FeaturedCategories + FeaturedProducts + PromoBanner + Newsletter
- SSG with revalidate: 3600 (1 hour)
```

7.2 Week 10 — Product Listing + Product Detail

Step 1: Product Components

```
Create: src/components/product/ProductCard.jsx
- Image (aspect 1:1, next/image)
- Wishlist icon (hover overlay)
- Category label, Name (2-line clamp)
- Star rating + count
- Price + original price + discount badge
- Add to Cart button
- Hover lift animation

Create: src/components/product/ProductGrid.jsx
- Responsive grid wrapper (2/3/4 cols)
- Skeleton loading state (8 skeleton cards)
- Empty state

Create: src/components/product/ProductFilters.jsx
- Category checkboxes
- Price range slider (min/max inputs)
- Brand checkboxes
- Rating filter (star buttons)
- In Stock toggle
- Clear All Filters
```

Step 2: Product Listing Page

```
Create: src/app/products/page.jsx
- ISR: revalidate every 60 seconds
- Layout: sidebar filters + product grid
- Sort dropdown (price asc/desc, newest, rating)
- Active filter chips (dismissible)
- Pagination
- Mobile: filter in slide-in drawer

Create: src/app/category/[slug]/page.jsx
- Category header (name + description + image)
- Product grid filtered by category
```

Step 3: Product Detail Page

```
Create: src/components/product/ProductImageGallery.jsx
- Main image + thumbnail strip
- Click thumbnail → swap main
- Click main → zoom modal
- Swiper on mobile

Create: src/components/product/ReviewCard.jsx
- User avatar + name + date
- Star rating
- Review title + comment
- "Verified Purchase" badge

Create: src/app/products/[slug]/page.jsx
- SSR (fresh data each request)
- Left: image gallery | Right: product info
- Variant selector (pill buttons, selected state)
- Quantity: +/- with stock limit
- Add to Cart button (disabled if OOS)
- Add to Wishlist (heart icon, toggle)
- Delivery info row
- Tabs: Description | Specifications | Reviews
- Related products grid (same category, different product)
```

7.3 Week 11 — Cart + Checkout + Search

Step 1: Cart

```
Create: src/components/cart/CartItem.jsx
- Product image + name + variant + price
- Quantity +/- controls
- Remove button

Create: src/components/cart/CartSummary.jsx
- Subtotal, discount, shipping, tax, total
- Coupon input + Apply button
- Checkout button

Create: src/app/cart/page.jsx
- Guest: read from CartContext (localStorage)
- Auth: GET /api/cart
- CartItem list
- CartSummary panel
- Empty cart state
```

Step 2: Checkout

```
Create: src/components/checkout/AddressForm.jsx
- All address fields
- Saved addresses as selectable cards

Create: src/components/checkout/OrderSummary.jsx
- Read-only items list + price breakdown

Create: src/app/checkout/page.jsx
- Protected route (redirect to login if guest)
- 3-step flow: Address → Review → Payment
- Step indicator bar
- "Place Order" → POST /api/orders
- Loading state during order creation

Create: src/app/order-success/page.jsx
- Success animation (checkmark)
- Order number + estimated delivery
- Items summary
- "Track My Order" + "Continue Shopping" CTAs
```

Step 3: Search Results

```
Create: src/app/search/page.jsx
- Read ?q= from URL
- GET /api/products?search=<query>
- Results grid with filters
- "Showing X results for <query>"
- No results state with suggestions
```

7.4 Week 12 — User Profile + Wishlist + Order Tracking

Step 1: Profile Layout

```
Create: src/app/profile/layout.jsx
- Protected route wrapper
- Left sidebar nav (desktop) / tab bar (mobile)
- Links: Profile, Orders, Wishlist, Addresses
```

Step 2: Profile Pages

```
Create: src/app/profile/page.jsx
- Profile form (name, phone, avatar upload)
- Change password section
- PUT /api/users/:id

Create: src/app/profile/addresses/page.jsx
- Address cards with Edit / Delete / Set Default
- "Add New Address" form
- POST/PUT/DELETE /api/users/:id/addresses/:addressId

Create: src/app/profile/orders/page.jsx
- Order list table: Order#, Date, Items, Total, Status
- Status badge (colored)
- Pagination
- GET /api/orders/my-orders

Create: src/app/profile/orders/[id]/page.jsx
- Full order detail
- Status timeline stepper
- Cancel order button (if pending)
- GET /api/orders/:id

Create: src/app/profile/wishlist/page.jsx
- Product grid of wishlisted items
- Remove from wishlist
- Move to cart
- GET /api/wishlist
```

Step 3: Website Polish

- ✓ All pages have metadata (title + description) via Next.js metadata API
- ✓ All pages have loading.jsx skeleton files
- ✓ All pages have error.jsx error boundaries
- ✓ Skeleton loaders for all async sections
- ✓ Toast notifications for all user actions
- ✓ Mobile navigation works perfectly
- ✓ All images use next/image
- ✓ Search autocomplete keyboard navigable
- ✓ Cart count updates in real time
- ✓ Wishlist heart toggles correctly
- ✓ Checkout redirects guest to login
- ✓ 404 page styled and helpful
- ✓ Footer newsletter form works

8. Stage 5 — Integration & Testing

Weeks 13–14

8.1 Integration Testing

Test Scenario 1: Full Purchase Flow

Register → Browse Products → Filter → View Product → Add to Cart
→ Apply Coupon → Checkout → Place Order → Check Email
→ View Order in Profile → Admin: Update Status → Email received

Test Scenario 2: Guest Flow

Visit → Browse → Add to Cart (guest) → Login → Cart Merged
→ Checkout → Order Success

Test Scenario 3: Admin Flow

Admin Login → Add Product with Images → Publish
→ Verify on Website → Edit Product → Toggle Inactive
→ Verify hidden on Website

Test Scenario 4: Review Flow

Login → Purchase Product → Deliver Order (admin) → Write Review
→ Verify ratings updated on product → Edit Review → Delete Review

Test Scenario 5: Coupon Flow

Create Coupon (admin) → Apply on Cart → Verify Discount
→ Try expired coupon → Try exceeding usage limit

8.2 Performance Checks

- ✓ Lighthouse score ≥ 90 (SEO, Performance, Accessibility, Best Practices)
- ✓ Product listing loads in $< 2s$ on 4G
- ✓ Product detail page SSR under 500ms TTFB
- ✓ API responses $< 200ms$ (measured in Postman)
- ✓ Images served as WebP from Cloudinary CDN
- ✓ No unnecessary re-renders (React DevTools profiler)
- ✓ Bundle size analyzed (Next.js build output)

8.3 Security Checks

- ✓ JWT not stored in localStorage (HTTP-only cookie)
- ✓ Admin routes return 403 for customer role
- ✓ Customer cannot access other user's orders
- ✓ Rate limiting blocks after 100 req/min
- ✓ SQL injection attempt returns 400 (sanitized)
- ✓ XSS attempt sanitized in product description
- ✓ CORS blocks requests from non-whitelisted origins
- ✓ Sensitive fields not returned (password, resetToken)
- ✓ .env file not in git (check .gitignore)

8.4 Cross-Browser + Device Testing

Browsers:

- ✓ Chrome (latest)
- ✓ Firefox (latest)
- ✓ Safari (latest)
- ✓ Edge (latest)

Devices:

- ✓ iPhone SE (375px)
- ✓ iPhone 14 (390px)
- ✓ Samsung Galaxy (360px)
- ✓ iPad (768px)
- ✓ iPad Pro (1024px)
- ✓ Desktop (1280px, 1440px, 1920px)

9. Task Checklist

Backend

```
[ ] Project setup (package.json, folder structure, .env)
[ ] MongoDB connection
[ ] Cloudinary config
[ ] Email service (Nodemailer)
[ ] Utility classes (ApiError, ApiResponse, asyncHandler)
[ ] User model + hooks
[ ] Auth middleware (JWT verify, refresh)
[ ] Admin middleware (role check)
[ ] Error middleware (global handler)
[ ] Auth controller (register, login, logout, me, forgot, reset)
[ ] User controller (CRUD, addresses, block)
[ ] Category model + controller + routes
[ ] Product model + hooks + text index
[ ] Product controller (CRUD, filters, search)
[ ] Upload middleware (multer) + Cloudinary service
[ ] Upload controller + routes
[ ] Cart model + controller (add, update, remove, merge)
[ ] Wishlist model + controller
[ ] Order model + hooks (order number, status history)
[ ] Order controller (place, list, status update, cancel)
[ ] Review model + post-save hook (rating recalculation)
[ ] Review controller (submit, edit, delete, summary)
[ ] Coupon model + methods (validate, calculateDiscount)
[ ] Coupon controller (validate, CRUD, toggle)
[ ] Analytics controller (dashboard, revenue, top products, orders)
[ ] All routes registered in app.js
[ ] Postman collection complete + tested
[ ] Seed script (admin user, categories, products, coupons)
```

Admin Dashboard

```
[ ] Tailwind + CSS variables setup
[ ] Axios instance (withCredentials)
[ ] Redux store + auth slice
[ ] RTK Query API files (all features)
[ ] Login page
[ ] Protected route + role check
[ ] Sidebar + Header + Layout shell
[ ] Common components (Button, Input, Badge, Modal, Dialog, Table, Pagination)
[ ] Dashboard overview page (stats, charts, recent orders, top products)
[ ] Product list page (table, search, filter)
[ ] Add product page (full form, image upload, variants)
[ ] Edit product page (pre-filled)
[ ] Order list page (table, filters, export)
[ ] Order detail page (status update, timeline)
[ ] Customer list page
[ ] Customer detail page
[ ] Category management page (modal CRUD)
[ ] Coupon management page (modal CRUD)
[ ] Analytics page (charts, export)
[ ] Settings page (profile + store)
[ ] All loading skeletons
[ ] All toast notifications
[ ] All empty states
[ ] All confirm dialogs
[ ] Responsive layout (tablet support)
```

Client Website

```
[ ] Tailwind + CSS variables setup
[ ] Google Fonts import
[ ] Axios instance
[ ] AuthContext (user state, login, logout)
[ ] CartContext (items, add, remove, update, merge)
[ ] Navbar (desktop + mobile drawer)
[ ] Footer
[ ] Search bar with autocomplete
[ ] Login page
[ ] Register page
[ ] Forgot password page
[ ] Reset password page
[ ] Home page (Hero, Categories, Featured, Promo, Newsletter)
[ ] Product listing page (filters, sort, pagination)
[ ] Category listing page
[ ] Product detail page (gallery, variants, qty, cart, wishlist, reviews)
[ ] Search results page
[ ] Cart page (items, coupon, summary)
[ ] Checkout page (3-step flow)
[ ] Order success page
[ ] Profile layout (sidebar nav)
[ ] Profile page (edit info, change password, avatar)
[ ] Addresses page (CRUD)
[ ] Orders list page
[ ] Order detail page (timeline, cancel)
[ ] Wishlist page
[ ] All page metadata (title + description)
[ ] All loading.jsx skeletons
[ ] All error.jsx boundaries
[ ] 404 page
[ ] Lighthouse score  $\geq 90$ 
```

10. Coding Order (Dependencies First)

1. Backend config + utilities	(no dependencies)
2. User model	(no dependencies)
3. Auth middleware	(depends: User model)
4. Auth controller + routes	(depends: User model, email service)
5. Category model + controller	(depends: auth middleware)
6. Product model + controller	(depends: Category model)
7. Upload service + controller	(depends: Cloudinary config)
8. Cart model + controller	(depends: User, Product)
9. Wishlist model + controller	(depends: User, Product)
10. Order model + controller	(depends: User, Product, Cart, Coupon)
11. Review model + controller	(depends: User, Product, Order)
12. Coupon model + controller	(depends: User)
13. Analytics controller	(depends: Order, Product, User)
14. Dashboard: Auth + Layout	(depends: backend Auth)
15. Dashboard: Products	(depends: backend Products, Categories, Upload)
16. Dashboard: Orders	(depends: backend Orders, Users)
17. Dashboard: Customers	(depends: backend Users)
18. Dashboard: Categories	(depends: backend Categories)
19. Dashboard: Coupons	(depends: backend Coupons)
20. Dashboard: Analytics	(depends: backend Analytics)
21. Website: Auth + Context	(depends: backend Auth)
22. Website: Home	(depends: backend Products, Categories)
23. Website: Product Listing	(depends: backend Products, Categories)
24. Website: Product Detail	(depends: backend Products, Reviews)
25. Website: Search	(depends: backend Products)
26. Website: Cart	(depends: CartContext, backend Cart)
27. Website: Checkout	(depends: backend Orders, Coupons)
28. Website: Profile + Orders	(depends: backend Users, Orders)
29. Website: Wishlist	(depends: backend Wishlist)

11. Branch Strategy Per Stage

```
main
└─ develop
    │ feature/backend-auth
    │ feature/backend-products
    │ feature/backend-orders
    │ feature/backend-reviews
    │ feature/backend-coupons
    │ feature/backend-analytics
    │ feature/dashboard-setup
    │ feature/dashboard-products
    │ feature/dashboard-orders
    │ feature/dashboard-analytics
    │ feature/website-setup
    │ feature/website-home
    │ feature/website-products
    │ feature/website-cart-checkout
    │ feature/website-profile
    │ fix/<bug-description>
    └─ chore/<task-description>
```

12. Definition of Done

A feature is considered **Done** when ALL of the following are true:

Code Quality:

- ✓ Code follows ESLint + Prettier rules (no warnings)
- ✓ No console.log statements in production code
- ✓ No hardcoded secrets or config values

Functionality:

- ✓ Feature works as per PRD acceptance criteria
- ✓ All edge cases handled (empty state, error state, loading state)
- ✓ Mobile responsive (375px → 1920px)

API (Backend):

- ✓ Endpoint tested in Postman for all scenarios
- ✓ Proper HTTP status codes returned
- ✓ Input validation working
- ✓ Auth/role protection working

UI (Frontend):

- ✓ Matches UI/UX Brief design system
- ✓ Loading skeleton shown during data fetch
- ✓ Error toast shown on API failure
- ✓ Empty state shown when no data

Version Control:

- ✓ Feature branch merged to develop via PR
- ✓ PR reviewed and approved
- ✓ No merge conflicts
- ✓ Commit messages follow convention

Documentation:

- ✓ Any new endpoint added to API docs
- ✓ .env.example updated if new env var added

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial Implementation Plan

Previous Phase → [Phase 6: API Documentation](#)

Next Phase → [Phase 8: Deployment Guide](#)

Section 9: Phase 8: Deployment Guide & Infrastructure

Deployment Guide

Project: Production-Ready E-Commerce Platform
Version: 1.0
Status: Planning
Date: July 2026
Phase: 8 of 9

Table of Contents

- 1. [Overview](#)
- 2. [Deployment Architecture](#)
- 3. [Pre-Deployment Checklist](#)
- 4. [Service Account Setup](#)
- 5. [MongoDB Atlas Setup](#)
- 6. [Cloudinary Setup](#)
- 7. [Email Service Setup](#)
- 8. [Backend Deployment \(Railway\)](#)
- 9. [Admin Dashboard Deployment \(Vercel\)](#)
- 10. [Client Website Deployment \(Vercel\)](#)
- 11. [Domain & DNS Configuration](#)
- 12. [SSL Certificate](#)
- 13. [Environment Variables Reference](#)
- 14. [Post-Deployment Verification](#)
- 15. [Rollback Procedure](#)
- 16. [CI/CD Pipeline](#)

1. Overview

Deployment Targets

Application	Platform	URL Pattern
Backend API	Railway	<code>api.shopease.com</code>
Client Website	Vercel	<code>shopease.com</code> / <code>www.shopease.com</code>
Admin Dashboard	Vercel	<code>admin.shopease.com</code>
Database	MongoDB Atlas	Internal connection string
Images	Cloudinary	<code>res.cloudinary.com/shopease/...</code>
Email	Gmail SMTP → SendGrid	SMTP

Deployment Flow

GitHub (main branch)

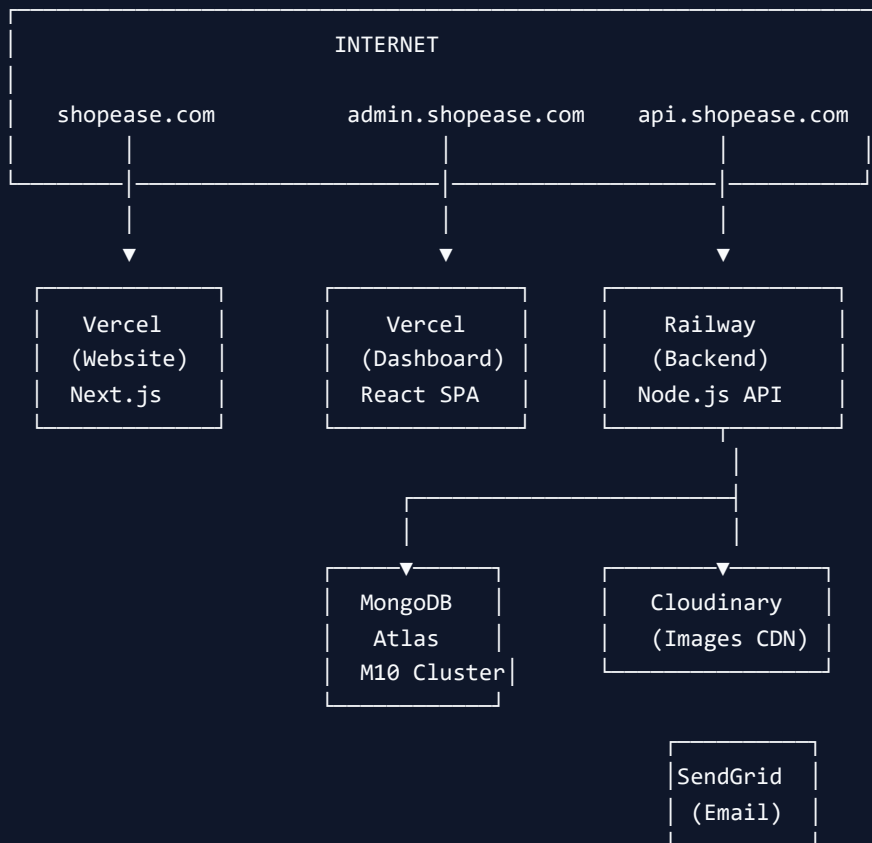
→ Railway (auto-deploy backend)

→ Vercel Project 1 (auto-deploy website)

→ Vercel Project 2 (auto-deploy dashboard)

All three apps are independently deployed.
A push to main triggers all three deployments simultaneously.

2. Deployment Architecture



3. Pre-Deployment Checklist

Code Readiness

- [] All features tested locally end-to-end
- [] No console.log() statements in production code
- [] No hardcoded localhost URLs – all use env variables
- [] All .env variables documented in .env.example
- [] .env files are in .gitignore (never committed)
- [] ESLint passes with zero errors: npm run lint
- [] Build succeeds without errors:
 - Backend: node server.js (no crash on start)
 - Dashboard: npm run build (no build errors)
 - Website: npm run build (no build errors)
- [] Git: all changes committed and pushed to main
- [] Git: no merge conflicts in main branch
- [] README.md updated with setup instructions

Security Readiness

```
[ ] All JWT secrets are strong random strings (≥ 64 chars)
[ ] bcrypt salt rounds = 12
[ ] CORS whitelist contains only production domains
[ ] Rate limiting is enabled
[ ] Helmet.js middleware is active
[ ] Cookie flags: httpOnly=true, secure=true, sameSite='strict'
[ ] No sensitive data logged in production
[ ] MongoDB Atlas IP whitelist configured
[ ] Cloudinary API keys have restricted permissions
```

Performance Readiness

```
[ ] Next.js build output analyzed (no large bundles)
[ ] All images go through Cloudinary (no local file serving)
[ ] Mongoose queries use .lean() for read-only operations
[ ] Database indexes are created
[ ] API pagination implemented on all list endpoints
[ ] gzip compression enabled (compression middleware)
```

4. Service Account Setup

4.1 Accounts Required

Service	URL	Plan
GitHub	github.com	Free
MongoDB Atlas	mongodb.com/atlas	Free (M0) → M10 for production
Cloudinary	cloudinary.com	Free tier (25GB storage, 25GB bandwidth)
Railway	railway.app	Hobby (\$5/month)
Vercel	vercel.com	Free (Hobby) — up to 2 projects
SendGrid	sendgrid.com	Free (100 emails/day)
Domain Registrar	namecheap.com / godaddy.com	~\$10–15/year

4.2 Create Accounts

1. Sign up at all services above
2. Link GitHub account to:
 - Railway (for auto-deploy)
 - Vercel (for auto-deploy)
3. Verify email addresses for all services

5. MongoDB Atlas Setup

5.1 Create Cluster

1. Login → mongodb.com/atlas
2. Click "Build a Database"
3. Choose: M0 Free Tier (for start) OR M10 (for production)
4. Provider: AWS | Region: Mumbai (ap-south-1) – closest to India
5. Cluster Name: ecommerce-cluster
6. Click "Create Cluster" (takes ~3 minutes)

5.2 Database User

1. Security → Database Access → Add New Database User
2. Username: ecommerce_user
3. Password: <generate strong password, save securely>
4. Database User Privileges: Read and write to any database
5. Click "Add User"

5.3 Network Access (IP Whitelist)

1. Security → Network Access → Add IP Address
2. For Railway: Add "Allow Access from Anywhere" (0.0.0.0/0)
(Railway uses dynamic IPs)
3. Click "Confirm"

NOTE: In a high-security setup, use Railway's static IP add-on and whitelist only that IP instead of 0.0.0.0/0

5.4 Get Connection String

1. Database → Connect → Connect your application
2. Driver: Node.js | Version: 5.5 or later
3. Copy connection string:
`mongodb+srv://ecommerce_user:<password>@ecommerce-cluster.xxxxx.mongodb.net/`
4. Replace <password> with your actual password
5. Add database name at the end:
`mongodb+srv://ecommerce_user:<password>@ecommerce-cluster.xxxxx.mongodb.net/ecommerce`
6. Save this as MONGODB_URI in your backend .env

5.5 Create Indexes (After First Deployment)

```
# Connect via MongoDB Atlas Data Explorer or mongosh
# Run index creation script from Phase 5 (Schema doc)

# Or add to seed script:
node src/utils/seed.js --indexes
```

6. Cloudinary Setup

6.1 Create Account & Get Credentials

1. Login → cloudinary.com/console
2. Dashboard shows:
 - Cloud Name: your-cloud-name
 - API Key: 123456789012345
 - API Secret: xxxxxxxxxxxxxxxxxxxxxx
3. Save all three in backend .env:
`CLOUDINARY_CLOUD_NAME=your-cloud-name`
`CLOUDINARY_API_KEY=123456789012345`
`CLOUDINARY_API_SECRET=xxxxxxxxxxxxxxxxxxxxxx`

6.2 Create Upload Folders

1. Media Library → Create Folder: ecommerce
2. Inside ecommerce, create sub-folders:
 - ecommerce/products
 - ecommerce/categories
 - ecommerce/avatars

6.3 Upload Presets (Optional but Recommended)

1. Settings → Upload → Add upload preset
2. Preset name: ecommerce_products
3. Signing Mode: Signed
4. Folder: ecommerce/products
5. Transformations: f_webp, q_auto
6. Save

Repeat for categories and avatars presets.

7. Email Service Setup

7.1 Development: Gmail SMTP

1. Use a dedicated Gmail account (not personal)
2. Enable 2-Factor Authentication on Gmail
3. Generate App Password:
Google Account → Security → 2-Step Verification → App Passwords
App: Mail | Device: Other → Name: "ShopEase Backend"
Copy the 16-character password
4. Save in .env:
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=shopease.noreply@gmail.com
EMAIL_PASS=xxxx xxxx xxxx xxxx ← 16-char app password
EMAIL_FROM=ShopEase <shopease.noreply@gmail.com>

7.2 Production: SendGrid

1. Login → sendgrid.com
2. Settings → API Keys → Create API Key
3. Name: ShopEase Production
4. Permissions: Full Access OR Mail Send only
5. Copy API Key (shown only once!)
6. Verify Sender Identity:
Settings → Sender Authentication → Single Sender Verification
Enter: From Name, From Email, Reply To, Company, Address
Verify via email link
7. Update backend .env for production:
EMAIL_SERVICE=sendgrid
SENDGRID_API_KEY=SG.xxxxxxxxxxxxxxxxxxxx
EMAIL_FROM=ShopEase <noreply@shopease.com>
8. Update email.service.js for production:
Use @sendgrid/mail package instead of Nodemailer
OR use Nodemailer with SendGrid SMTP:
EMAIL_HOST=smtp.sendgrid.net
EMAIL_PORT=587
EMAIL_USER=apikey
EMAIL_PASS=<SENDGRID_API_KEY>

8. Backend Deployment (Railway)

8.1 Prepare Backend for Deployment

```
# In backend/package.json – add start script:
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}
```

```
// In server.js – use process.env.PORT (Railway assigns dynamic port)
const PORT = process.env.PORT || 5000
app.listen(PORT, () => console.log(`Server running on port ${PORT}`))
```

8.2 Create Railway Project

1. Login → railway.app
2. New Project → Deploy from GitHub repo
3. Select: ecommerce-project repository
4. Select: backend/ as root directory
(Or use Railway monorepo support → set root to /backend)
5. Railway auto-detects Node.js → runs npm install + npm start

8.3 Configure Environment Variables on Railway

1. Railway Project → Variables tab
2. Add all variables from backend .env:

```
NODE_ENV=production
PORT=5000 (Railway overrides this automatically)
MONGODB_URI=mongodb+srv://...
JWT_ACCESS_SECRET=<64-char random string>
JWT_REFRESH_SECRET=<64-char different random string>
JWT_ACCESS_EXPIRE=15m
JWT_REFRESH_EXPIRE=7d
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
EMAIL_HOST=smtp.sendgrid.net
EMAIL_PORT=587
EMAIL_USER=apikey
EMAIL_PASS=<sendgrid_api_key>
EMAIL_FROM=ShopEase <noreply@shopease.com>
CLIENT_URL=https://shopease.com
DASHBOARD_URL=https://admin.shopease.com
COOKIE_SECURE=true
```

Generate strong secrets:

```
bash node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

8.4 Add Procfile (Optional)

```
# backend/Procfile
web: node server.js
```

8.5 Deploy & Verify

1. Railway auto-deploys on push to main
2. View deployment logs in Railway dashboard
3. Wait for "Deploy successful" status
4. Get Railway URL: `https://ecommerce-backend-production.up.railway.app`
5. Test with Postman:
GET `https://ecommerce-backend-production.up.railway.app/api/auth/me`
Expected: 401 Unauthorized (correct – not logged in)
6. Run seed script against production DB:
Add temporary env var: `SEED=true`
OR run locally with production `MONGODB_URI`:
`MONGODB_URI=<prod-uri> node src/utils/seed.js`

8.6 Custom Domain on Railway

1. Railway Project → Settings → Domains
2. Add custom domain: `api.shopease.com`
3. Copy the CNAME record Railway provides
4. Go to your domain registrar → DNS settings
5. Add CNAME record:
Type: CNAME
Name: `api`
Value: `ecommerce-backend-production.up.railway.app`
6. Wait for DNS propagation (5 min – 48 hours)
7. Railway auto-provisions SSL for custom domain

9. Admin Dashboard Deployment (Vercel)

9.1 Prepare Dashboard for Deployment

```
# In dashboard/package.json – verify build script exists:
{
  "scripts": {
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

```
// In dashboard/vite.config.js – ensure base is '/'
export default defineConfig({
  plugins: [react()],
  base: '/',
})
```

```
// In dashboard/src/services/api.js – use env variable for base URL:
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,
})
```

9.2 Create Vercel Project (Dashboard)

1. Login → vercel.com
2. New Project → Import Git Repository
3. Select: ecommerce-project
4. Root Directory: dashboard
5. Framework Preset: Vite
6. Build Command: npm run build
7. Output Directory: dist
8. Install Command: npm install

9.3 Configure Environment Variables (Dashboard)

In Vercel Project Settings → Environment Variables:

```
VITE_API_URL=https://api.shopease.com/api
```

9.4 Configure SPA Routing

React Router needs a `vercel.json` for client-side routing:

```
// dashboard/vercel.json
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

9.5 Deploy Dashboard

1. Click "Deploy" in Vercel
2. Wait for build to complete (~2 minutes)
3. Vercel provides URL: `https://ecommerce-dashboard.vercel.app`
4. Test:
 - Visit the URL
 - Login page loads
 - Login with admin credentials (seeded: `admin@shopease.com` / `Admin@123456`)
 - Dashboard overview loads
 - Products list loads with data from API

9.6 Custom Domain (Dashboard)

1. Vercel Project → Settings → Domains
2. Add: `admin.shopease.com`
3. Vercel provides DNS records to add:
Type: CNAME | Name: admin | Value: `cname.vercel-dns.com`
4. Add in domain registrar DNS settings
5. SSL auto-provisioned by Vercel (Let's Encrypt)

10. Client Website Deployment (Vercel)

10.1 Prepare Website for Deployment

```
// website/next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    domains: ['res.cloudinary.com'], // allow Cloudinary images
  },
  env: {
    NEXT_PUBLIC_API_URL: process.env.NEXT_PUBLIC_API_URL,
    NEXT_PUBLIC_SITE_URL: process.env.NEXT_PUBLIC_SITE_URL,
  },
}

module.exports = nextConfig
```

10.2 Create Vercel Project (Website)

1. Vercel → New Project → Import Git Repository
2. Select: ecommerce-project
3. Root Directory: website
4. Framework Preset: Next.js (auto-detected)
5. Build Command: npm run build (auto)
6. Output Directory: .next (auto)
7. Install Command: npm install (auto)

10.3 Configure Environment Variables (Website)

In Vercel Project Settings → Environment Variables:

```
NEXT_PUBLIC_API_URL=https://api.shopease.com/api
NEXT_PUBLIC_SITE_URL=https://shopease.com
```

10.4 Deploy Website

1. Click "Deploy"
2. Wait for build (~3-5 minutes for Next.js)
3. Vercel URL: <https://ecommerce-website.vercel.app>
4. Test:
 - Homepage loads with hero + products
 - Product listing page loads and filters work
 - Product detail page loads
 - Login and register work
 - Add to cart works
 - Checkout completes (place a test order)
 - Admin dashboard shows the test order

10.5 Custom Domain (Website)

1. Vercel Project → Settings → Domains
2. Add: shopease.com
3. Add: www.shopease.com (Vercel auto-redirects to shopease.com)
4. Vercel provides:

Type: A	Name: @	Value: 76.76.21.21
Type: CNAME	Name: www	Value: cname.vercel-dns.com
5. Add in domain registrar DNS settings
6. SSL auto-provisioned by Vercel

11. Domain & DNS Configuration

11.1 DNS Records Summary

Add all these records in your domain registrar (Namecheap / GoDaddy):

Type	Name	Value	TTL
A	@	76.76.21.21 (Vercel)	Auto
CNAME	www	cname.vercel-dns.com	Auto
CNAME	admin	cname.vercel-dns.com	Auto
CNAME	api	<railway-app>.up.railway.app	Auto

11.2 Verify DNS Propagation

```
# Check from terminal (wait 5-30 minutes after adding records):
nslookup shopease.com
nslookup www.shopease.com
nslookup admin.shopease.com
nslookup api.shopease.com

# Or use online tool: dnschecker.org
```

11.3 Update CORS After Domain Setup

```
// backend src/app.js – update CORS with final domains:
app.use(cors({
  origin: [
    'https://shopease.com',
    'https://www.shopease.com',
    'https://admin.shopease.com',
  ],
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}))
```

12. SSL Certificate

Automatic SSL (All Platforms)

Platform	SSL Provider	Auto-Renewal
Vercel	Let's Encrypt	✓ Automatic
Railway	Let's Encrypt	✓ Automatic
MongoDB Atlas	AWS ACM	✓ Automatic

No manual SSL setup needed. All platforms provision and renew SSL certificates automatically when you add custom domains.

Verify SSL

```
# Test HTTPS is working:
curl -I https://shopease.com
curl -I https://api.shopease.com/api/products

# Expected: HTTP/2 200, with server headers
# Check certificate: lock icon in browser
```

13. Environment Variables Reference

13.1 Backend (Railway)

```
# — Server —
NODE_ENV=production
PORT=5000

# — Database —
MONGODB_URI=mongodb+srv://ecommerce_user:<password>@ecommerce-cluster.xxxxx.mongodb.net/ecommerce

# — JWT —
JWT_ACCESS_SECRET=<generate: node -e "console.log(require('crypto').randomBytes(64).toString('hex'))">
JWT_REFRESH_SECRET=<generate: different from access secret>
JWT_ACCESS_EXPIRE=15m
JWT_REFRESH_EXPIRE=7d

# — Cloudinary —
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=123456789012345
CLOUDINARY_API_SECRET=xxxxxxxxxxxxxxxxxxxxxx

# — Email (SendGrid) —
EMAIL_HOST=smtp.sendgrid.net
EMAIL_PORT=587
EMAIL_USER=apikey
EMAIL_PASS=SG.xxxxxxxxxxxxxxxxxxxxxxxxxxxx
EMAIL_FROM=ShopEase <noreply@shopease.com>

# — Frontend URLs (CORS whitelist) —
CLIENT_URL=https://shopease.com
DASHBOARD_URL=https://admin.shopease.com

# — Cookie —
COOKIE_SECURE=true

# — App —
FREE_SHIPPING_THRESHOLD=499
SHIPPING_CHARGE=99
TAX_RATE=0.18
LOW_STOCK_THRESHOLD=10
```

13.2 Dashboard (Vercel)

```
VITE_API_URL=https://api.shopease.com/api
```

13.3 Website (Vercel)

```
NEXT_PUBLIC_API_URL=https://api.shopease.com/api
NEXT_PUBLIC_SITE_URL=https://shopease.com
```

14. Post-Deployment Verification

14.1 Backend API Tests

```
# 1. Health check
GET https://api.shopease.com/api/products
Expected: 200 OK with products array

# 2. Auth flow
POST https://api.shopease.com/api/auth/login
Body: { "email": "admin@shopease.com", "password": "Admin@123456" }
Expected: 200 OK, cookies set

# 3. Protected route
GET https://api.shopease.com/api/auth/me
Expected: 200 with user data (using session cookie from step 2)

# 4. CORS check
Open browser console on https://shopease.com
fetch('https://api.shopease.com/api/products').then(r => r.json()).then(console.log)
Expected: No CORS error, products returned
```

14.2 Website Checks

```
[ ] Homepage loads in < 3 seconds
[ ] All product images load (Cloudinary URLs working)
[ ] Search returns results
[ ] Product detail page loads with correct data
[ ] Register new account → welcome email received
[ ] Login with new account
[ ] Add products to cart
[ ] Apply coupon code (WELCOME20)
[ ] Complete checkout with COD
[ ] Order confirmation email received
[ ] Order visible in Profile → Orders
```

14.3 Dashboard Checks

```
[ ] Admin login works
[ ] Dashboard stats show correct numbers
[ ] Revenue chart renders
[ ] Product list loads
[ ] Add a test product with images → verify on website
[ ] View the test order from website
[ ] Update order status → email sent to customer
[ ] Category management works
[ ] Coupon management works
```


14.4 Lighthouse Scores (Target ≥ 90)

```
# Run Lighthouse in Chrome DevTools (Incognito mode):
# DevTools → Lighthouse → Analyze page load

Target Scores:
  Performance:    ≥ 90
  Accessibility: ≥ 90
  Best Practices: ≥ 90
  SEO:            ≥ 95

# Run for all key pages:
https://shopease.com           (Home)
https://shopease.com/products  (Product Listing)
https://shopease.com/products/<slug> (Product Detail)
```

14.5 Security Verification

```
# Check security headers (via securityheaders.com):
# Enter: https://api.shopease.com
# Expected rating: A or B

# Test rate limiting:
# Make 110 requests in 1 minute to any endpoint
# Expected: 429 Too Many Requests after 100

# Check cookies (Chrome DevTools → Application → Cookies):
# accessToken: HttpOnly=true, Secure=true, SameSite=Strict
# refreshToken: HttpOnly=true, Secure=true, SameSite=Strict
```

15. Rollback Procedure

15.1 Vercel Rollback (Website / Dashboard)

1. Vercel Dashboard → Select Project
2. Deployments tab → find last working deployment
3. Click "..." → Promote to Production
4. Rollback takes ~30 seconds

Alternatively via CLI:
npx vercel rollback

15.2 Railway Rollback (Backend)

1. Railway Dashboard → Select Service
2. Deployments tab → find last working deployment
3. Click on deployment → "Rollback to this deploy"
4. Confirm rollback

Alternatively:

```
git revert <commit-hash>
git push origin main
(triggers new deployment with reverted code)
```

15.3 Database Rollback

MongoDB Atlas does NOT support one-click rollback.

Prevention: Always take a backup before major migrations.

Manual restore from backup:

1. MongoDB Atlas → Database → ... → Load Sample Data
OR restore from backup file:
2. `mongorestore --uri="<MONGODB_URI>" --db=ecommerce ./backup/`

Create backup before deployment:

```
mongodump --uri="<MONGODB_URI>" --db=ecommerce --out=./backup/
```

15.4 Emergency Procedures

Issue: API is down (5xx errors)

1. Check Railway logs for error message
2. Fix code → push to main → auto-redeploy
3. If DB connection: check MongoDB Atlas status page
4. If env var issue: update in Railway → redeploy

Issue: Website shows blank page

1. Check browser console for errors
2. Verify NEXT_PUBLIC_API_URL in Vercel env vars
3. Check if API is reachable
4. Rollback to last working Vercel deployment

Issue: Login not working (cookie not set)

1. Verify COOKIE_SECURE=true in production
2. Verify CLIENT_URL matches exact domain (https://shopease.com)
3. Verify CORS allows the frontend domain
4. Check Chrome DevTools → Network → response Set-Cookie header

Issue: Emails not sending

1. Check Railway logs for email errors
2. Verify SendGrid API key is correct
3. Check SendGrid Activity Feed for bounces/blocks
4. Verify sender is verified in SendGrid

16. CI/CD Pipeline

16.1 GitHub Actions Workflow

```

# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  # — Backend: Lint + Test —————
  backend-check:
    name: Backend Lint & Build Check
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: backend/package-lock.json

      - name: Install dependencies
        run: npm ci

      - name: Run ESLint
        run: npm run lint

      - name: Check startup (no crash)
        run: |
          timeout 10 node server.js || true
        env:
          NODE_ENV: test
          PORT: 5001
          MONGODB_URI: ${ secrets.MONGODB_URI_TEST }
          JWT_ACCESS_SECRET: test-secret-for-ci
          JWT_REFRESH_SECRET: test-refresh-secret-for-ci

  # — Dashboard: Build Check —————
  dashboard-build:
    name: Dashboard Build Check
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: dashboard
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:

```

```

    node-version: '20'
    cache: 'npm'
    cache-dependency-path: dashboard/package-lock.json

  - name: Install dependencies
    run: npm ci

  - name: Build
    run: npm run build
    env:
      VITE_API_URL: https://api.shopease.com/api

# — Website: Build Check —————
website-build:
  name: Website Build Check
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: website
  steps:
    - uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '20'
        cache: 'npm'
        cache-dependency-path: website/package-lock.json

    - name: Install dependencies
      run: npm ci

    - name: Build
      run: npm run build
      env:
        NEXT_PUBLIC_API_URL: https://api.shopease.com/api
        NEXT_PUBLIC_SITE_URL: https://shopease.com

# — Deploy (after all checks pass) —————
deploy:
  name: Deploy All Services
  needs: [backend-check, dashboard-build, website-build]
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Trigger Railway Deploy (Backend)
      run: |
        curl -X POST "${{ secrets.RAILWAY_DEPLOY_HOOK }}"

    - name: Vercel auto-deploys website + dashboard
      run: echo "Vercel detects push to main and deploys automatically"

```

16.2 GitHub Secrets Required

Go to: GitHub Repo → Settings → Secrets and variables → Actions

Add:

RAILWAY_DEPLOY_HOOK ← from Railway Project → Settings → Deploy Hook
MONGODB_URI_TEST ← test database connection string (optional)

16.3 Branch Protection Rules

Go to: GitHub → Settings → Branches → Add rule

Branch: main

Rules:

- ✓ Require a pull request before merging
- ✓ Require at least 1 approval
- ✓ Require status checks to pass before merging
 - backend-check
 - dashboard-build
 - website-build
- ✓ Do not allow bypassing the above settings

16.4 Deployment Status Badges (README.md)

```
![Backend](https://img.shields.io/badge/backend-railway-success)
![Website](https://img.shields.io/badge/website-vercel-success)
![Dashboard](https://img.shields.io/badge/dashboard-vercel-success)
```

Deployment Summary

Service	URL	Platform	Status Check
Backend API	https://api.shopease.com	Railway	GET /api/products
Client Website	https://shopease.com	Vercel	Homepage loads
Admin Dashboard	https://admin.shopease.com	Vercel	Login page loads
Database	MongoDB Atlas (internal)	Atlas	Atlas dashboard
Images	Cloudinary CDN	Cloudinary	Media library
Email	SendGrid	SendGrid	Activity feed

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial Deployment Guide

Previous Phase → [Phase 7: Implementation Plan](#)
Next Phase → [Phase 9: Maintenance Guide](#)

Section 10: Phase 9: Maintenance & Operations Guide

Maintenance Guide

Project: Production-Ready E-Commerce Platform
Version: 1.0
Status: Planning
Date: July 2026
Phase: 9 of 9 — FINAL

Table of Contents

- 1. Overview
- 2. Monitoring Setup
- 3. Log Management
- 4. Database Maintenance
- 5. Backup Strategy
- 6. Security Maintenance
- 7. Performance Monitoring
- 8. Dependency Management
- 9. Bug Reporting & Tracking
- 10. Hotfix Procedure
- 11. Feature Release Process
- 12. Routine Maintenance Schedule
- 13. Scaling Guide
- 14. Incident Response
- 15. Future Enhancements Roadmap

1. Overview

This guide covers all ongoing maintenance tasks required to keep the ShopEase e-commerce platform running securely, performantly, and reliably after initial deployment.

Maintenance Principles

Principle	Description
Proactive	Monitor before problems occur, not after
Automated	Automate repetitive tasks (backups, alerts)
Documented	Log every change, incident, and resolution
Minimal Downtime	Use zero-downtime deployment strategies
Security First	Patch vulnerabilities within 24–48 hours

Maintenance Ownership

Area	Owner	Frequency
Bug fixes	Backend / Frontend Dev	As needed
Security patches	Full Stack / DevOps	Within 48 hours
Dependency updates	Backend Dev	Monthly
DB backups	Automated	Daily
Performance review	Full Stack	Weekly
Feature releases	Full team	Bi-weekly sprint
Cost review	Product Manager	Monthly

2. Monitoring Setup

2.1 Uptime Monitoring — UptimeRobot (Free)

Setup:

1. Create account → uptimerobot.com
2. Add Monitor → HTTP(s)
3. Add the following monitors:

Monitor 1: API Health

URL: <https://api.shopease.com/api/products>
Interval: Every 5 minutes
Alert: Email + SMS on downtime

Monitor 2: Website

URL: <https://shopease.com>
Interval: Every 5 minutes
Alert: Email on downtime

Monitor 3: Admin Dashboard

URL: <https://admin.shopease.com>
Interval: Every 5 minutes
Alert: Email on downtime

4. Create a public status page:

status.shopease.com
(Shows uptime history to customers if site is down)

2.2 Error Tracking — Sentry (Free Tier)

Setup:

1. Create account → sentry.io
2. Create Project: Node.js (Backend)
3. Create Project: Next.js (Website)
4. Create Project: React (Dashboard)

Backend Integration:

```
npm install @sentry/node
```

```
// In server.js – before all routes:
import * as Sentry from '@sentry/node'
Sentry.init({
  dsn: process.env.SENTRY_DSN_BACKEND,
  environment: process.env.NODE_ENV,
  tracesSampleRate: 0.1, // 10% of requests
})
```

```
// In error middleware – report to Sentry:
Sentry.captureException(err)
```

Website Integration:

```
npm install @sentry/nextjs
npx @sentry/wizard@latest -i nextjs
```

Dashboard Integration:

```
npm install @sentry/react
Sentry.init({ dsn: process.env.VITE_SENTRY_DSN })
```

Benefits:

- Real-time error alerts (email/Slack)
- Full stack traces
- Error frequency tracking
- User context on errors

2.3 Analytics — Vercel Analytics (Built-in)

Enable in Vercel Dashboard:

1. Website Project → Analytics tab → Enable

Tracks:

- Page views
- Core Web Vitals (LCP, FID, CLS)
- Top pages
- Geographic distribution
- Device breakdown

Cost: Free for Hobby plan

2.4 Railway Metrics (Built-in)

Railway Dashboard → Service → Metrics tab

Tracks:

- CPU usage
- Memory usage
- Network I/O
- Request volume

Set alerts:

Railway → Service → Settings → Notifications

Alert on: Deployment success/failure, crash restart

2.5 Add Health Check Endpoint

```
// backend/src/routes/health.routes.js
import express from 'express'
import mongoose from 'mongoose'

const router = express.Router()

router.get('/health', async (req, res) => {
  const dbStatus = mongoose.connection.readyState === 1 ? 'connected' : 'disconnected'

  res.status(200).json({
    status: 'ok',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    environment: process.env.NODE_ENV,
    database: dbStatus,
    memory: {
      used: Math.round(process.memoryUsage().heapUsed / 1024 / 1024) + 'MB',
      total: Math.round(process.memoryUsage().heapTotal / 1024 / 1024) + 'MB',
    },
  })
})

export default router

// Mount in app.js:
import healthRouter from './routes/health.routes.js'
app.use('/api', healthRouter)
```

3. Log Management

3.1 Logging Levels

```
// Use Morgan for HTTP request logs
// Use console levels for app logs:

console.error() // Critical errors (always log)
console.warn()  // Warnings (potential issues)
console.info()  // Important events (order placed, user registered)
console.log()   // Debug (development only, remove in production)
```

3.2 What to Log

```
// ✅ LOG these events:
- Server startup + port
- DB connection success / failure
- Failed login attempts (email, IP, timestamp)
- Order placed (orderId, userId, amount)
- Order status changed
- Admin actions (product created/deleted, user blocked)
- File upload success / failure
- Password reset requested
- Unhandled exceptions (via Sentry)

// ❌ NEVER LOG:
- Passwords (even hashed)
- JWT tokens
- Credit card numbers
- Full request bodies with sensitive fields
```

3.3 Viewing Logs in Production

```
# Railway logs:
railway logs           # last 100 lines
railway logs --tail    # live streaming
railway logs --filter "ERROR" # filter by keyword

# Vercel Function logs (for API routes):
vercel logs https://shopease.com # recent function logs

# MongoDB Atlas logs:
Atlas Dashboard → Database → Monitoring → Real Time
```

3.4 Log Retention

Platform	Retention	How to extend
Railway	7 days	Export to external service
Vercel	1 hour	Export to Datadog / LogDNA
Sentry	30 days (free)	Upgrade plan

4. Database Maintenance

4.1 Regular Checks

```
# Connect to MongoDB Atlas:
# Atlas → Database → Browse Collections

# Check collection sizes weekly:
db.orders.countDocuments()
db.products.countDocuments()
db.users.countDocuments()
db.reviews.countDocuments()

# Check slow queries (Atlas → Monitoring → Query Profiler):
# Enable: Atlas → Performance Advisor → Enable Profiler
# Look for queries > 100ms and add indexes
```

4.2 Index Maintenance

```
// Check existing indexes monthly:
db.products.getIndexes()
db.orders.getIndexes()

// Remove unused indexes (Atlas Performance Advisor flags these):
db.collectionName.dropIndex('index_name')

// Rebuild indexes if fragmented (do during low-traffic hours):
db.products.reIndex()
```

4.3 Data Cleanup Tasks

```
// Run monthly via a script or MongoDB Atlas Scheduled Trigger:

// 1. Remove expired coupons (older than 6 months)
await Coupon.deleteMany({
  expiresAt: { $lt: new Date(Date.now() - 180 * 24 * 60 * 60 * 1000) },
  isActive: false,
})

// 2. Clear abandoned carts (untouched for 30 days)
await Cart.deleteMany({
  items: { $ne: [] },
  updatedAt: { $lt: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) },
})

// 3. Clear expired password reset tokens
await User.updateMany(
  { resetPasswordExpire: { $lt: new Date() } },
  { $unset: { resetPasswordToken: '', resetPasswordExpire: '' } }
)

// 4. Archive old delivered/cancelled orders (> 1 year)
// Move to an 'archived_orders' collection
```

4.4 MongoDB Atlas Performance Advisor

Atlas Dashboard → Database → Performance Advisor

Reviews:

- Slow query suggestions
- Missing index recommendations
- Unused index warnings

Action: Review weekly, implement recommendations in next sprint.

5. Backup Strategy

5.1 MongoDB Atlas Automatic Backups

Atlas Free (M0): No automatic backups – manual only
Atlas M10+: Continuous backup (point-in-time restore)

Recommendation: Upgrade to M10 for production (\$57/month)
- Continuous backup (restore to any point in last 24 hours)
- Snapshots retained for 7 days (daily), 4 weeks (weekly)

Enable:
Atlas → Backup → Enable Backup for Cluster

5.2 Manual Backup Script

```
#!/bin/bash
# backup.sh – run via cron or GitHub Actions

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="./backups/$DATE"
MONGODB_URI="mongodb+srv://..."

echo "Starting backup: $DATE"

# Create backup
mongodump \
  --uri="$MONGODB_URI" \
  --db=ecommerce \
  --out="$BACKUP_DIR" \
  --gzip

# Upload to cloud storage (e.g., AWS S3 or Cloudinary)
# aws s3 cp "$BACKUP_DIR" s3://shopease-backups/$DATE --recursive

echo "Backup complete: $BACKUP_DIR"

# Cleanup backups older than 30 days
find ./backups -type d -mtime +30 -exec rm -rf {} +
```

5.3 Automated Backup via GitHub Actions


```
# .github/workflows/backup.yml
name: Daily Database Backup

on:
  schedule:
    - cron: '0 1 * * *'    # Every day at 1:00 AM UTC (6:30 AM IST)

jobs:
  backup:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install mongodump
        run: |
          wget -qO - https://www.mongodb.org/static/pgp/server-7.0.asc | sudo apt-key add -
          echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/7.0 mu
          sudo apt-get update && sudo apt-get install -y mongodb-database-tools

      - name: Run backup
        env:
          MONGODB_URI: ${ secrets.MONGODB_URI }
        run: bash scripts/backup.sh

      - name: Upload backup artifact
        uses: actions/upload-artifact@v4
        with:
          name: db-backup-${ github.run_id }
          path: ./backups/
          retention-days: 30
```

5.4 Backup Retention Policy

Backup Type	Retention	Storage
Daily automated	30 days	GitHub Artifacts
Pre-deployment	90 days	GitHub Artifacts
Monthly	12 months	External cloud storage

6. Security Maintenance

6.1 Vulnerability Scanning

```
# Run weekly in each app directory:
cd backend    && npm audit
cd dashboard && npm audit
cd website    && npm audit

# Auto-fix low/moderate vulnerabilities:
npm audit fix

# Check what will be changed before fixing:
npm audit fix --dry-run

# For breaking-change fixes (review carefully):
npm audit fix --force
```

6.2 Dependency Security Policy

CRITICAL vulnerability:	Patch within 24 hours
HIGH vulnerability:	Patch within 48 hours
MODERATE vulnerability:	Patch within 1 week
LOW vulnerability:	Patch in next monthly update

6.3 GitHub Dependabot (Automated)

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: npm
    directory: /backend
    schedule:
      interval: weekly
      day: monday
    open-pull-requests-limit: 10
    labels: [dependencies, backend]

  - package-ecosystem: npm
    directory: /dashboard
    schedule:
      interval: weekly
      day: monday
    labels: [dependencies, dashboard]

  - package-ecosystem: npm
    directory: /website
    schedule:
      interval: weekly
      day: monday
    labels: [dependencies, website]

  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: monthly
    labels: [dependencies, ci]
```

6.4 JWT Secret Rotation

```
# Rotate JWT secrets every 90 days:

# Step 1: Generate new secrets
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"

# Step 2: Update in Railway environment variables
# Railway → Project → Variables → update JWT_ACCESS_SECRET + JWT_REFRESH_SECRET

# Step 3: Redeploy (Railway auto-redeploys on env var change)

# Note: All current user sessions will be invalidated.
# Users will need to log in again – this is expected and secure.
# Schedule during low-traffic hours (e.g., 3 AM IST).
```

6.5 Security Audit Checklist (Monthly)

```
[ ] npm audit – zero critical/high vulnerabilities
[ ] Check Sentry for new error patterns (possible attacks)
[ ] Review Railway logs for suspicious activity (many 401s, unusual IPs)
[ ] Check MongoDB Atlas access logs (Atlas → Security → Access History)
[ ] Verify CORS still only allows whitelisted origins
[ ] Verify rate limiting is working (test with curl)
[ ] Review admin accounts – remove inactive ones
[ ] Check Cloudinary – no unauthorized uploads
[ ] Verify SSL certificates are valid (check expiry in browser)
[ ] Review SendGrid – no spike in bounces (possible list compromise)
```

7. Performance Monitoring

7.1 Core Web Vitals Tracking

Run monthly via Google PageSpeed Insights:

<https://pagespeed.web.dev/>

Target URLs:

<https://shopease.com>

<https://shopease.com/products>

<https://shopease.com/products/<best-selling-slug>>

Target Scores:

LCP (Largest Contentful Paint): < 2.5s 

FID (First Input Delay): < 100ms 

CLS (Cumulative Layout Shift): < 0.1 

TTFB (Time to First Byte): < 600ms 

Document scores in a spreadsheet monthly.

If any score drops below target → investigate immediately.

7.2 API Response Time Monitoring

```
# Test key API endpoints monthly:
# Using curl with timing:

curl -o /dev/null -s -w "%{time_total}s\n" \
  https://api.shopease.com/api/products

curl -o /dev/null -s -w "%{time_total}s\n" \
  "https://api.shopease.com/api/products?search=headphones&category=electronics"

# Targets:
# GET /products (no filters): < 150ms
# GET /products (with filters): < 250ms
# POST /orders: < 500ms
# GET /analytics/dashboard: < 800ms
```

7.3 MongoDB Query Performance

```
// Enable slow query logging (> 100ms) in MongoDB Atlas:
// Atlas → Database → Monitoring → Real Time Performance Panel

// Check for slow queries monthly:
// Atlas → Performance Advisor → Slow Queries

// Common fixes:
// - Missing index → add index
// - N+1 query → use populate or aggregation
// - Large result set → add pagination or lean()
// - Unoptimized aggregation → add $match early in pipeline
```

7.4 Bundle Size Monitoring

```
# Check website bundle size after each major release:
cd website
npm run build

# Review .next/analyze output
# Install bundle analyzer:
npm install --save-dev @next/bundle-analyzer

# In next.config.js:
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
})
module.exports = withBundleAnalyzer(nextConfig)

# Run analysis:
ANALYZE=true npm run build

# Dashboard bundle:
cd dashboard
npm run build
# Check dist/ folder sizes – total JS should be < 500KB gzipped
```

8. Dependency Management

8.1 Monthly Update Routine

```

# Step 1: Check outdated packages
cd backend    && npm outdated
cd dashboard && npm outdated
cd website    && npm outdated

# Step 2: Update minor + patch versions (safe)
npm update

# Step 3: Review major version updates manually
# Use npm-check-updates for major bumps:
npx ncu -u --target minor    # minor updates only
npx ncu -u                   # all updates (review changelog first!)

# Step 4: Test after updates
npm run lint
npm run build (dashboard + website)
node server.js (backend – verify no crash)

# Step 5: Run integration tests

# Step 6: Commit and push if all tests pass
git add package.json package-lock.json
git commit -m "chore(deps): update dependencies - $(date +%Y-%m)"
git push origin develop

```

8.2 Node.js Version Updates

Node.js LTS release schedule:

- New LTS: October each year
- Active support: 18 months
- Maintenance: 12 months

Action:

- Update to new LTS within 1 month of release
- Update in Railway: Settings → Environment → NODE_VERSION=22
- Update in package.json engines field
- Test locally before deploying

8.3 Breaking Changes Protocol

Before updating a major version:

1. Read the package CHANGELOG / migration guide
2. Check GitHub issues for reported breaking changes
3. Test in a feature branch first
4. Deploy to staging (if available) and test
5. Only merge to main after full verification
6. Document the update in the changelog

9. Bug Reporting & Tracking

9.1 Bug Report Template

```
## Bug Report

**Title:** [Short description of the bug]

**Severity:** Critical / High / Medium / Low

**Environment:** Production / Staging / Local

**Steps to Reproduce:**
1. Go to ...
2. Click on ...
3. See error

**Expected Behavior:**
What should happen

**Actual Behavior:**
What actually happens

**Screenshots / Logs:**
[Attach screenshot or paste error from Sentry/Railway logs]

**Browser / Device:**
Chrome 125 / iPhone 14 / Windows 11

**Reported By:** [Name]
**Date:** [Date]
```

9.2 Severity Levels

Severity	Description	Response Time	Examples
Critical	System down or data loss	< 2 hours	API down, checkout broken, login broken
High	Major feature broken	< 24 hours	Orders not sending email, images not loading
Medium	Minor feature broken	< 72 hours	Filter not working, pagination off
Low	Cosmetic / minor UX	Next sprint	Alignment issue, typo

9.3 GitHub Issues Workflow

Labels to create in GitHub:

-  critical – fix immediately
-  high – fix this week
-  medium – fix this sprint
-  low – backlog
-  bug – confirmed bug
-  enhancement – new feature request
-  docs – documentation update
-  security – security vulnerability

Workflow:

Bug reported → Create GitHub Issue → Label → Assign → Fix branch
→ PR → Review → Merge to develop → Deploy to staging → Verify
→ Merge to main → Deploy → Close issue

10. Hotfix Procedure

For **Critical** bugs that need immediate fix in production:

```
# Step 1: Create hotfix branch from main (NOT develop)
git checkout main
git pull origin main
git checkout -b hotfix/checkout-payment-crash

# Step 2: Fix the bug
# Make minimal changes – ONLY what is needed to fix the bug

# Step 3: Test the fix locally
npm run dev # verify fix works
npm run lint # verify no lint errors

# Step 4: Commit with clear message
git add .
git commit -m "fix(checkout): resolve payment calculation crash on zero-stock item"

# Step 5: Push and create PR to main
git push origin hotfix/checkout-payment-crash
# Create PR → merge to main (emergency – 1 approval needed)

# Step 6: After merge → auto-deployed to production

# Step 7: Verify fix in production (test the exact scenario that was broken)

# Step 8: Merge hotfix back to develop (so develop is not behind)
git checkout develop
git merge hotfix/checkout-payment-crash
git push origin develop

# Step 9: Delete hotfix branch
git branch -d hotfix/checkout-payment-crash
git push origin --delete hotfix/checkout-payment-crash

# Step 10: Document the incident (see Incident Response section)
```

11. Feature Release Process

11.1 Release Cycle

```
Every 2 weeks (end of sprint):
1. All sprint features merged to develop
2. QA testing on develop branch
3. Create release branch: release/v1.1.0
4. Final testing + bug fixes on release branch
5. Merge release → main (triggers production deploy)
6. Merge release → develop (keep in sync)
7. Tag the release: git tag v1.1.0
8. Write release notes in GitHub Releases
```

11.2 Semantic Versioning

Format: MAJOR.MINOR.PATCH

MAJOR: Breaking changes (v2.0.0 – new architecture)

MINOR: New features, backward compatible (v1.1.0 – wishlist added)

PATCH: Bug fixes (v1.0.1 – cart calculation fix)

Examples:

v1.0.0 – Initial production launch

v1.0.1 – Hotfix: checkout email not sending

v1.1.0 – Feature: online payment gateway

v1.2.0 – Feature: product reviews + ratings

v2.0.0 – Major: multi-vendor marketplace

11.3 Release Notes Template

ShopEase v1.1.0 – Release Notes

Release Date: August 2026

Type: Minor Release

🌟 New Features

- Online payment via Razorpay
- Product variant stock tracking
- Admin: bulk product status update

🐛 Bug Fixes

- Fixed cart total calculation with percentage coupon
- Fixed mobile menu not closing after navigation
- Fixed product slug collision on rename

⚡ Performance

- Improved product listing API response by 40% (added compound index)
- Reduced dashboard bundle size by 22% (code splitting)

🔒 Security

- Updated bcryptjs to v3.0 (security patch)
- Added additional rate limiting on auth endpoints

📖 Documentation

- Updated API docs for new payment endpoints
- Added Razorpay integration guide

12. Routine Maintenance Schedule

Daily (Automated)

- ✓ UptimeRobot checks API + Website + Dashboard every 5 min
- ✓ Database backup runs at 1:00 AM UTC
- ✓ Sentry monitors for new errors
- ✓ Railway monitors CPU and memory

Weekly (Manual — Every Monday)

- [] Review UptimeRobot report (check uptime %)
- [] Review Sentry – any new errors this week?
- [] Review Railway logs – any warnings?
- [] Check API response times (key endpoints)
- [] Review Core Web Vitals in Vercel Analytics
- [] Check MongoDB Atlas Performance Advisor
- [] Check low stock alerts in dashboard
- [] Review any new GitHub issues / bug reports
- [] Review Dependabot PRs (merge safe updates)
- [] Check email delivery rate in SendGrid

Monthly (First Monday of month)

- [] Run npm audit in all 3 apps – fix vulnerabilities
- [] Review and update dependencies (minor/patch)
- [] Run Lighthouse scores on all key pages
- [] Review MongoDB Atlas storage usage
- [] Review Cloudinary storage + bandwidth usage
- [] Review Railway CPU/memory – scale if needed
- [] Review error budget (uptime SLA)
- [] Rotate JWT secrets (every 90 days)
- [] Review admin user accounts – remove inactive
- [] Review coupon list – deactivate expired ones
- [] Test password reset flow end-to-end
- [] Test order flow end-to-end (place test order)
- [] Check SSL certificate expiry dates
- [] Review SendGrid bounce/spam rates
- [] Document any incidents from the past month
- [] Update CHANGELOG.md with all changes
- [] Cost review: Railway + Vercel + Atlas + Cloudinary bills

Quarterly

- [] Full security audit (see Section 6.5)
- [] Major dependency updates (review changelogs)
- [] Node.js version upgrade (if new LTS released)
- [] Review and update documentation
- [] Performance baseline review + optimization sprint
- [] Database index analysis and optimization
- [] Review and archive old orders (> 1 year)
- [] Review feature roadmap and prioritize next quarter
- [] Team retrospective and process improvements

13. Scaling Guide

13.1 When to Scale

Metric	Threshold	Action
Railway CPU	> 80% consistently	Upgrade Railway plan
Railway Memory	> 80% consistently	Upgrade Railway plan
API response time	> 500ms average	Optimize queries or scale
MongoDB storage	> 80% of tier	Upgrade Atlas tier
Cloudinary bandwidth	> 80% of quota	Upgrade Cloudinary plan
Concurrent users	> 500 simultaneous	Add Redis caching

13.2 Vertical Scaling (Railway)

Railway → Service → Settings → Resources

Increase: CPU (0.5 → 1 → 2 vCPU)

Increase: Memory (512MB → 1GB → 2GB)

Cost impact:

\$5/month → 0.5 vCPU, 512MB RAM

\$10/month → 1 vCPU, 1GB RAM

\$20/month → 2 vCPU, 2GB RAM

13.3 Horizontal Scaling (Future)

When vertical scaling is not enough:

1. Add Redis for session caching + rate limiting store
2. Add Redis for API response caching (product lists)
3. Use Railway's horizontal scaling (multiple instances)
4. Add a load balancer (Railway handles this automatically)
5. Use MongoDB Atlas M30+ with sharding for very large datasets

13.4 Redis Caching (Future Implementation)

```
// Cache frequently-accessed, rarely-changing data:
// - Product listing (cache 60 seconds)
// - Category list (cache 5 minutes)
// - Featured products (cache 5 minutes)
// - Dashboard stats (cache 5 minutes)

// Implementation:
npm install ioredis

// Cache middleware pattern:
const cacheMiddleware = (duration) => async (req, res, next) => {
  const key = `cache:${req.originalUrl}`
  const cached = await redis.get(key)
  if (cached) return res.json(JSON.parse(cached))
  res.sendResponse = res.json.bind(res)
  res.json = (data) => {
    redis.setex(key, duration, JSON.stringify(data))
    res.sendResponse(data)
  }
  next()
}

// Usage:
router.get('/products', cacheMiddleware(60), getProducts)
router.get('/categories', cacheMiddleware(300), getCategories)
```

13.5 MongoDB Atlas Upgrade Path

M0 (Free):	512MB storage, shared cluster – Development only
M2 (\$9/mo):	2GB storage, shared – Early production
M5 (\$25/mo):	5GB storage, shared – Small production
M10 (\$57/mo):	10GB storage, dedicated – Production recommended
M20 (\$140/mo):	20GB storage, dedicated – Growing business
M30 (\$230/mo):	40GB storage, dedicated + auto-scaling

14. Incident Response

14.1 Incident Severity Levels

Level	Impact	Response	Examples
P0 — Critical	Full outage	Immediate (< 30 min)	API down, DB unreachable
P1 — High	Core feature broken	< 2 hours	Checkout broken, login broken
P2 — Medium	Non-core broken	< 24 hours	Email not sending, search broken
P3 — Low	Minor issue	Next sprint	UI bug, slow page

14.2 Incident Response Steps

DETECT:

- [] Alert received (UptimeRobot / Sentry / User report)
- [] Verify the issue (reproduce manually)
- [] Determine severity level

COMMUNICATE:

- [] Notify team via WhatsApp/Slack group
- [] Update public status page (status.shopease.com)
- [] If P0/P1: Post status: "We are aware of the issue and working on a fix"

INVESTIGATE:

- [] Check Railway logs (most common source of truth)
- [] Check Sentry for error details and stack trace
- [] Check MongoDB Atlas status page (mongodbstatus.com)
- [] Check Cloudinary status (status.cloudinary.com)
- [] Identify root cause

RESOLVE:

- [] Apply fix (hotfix procedure if code change needed)
- [] Verify fix in production
- [] Monitor for 30 minutes after fix

COMMUNICATE (POST-FIX):

- [] Update status page: "Issue resolved"
- [] Notify affected users (if significant)
- [] Post-mortem within 24 hours (for P0/P1)

DOCUMENT:

- [] Write incident report (template below)
- [] Add to incident log
- [] Update runbook with new learnings

14.3 Incident Report Template

Incident Report – [Date]

Incident ID: INC-2026-001

Severity: P1 – High

Status: Resolved

Timeline

Time (IST)	Event
14:32	UptimeRobot alert – API returning 500 errors
14:35	Team notified via WhatsApp
14:38	Status page updated: "Investigating issue"
14:45	Root cause identified: MongoDB connection pool exhausted
15:00	Fix deployed: increased connection pool size
15:05	API returning 200 – issue resolved
15:35	30-minute monitoring period complete
15:36	Status page updated: "Resolved"

Root Cause

A traffic spike caused MongoDB connection pool (default 100) to be exhausted. All new requests were queued and timing out after 30 seconds, returning 500 errors.

Impact

- Duration: 33 minutes
- Affected: All API endpoints
- Users impacted: ~45 customers attempted checkout during outage

Resolution

Updated MongoDB connection string:

maxPoolSize: 100 → 200
socketTimeoutMS: 30000 → 45000

Prevention

- Add MongoDB connection pool monitoring alert
- Implement connection pool metrics in health endpoint
- Consider Redis caching to reduce DB load during spikes

Action Items

- [] Add connection pool alert in Railway (threshold: > 80% pool usage)
- [] Add /api/health to include DB connection pool stats
- [] Schedule Redis caching implementation for v1.1

15. Future Enhancements Roadmap

v1.1 — Payment & Variants (2 months after launch)

- [] Online payment integration (Razorpay)
 - Payment gateway setup
 - Webhook for payment success/failure
 - Payment status update flow
 - Refund handling
- [] Product variant stock tracking
 - Stock per variant (not just total)
 - Low stock alert per variant
- [] Admin: Export to CSV
 - Export orders list
 - Export customers list
- [] Admin: Invoice PDF
 - Generate PDF invoice for each order
 - Email invoice on order delivery

v1.2 — Reviews & Search (3 months after launch)

- [] Advanced search
 - Elasticsearch or MongoDB Atlas Search
 - Fuzzy matching, typo tolerance
 - Relevance scoring
- [] Review images
 - Customers can upload photos with reviews
- [] Review helpfulness voting
 - "Was this helpful?" up/down vote
- [] Admin: Review moderation
 - Approve / reject reviews before publishing

v1.3 — Marketing (4 months after launch)

- [] Email marketing
 - Abandoned cart reminder email (after 2 hours)
 - Back in stock notification
 - Price drop alert for wishlisted products
 - Weekly newsletter with featured products
- [] Promotional banners management
 - Admin can create/edit/schedule banners
 - Homepage banners from dashboard
- [] Flash sale support
 - Time-limited price discount on products
 - Sale countdown timer on product card

v2.0 — Platform Expansion (6+ months)

- [] Multi-vendor marketplace
 - Seller registration and approval
 - Seller dashboard (mini admin)
 - Commission management
 - Seller payouts
- [] Mobile apps
 - React Native customer app (iOS + Android)
 - Push notifications for orders
- [] AI features
 - Product recommendations (collaborative filtering)
 - Smart search suggestions
 - Auto-tagging for products
- [] Loyalty program
 - Points earned on purchases
 - Redeem points for discounts
 - Tier system (Silver/Gold/Platinum)
- [] Multi-language support
 - English + Hindi + Gujarati
 - i18n with next-intl
- [] Multi-currency support
 - INR (default), USD, AED
 - Real-time exchange rates

Maintenance Contacts

Role	Responsibility	Contact
Backend Lead	API, DB, infrastructure issues	backend@shopease.com
Frontend Lead	Website, Dashboard issues	frontend@shopease.com
DevOps	Deployment, CI/CD, scaling	devops@shopease.com
Product Manager	Feature prioritization, roadmap	pm@shopease.com

External Support Contacts

Service	Support URL	Priority Support
Railway	railway.app/help	Discord community
Vercel	vercel.com/support	Support ticket
MongoDB Atlas	mongodb.com/support	Atlas dashboard
Cloudinary	cloudinary.com/support	Support ticket
SendGrid	sendgrid.com/support	Support ticket
Sentry	sentry.io/support	In-app support

Document History

Version	Date	Author	Notes
1.0	July 2026	Team	Initial Maintenance Guide — Final Phase

Previous Phase → [Phase 8: Deployment Guide](#)

Documentation Complete

All 9 phases of documentation have been completed:

#	Document	File
1	Product Requirements Document	phase-1-PRD.md
2	Technical Requirements Document	phase-2-TRD.md
3	Application Flow	phase-3-AppFlow.md
4	UI/UX Brief	phase-4-UIUXBrief.md
5	Backend Schema	phase-5-Schema.md
6	API Documentation	phase-6-APIDocs.md
7	Implementation Plan	phase-7-ImplementationPlan.md
8	Deployment Guide	phase-8-DeploymentGuide.md
9	Maintenance Guide	phase-9-MaintenanceGuide.md

Next Step → Begin Stage 1: Project Setup (Phase 7 — Implementation Plan)

Section 11: Product Catalog & Taxonomy

Reference

E-Commerce Product Categories Master List

Category	Description
 New Arrivals	Recently launched products and the latest additions to the store.
 Trending Products	Popular products that are currently in high demand.
 Best Sellers	Top-selling products based on customer purchases.
 Deals & Offers	Discounted products, seasonal promotions, and limited-time offers.
 Flash Sale	Time-limited promotional products with special pricing.

Electronics

Subcategory	Description
Smartphones	Android and iPhone devices from various brands.
Tablets	Android, iPad, and Windows tablets.
Laptops	Gaming, business, student, and ultrabook laptops.
Desktop Computers	Personal computers and all-in-one desktops.
Computer Components	CPUs, GPUs, RAM, SSDs, motherboards, and power supplies.
Monitors	Gaming, office, and professional displays.
Printers & Scanners	Home and office printing solutions.
Networking	Wi-Fi routers, switches, extenders, and modems.
Storage Devices	External drives, SSDs, HDDs, USB drives, and memory cards.
Smart Home	Smart lights, cameras, locks, plugs, and assistants.

Mobile Accessories

Subcategory	Description
Phone Cases	Protective covers for smartphones.
Screen Protectors	Tempered glass and film protectors.
Chargers	Wall chargers, wireless chargers, and car chargers.
Power Banks	Portable battery charging solutions.
Cables	USB-C, Lightning, Micro-USB, and HDMI cables.
Earbuds & Headphones	Wireless and wired audio devices.
Smartwatches	Fitness and lifestyle smartwatches.
Phone Holders	Vehicle mounts and desktop stands.

Fashion

Men

T-Shirts
Shirts
Jeans
Trousers
Shorts
Jackets
Hoodies
Suits
Ethnic Wear
Footwear
Watches
Wallets
Belts

Woman

Dresses
Tops
Kurtis
Sarees
Jeans
Leggings
Skirts
Handbags
Jewelry
Footwear
Watches

Kids

Boys Clothing
Girls Clothing

Baby Clothing
School Uniforms
Baby Accessories

Beauty & Personal Care

Skincare
Hair Care
Makeup
Fragrances
Men's Grooming
Bath & Body
Personal Hygiene
Beauty Tools

Home & Kitchen

Furniture
Kitchen Appliances
Cookware
Dinnerware
Storage Solutions
Home Decor
Lighting
Curtains
Bedding
Cleaning Supplies

Grocery & Essentials

Fruits & Vegetables
Dairy Products
Snacks
Beverages
Rice & Grains
Spices
Oils

Bakery
Frozen Foods
Organic Foods

Health & Wellness

Vitamins & Supplements
Medical Equipment
Personal Care
Fitness Nutrition
First Aid Supplies
Wellness Products

Sports & Fitness

Gym Equipment
Yoga Equipment
Cricket
Football
Basketball
Badminton
Cycling
Running
Outdoor Sports
Fitness Accessories

Books & Stationery

Fiction
Non-Fiction
Academic Books
Children's Books
Notebooks
Office Supplies
Art Supplies
School Essentials

Toys & Games

Educational Toys
Action Figures
Board Games
Puzzles
Outdoor Toys
RC Toys
Dolls
Building Blocks

Automotive

Car Accessories
Bike Accessories
Helmets
Car Care
Engine Oil
Tires
Vehicle Electronics

Pet Supplies

Dog Food
Cat Food
Bird Supplies
Fish Supplies
Pet Grooming
Pet Toys
Pet Beds

Office Supplies

Office Furniture
Office Electronics
Printers

Paper Products
Writing Instruments
Office Organization

Garden & Outdoor

Plants
Gardening Tools
Outdoor Furniture
BBQ Equipment
Outdoor Lighting
Garden Decor

Jewelry & Accessories

Rings
Necklaces
Earrings
Bracelets
Watches
Sunglasses
Fashion Accessories

Baby Products

Baby Clothing
Diapers
Feeding Essentials
Baby Toys
Baby Care
Baby Furniture
Strollers

Digital Products

Software
E-books
Online Courses
Digital Gift Cards
Digital Downloads

Gift Shop

Gift Shop
Birthday Gifts
Anniversary Gifts
Wedding Gifts
Festival Gifts
Corporate Gifts
Personalized Gifts

Seasonal Collections

Summer Collection
Winter Collection
Festival Specials
Holiday Offers
Back-to-School
Wedding Collection